A Secure and Auditable Framework for Legal Data Management: A Hybrid Approach Using Hyperledger Fabric and IPFS

Golam Bari Fardeen[a], Namare Shakib Angkon[a], Syed Zayan Anwar[a], Md. Masum Mizi[a], Shinthia Rahman[a], Md. Motaharul Islam[a,*]

[a] *United International University, United City, Madani Avenue, Dhaka, 1212, Bangladesh*

Abstract

Legal document management systems centralize trust in privileged administrators, making it difficult for external parties to verify whether access decisions or audit records have been altered. We present a hybrid legal-data management framework that places Hyperledger Fabric on the active document-release path rather than using it only as a passive audit log: every protected download must pass a chaincode access check before ciphertext or wrapped document keys are released (per-user enforcement is demonstrated for cross-organization access), and if Fabric is unavailable the path fails closed rather than falling back to a database access-control list. The framework combines browser-side encryption, ciphertext-only storage in a private IPFS swarm, and on-chain anchoring of hashes and content identifiers. Across sixteen experiments, the REST-gateway workload sustains 66–70 transactions per second at the default batch configuration and up to 193 with a reduced batch timeout, for the evaluated three-organization deployment; the ledger access check adds 4.4 ms over a database-only check (11.0 vs. 6.4 ms median), well inside the 50 ms budget, and 6.5 ms end to end over an audit-log-only baseline; outage experiments confirm zero successful downloads during Fabric unavailability with automatic recovery. Reads fail closed, but writes are availability-first: durable re-anchoring bounds the divergence from a revocation issued during an ordering-service outage to the outage itself, reconverging automatically in a median of 15.9 s. Prior permissioned-ledger document systems anchor

*Corresponding author.

Email address: motaharul@cse.uiu.ac.bd (Md. Motaharul Islam)

access decisions off the release path or price on-path authorization only for generic file sharing; none reports measured fail-closed behavior under ledger outage or the end-to-end cost of on-path enforcement against the audit-log architecture it replaces.

Keywords: Blockchain, Hyperledger Fabric, IPFS, Legal document management, Data integrity, Access control

---

1. Introduction

The legal industry depends on sensitive information. Law firms handle case strategy, client communications, privileged records, and electronic evidence, where confidentiality, integrity, and availability are essential. Yet many legal document management systems still rely on centralized infrastructure whose trust model has not kept pace with the threat landscape [1, 2].

Recent sector evidence motivates this security concern. A 2024 NetDocuments report found that more than half of identified data breaches at UK legal firms were caused by insiders [3]. The American Bar Association's 2023 Legal Technology Survey also found that 29% of respondents answered yes to a security-breach question [1]. These figures motivate the structural concern addressed here: NIST IR 8202 [4] identifies administrator control as a risk in permissioned systems, while NIST SP 800-92 [5] and NIST SP 800-53 AU-9 [6] emphasize protecting audit logs against unauthorized modification.

The core problem is architectural. Conventional legal document management systems centralize trust in a privileged database or application administrator. That administrator, or an attacker who compromises their credentials, may be able to read restricted documents, alter access-control tables, modify audit logs, or erase evidence of having done so [4]. A system may still encode a detailed role hierarchy, as illustrated in Fig. 1. The central question is not whether such a hierarchy can be represented in software, but whether its enforcement decisions and audit records can be verified by parties other than the administrator who recorded them.

1.1. Existing Challenges and Problem Definition

Three deficiencies in current legal document management systems motivate this work. First, audit trails are commonly stored in databases controlled by the same organization whose behavior they are supposed to evidence. Append-only database designs improve local tamper resistance. They do not

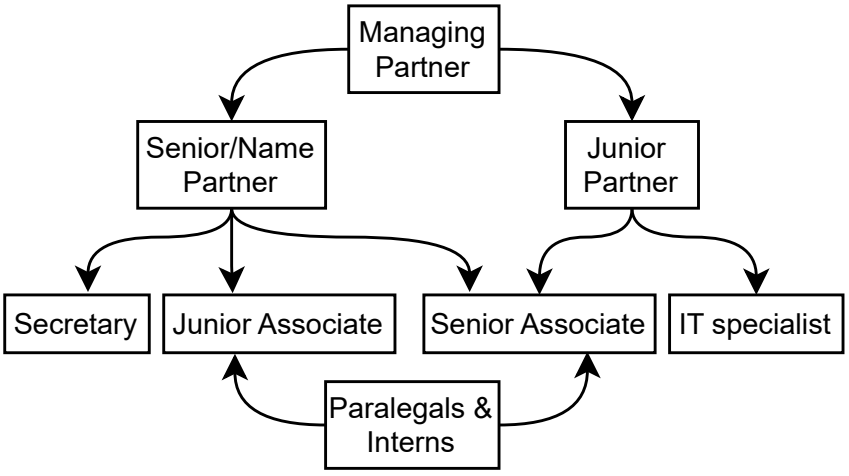


Figure 1: Role hierarchy in the proposed framework, from managing partners with full case visibility to paralegals restricted to assigned documents only. In the prototype this intra-organization hierarchy is enforced at the application (JWT/RBAC) layer; ledger-evaluated per-user enforcement is demonstrated for cross-organization access (Section 7).

provide independently verifiable immutability if verification still depends on the organization that controls the database [4].

Second, access control is usually enforced through folder permissions, application-layer role checks, or mutable ACL tables. A privileged administrator may bypass or alter these controls directly. There is often no verifiable record of when access was granted, under what conditions it was meant to expire, or whether revocation was enforced [7, 8].

Third, many systems record a username alongside an event without a cryptographic signature binding the stated actor to the document content or action being recorded. This leaves room for fabricated or misattributed records [9].

These deficiencies share a root cause: the system's correctness depends on administrative trust. In adversarial multi-firm legal workflows, documents may be shared across firms, clients, experts, and regulators. Auditability therefore requires more than a database log controlled by one party. The required shift is from administrative trust to verifiable enforcement and evidence: access decisions should be reproducible from policy state, audit

records should be independently checkable, and document integrity should be anchored cryptographically.

1.2. Proposed Solution and Research Objectives

This paper proposes a hybrid legal document management framework in which Hyperledger Fabric [10] is placed on the normal authorization path, rather than used only as a passive audit log. Under normal operation, a document download triggers a `CheckAccess` chaincode query. The query evaluates time-bounded grant state and capability-token semantics before the application releases ciphertext or a wrapped document key.

In this mode, the ledger does not merely record an application decision after the fact. It provides the verifiable policy state against which the access request is evaluated (cross-organization access; see Section 7 for intra-organization scope). When Fabric is unavailable, the evaluated document-download path returns HTTP 503 and does not fall back to a database ACL; no new documents are released through this path during an outage. The prototype's write paths (upload, grant, and revocation) are, by contrast, availability-first: on Fabric unavailability they complete the operational write and log the failed ledger anchoring rather than deny (Section 7).

Encrypted document payloads are stored off-chain in a private two-node InterPlanetary File System (IPFS) swarm [11]. SHA-256 hashes and content identifiers are anchored on-chain, keeping Fabric commit latency independent of document size. The same decomposition keeps document payloads erasable off-chain, with only fixed-size hashes, identifiers, and timestamps persisting on the ledger; Section 7.1.9 analyzes the resulting erasure and retention position. The ordering service is a 3-node EtcdRaft cluster distributed across LawFirmA, LawFirmB, and a regulator organization. This provides crash-fault tolerance under Raft assumptions, not Byzantine resistance to malicious orderer collusion [12, 13].

The framework design specifies browser-side AES-256-GCM encryption via the WebCrypto API [14, 15]. It also specifies an ECDH-HKDF-AES-GCM P-256 hybrid wrapping construction, implemented with the WebCrypto-supported ECDH, KDF, and AES-GCM primitives, for document-key wrapping. ECDSA P-256 document signatures are generated by the user [16]. The prototype implements these mechanisms partially and discloses the remaining engineering gaps in Table 7.

This work answers three research questions:

- **RQ1:** Can a permissioned blockchain architecture provide ledger-verifiable access-control state to consortium peers under normal operation, while enforcing a strict fail-closed policy for the evaluated document-download path during Fabric unavailability?
- **RQ2:** Does evaluating access control in chaincode at query time impose acceptable read-latency overhead, and does absolute write latency remain within the defined SLO for interactive legal workflows? We define acceptable overhead as median read overhead below 50 ms and total write latency below 5 s under realistic concurrency. The 50 ms threshold follows the cognitive heuristic that visual feedback under 100 ms is perceived as instantaneous [17, 18]. The 5.0 s write threshold is a conservative Service Level Objective (SLO) within the 10-second usability limit for background tasks before user context is lost [17].
- **RQ3:** Does the framework sustain throughput sufficient to absorb realistic legal workload spikes? The evaluation uses a synthetic steady-state baseline of a 1,000-user firm performing one document action per minute per user (16.7 TPS). This is an illustrative engineering assumption for capacity planning, not a measured industry workload [1, 19]. Because enterprise network traffic is bursty rather than uniform, a conservative 10× peak-load heuristic requires ≈167 TPS. Section 6.8 measures whether the tunable `BatchTimeout` configuration can reach this peak-load target.

Throughout the paper, we distinguish between the *proposed framework*, which describes the intended security architecture, and the *prototype*, which is the measured implementation. The prototype is useful for evaluating architectural viability and system costs, but it is not presented as a production-ready legal platform. Table 7 summarizes the gap between the complete framework design and the current implementation.

1.3. Contributions

The primary contributions of this work are fourfold, and all four are measurement results rather than design proposals. The framework does not propose a new access-control primitive, a Byzantine-fault-tolerant ordering protocol, or a production-ready legal deployment.

First, a measured relocation of the release-path authorization boundary, from application-only trust to ledger-verifiable access-control

state, *scoped to the read path*. Every protected download is authorized by a chaincode evaluate against committed ledger state before ciphertext or wrapped keys are released. Write-side authorization remains custodial, and we do not claim otherwise.

Second, measured asymmetric failure behavior: reads fail closed while writes are availability-first, with the resulting divergence quantified rather than argued. Under ledger unavailability successful downloads fall to exactly zero (Experiment 9), but a revocation issued during an ordering-service outage does not reach the ledger, and the release path continues to authorize from last-committed state for the outage's duration (Experiment 16). Durable re-anchoring bounds that divergence to the outage itself, reconverging without operator action in a median of 15.9 s (Experiment 16b). To our knowledge no prior Fabric-and-IPFS document system reports either half of this behavior under measurement.

Third, a scale characterization of the release path. The access check is effectively flat across three orders of magnitude of world-state growth, from 10^3 to 10^6 documents (Experiment 12), and across two to seven organizations (Experiment 13), with per-document storage measured at $\approx$7 KB per peer. Ledger growth is additionally characterized in time as well as in document count, since the time anchor accrues storage on a fixed schedule.

Fourth, the end-to-end price of on-path enforcement against a *durable* audit-log-only baseline — the architecture this design replaces — rather than against a fire-and-forget straw man (Experiment 14b).

Against the closest current systems the delta is measurement scope: RBAC-IPFS [20] prices on-path authorization for generic IPFS file sharing, zero-trust delegation [21] verifies capabilities on-ledger, and Steichen et al. [22] gate IPFS retrieval on a smart contract, but none reports measured fail-closed behavior under ledger outage, none quantifies the divergence a write-side outage induces, and none quantifies the end-to-end premium of on-path enforcement over the audit-log architecture it replaces.

The prototype supporting these results is implemented for a three-organization legal consortium using Hyperledger Fabric, a private two-node IPFS swarm, PostgreSQL operational storage, and a Spring Boot REST gateway. Cryptographic mechanisms are standard and are treated as engineering rather than contribution: AES-256-GCM for document encryption, a P-256 hybrid wrap for per-recipient key distribution, and ECDSA P-256 for document signatures, with Experiment 6 reporting their cost as Node.js WebCrypto fallback measurements rather than browser-performance claims.

The supporting evaluation uses the REST-gateway workload rather than an isolated peer/orderer microbenchmark, so reported throughput reflects the application path exercised by legal document registration and retrieval. It covers throughput, latency, file-size effects, audit-query cost, synthetic WAN latency, `BatchTimeout` sensitivity, cryptographic operation cost, ledger-history depth, and outage behavior. At the canonical batch configuration (`BatchTimeout=2s`, `MaxMessageCount=500`), the REST-gateway workload sustains approximately 66–70 TPS, about 4.0 to 4.2× above the estimated 16.7 TPS legal workload. Reducing `BatchTimeout` to 500 ms produced the highest measured throughput, 193.0 TPS, for the evaluated three-organization deployment.

Fabric `CheckAccess` latency exceeds the PostgreSQL-only ACL path by ≈4.4 ms after warm-up, an order of magnitude inside the 50 ms interactivity budget. Experiment 9 (Fig. 24) demonstrates strict fail-closed security enforcement on the document-download path: successful downloads fall to exactly zero for the full 45-second Fabric unavailability window ($t = 30$–75 s), with HTTP 503 denials averaging approximately 30 per second (peaking at 44). Normal operation resumes automatically after an approximately 20-second recovery lag dominated by the access-path circuit breaker completing its 30 s open-state window before half-open probes readmit Fabric calls.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the system architecture. Section 4 describes governance, security, and the threat model. Section 5 details the prototype. Section 6 reports the experiments. Section 7 discusses limitations and regulatory considerations. Section 8 discusses the research-question findings and interprets the experimental results. Section 9 concludes the paper.

2. Literature Review

Blockchain applications in data management have expanded rapidly, evolving from integrity-preservation prototypes to systems that also address provenance, access control, and compliance workflows [23, 24]. Two recent systematic reviews frame the specific problem addressed in this paper. Alkhatib [25] reviewed blockchain-based identity and access management systems and found that independently verifiable audit trails and insider-aware access control remain open deployment challenges. Many systems record access events after they occur but still rely on a centralized application server or institutional database to decide whether an access request should be allowed. Precht et al. [26] similarly identify the absence of production-grade access-control enforcement in document management prototypes as a barrier to adoption.

These reviews do not imply that prior systems lack all access control; rather, they show that the enforcement boundary often remains outside the ledger, so an auditor must still trust the system that made the decision.

Table 1: Comparison with representative blockchain-based legal, evidence-management, and access-controlled storage systems. Ledger-eval. authz. = the authorization decision for a specific document is evaluated against committed ledger state on the normal document-release path, before content or keys are returned. Network-level partitioning (for example Fabric channel membership) is ledger-backed but is not a per-document decision on that path, and is scored P rather than Y; Sig = user-level signature binding actor to document content; Rev = ledger-verifiable revocation or expiry. Y = supported; N = not supported; P = partial.

System	**Primary contribution**	**Tools**	**Ledger-eval. authz.**	**Sig**	**Rev**	**Main limitation relative to this work**
Liu and Zheng [27]	Judicial evidence preservation with on-chain anchoring and off-chain storage.	Blockchain + IPFS	N	N	N	Evidence integrity is anchored, but document access is not evaluated by ledger-visible policy before release.
Kim et al. [28]	Two-level evidence blockchain separating active and static investigation data.	Two-level blockchain	N	N	N	Improves evidence lifecycle organization, but does not provide query-time document-level chaincode authorization.
Onyeashie et al. [29]	Multi-party evidence workflow using Fabric channels and IPFS storage.	Fabric + IPFS	P	N	N	Separates law-enforcement, forensic, and judicial workflows into distinct channels, so authorization is ledger-backed but enforced by channel membership rather than as a per-evidence decision on the retrieval path.
Verma et al. [30]	Tamper-evident judicial record management and timestamped action logging.	Public blockchain	N	N	N	Provides timestamping and integrity evidence, but does not target confidential document retrieval or private access enforcement.

Table 1 (continued)

System	Primary contribution	Tools	Ledger-eval. authz.	Sig	Rev	Main limitation relative to this work
Hanafi et al. [31]	Chain-of-custody auditing with Fabric and IPFS.	Fabric + IPFS	N	N	N	Uses Fabric mainly for audit recording; per-request access enforcement and per-recipient key wrapping are not evaluated.
Guo et al. [32]	EMR storage and sharing over separate attribution and data channels, with smart-contract identity verification.	Fabric + IPFS	P	N	N	Verifies identity via zero-knowledge proofs in chaincode and partitions records across channels, but the decision is an identity check plus channel scoping rather than a per-record authorization evaluated before release; usage records are written for audit. Per-recipient key wrapping and enforcement cost are not evaluated.
Steichen et al. [22]	Modified IPFS whose retrieval is gated by an Ethereum smart-contract ACL — the architectural ancestor of on-path authorization.	Ethereum + IPFS	Y	N	P	Permissionless identities rather than consortium MSPs, no measured behavior when the chain is unavailable, no per-recipient key wrapping (so a distributed key cannot be revoked), and generic file storage rather than a legal workflow.

Table 1 (continued)

System	Primary contribution	Tools	Ledger-eval. authz.	Sig	Rev	Main limitation relative to this work
Liu et al. [20]	Role-based access control for IPFS with root-CID smart-contract authorization on the retrieval path.	Fabric + IPFS	Y	N	Y	Prices on-path authorization for generic file sharing (146–156 ms end-to-end under 50 ms emulated WAN), but does not evaluate ledger-outage behavior, an enforcement-cost baseline, per-recipient key wrapping, or a legal workflow.
Chen et al. [33]	Peer-to-peer personal and organizational file management on Fabric and IPFS.	Fabric + IPFS	P	N	N	Chaincode privilege checks are validated functionally but never timed; no fail-closed design, expiry, or per-recipient key distribution.
Mukta et al. [21]	Zero-trust capability delegation with verifiable credentials on Fabric.	Fabric + SSI	P	P	Y	Delegation verification is benchmarked as a chaincode operation (peak 29.4 TPS), but there is no storage tier: authorization is never enforced on a document-release path.
Peelam et al. [34]	NFT-based evidence registration with fog computing and IPFS.	Ethereum + IPFS	N	P	N	Evidence minting and transfer delays are seconds-scale; document access is not evaluated by ledger-visible policy before release.
Notash et al. [35]	Adaptive access-control framework for e-health data sharing with broadcast encryption.	Fabric + IPFS	P	N	P	Access policies are enforced and formally analyzed, but enforcement latency on the release path is not isolated (reported metrics are workflow-scale averages).

Table 1 (continued)

System	Primary contribution	Tools	Ledger-eval. authz.	Sig	Rev	Main limitation relative to this work
Capability and ABAC systems [36, 37, 38]	Fine-grained delegation or attribute-based authorization.	Capability / ABAC	P	N	Y	Policies can be expressive, but enforcement authority is typically a service layer rather than consortium-visible ledger state.
Proposed framework and prototype	Ledger-verifiable legal document authorization with encrypted off-chain storage.	Fabric + IPFS	Y	P	Y	Strict access uses Fabric fail-closed on reads; revocation is anchored durably, so an ordering-service outage delays rather than loses it (Experiment 16b), but the release path still authorizes from last-committed state for the outage's duration. Prototype limits include custodial Fabric identities, custodial transaction attribution for user-level signatures, and a 2-node application IPFS swarm (3-node storage-tier behavior measured separately in Experiment 10).

Table 1 compares prior systems based on where the access decision is made. The table does not imply that prior systems lack access control entirely. Instead, it distinguishes systems where access is mainly decided by the application server from systems where the decision is evaluated against ledger-visible policy before a document is released. The proposed contribution is therefore not a new access-control primitive, but a shift in the enforcement boundary: in normal operation, document access is checked through Fabric chaincode before protected material is returned to the user.

2.1. Foundational Technologies

Hyperledger Fabric is a permissioned blockchain platform in which participants are authenticated through Membership Service Providers (MSPs) that issue X.509 identities. Fabric supports configurable endorsement policies, private data collections, and channel-level governance, making it suitable for consortia of known institutions that require verifiable execution without exposing all data to a public network [10, 39, 40, 41]. These properties are relevant to legal workflows because firms, clients, experts, and regulators often need shared evidence of actions without merging their internal databases.

Fabric is not designed to store large binary documents directly on the ledger. For that reason, many systems pair Fabric or another blockchain with IPFS [11]. IPFS provides content-addressed storage: if a stored object changes, its content identifier changes, making tampering detectable by address mismatch. However, content addressing does not by itself guarantee availability. Objects must be pinned and replicated by designated nodes, and private deployments require governance over who pins, who can fetch, and how long content is retained [42, 43]. The proposed framework therefore uses IPFS only for encrypted off-chain payload storage, while Fabric stores the verifiable metadata needed for integrity and access-control auditing.

Chaincode (Hyperledger Fabric uses the terms *chaincode* and *smart contract* interchangeably; this paper uses *chaincode* throughout) can automate policy decisions, but the security benefit depends on where the policy is evaluated. If chaincode only records an event after an application server has already made the access decision, the blockchain improves auditability but does not remove the application server from the authorization trust boundary. The distinction used in this paper is narrower: the proposed framework places a chaincode query on the normal document-download path so that the access decision is evaluated against ledger state before the application releases the protected material.

2.2. Evolution of Blockchain in Legal Technology

2.2.1. Integrity-Only Systems

Early legal and forensic blockchain systems focused primarily on tamper-evident chain of custody. NyaYa [30] anchors judicial records to establish timestamped evidence integrity, while Forensic-chain [44] and Block4Forensic [45] apply similar ideas to digital-forensics and IoT evidence pipelines. These systems demonstrate the value of immutable timestamps and hash anchoring,

but they generally treat access control as an external concern. Applied legal-domain deployments continue this line: Hernando-Corrochano et al. [46] manage trusted wills for digital assets as a practical blockchain case, again centering instrument integrity and custody rather than access-controlled retrieval of confidential documents. In other words, the ledger can help prove that a record existed at a given time, but it does not necessarily decide who may retrieve a confidential document in a multi-party legal workflow.

2.2.2. Hybrid Blockchain–IPFS Storage Systems

A second line of work combines blockchain anchoring with off-chain storage to address scalability. Liu and Zheng [27] and Jlil et al. [47] show that legal or criminal records can be represented by on-chain hashes while the larger payload is stored elsewhere. This pattern is useful because it avoids placing large documents directly on-chain. Its limitation is that storage integrity and access authorization are different problems. A document hash can prove that a retrieved payload matches a prior record, but it does not by itself enforce time-bounded sharing, revocation, or per-recipient key distribution.

2.2.3. Multi-Party and Fabric-Based Evidence Systems

More recent systems use permissioned blockchains to support multi-institution evidence workflows. Kim et al. [28] organize evidence across active and static chains, and Onyeashie et al. [29] use Fabric and IPFS to separate law-enforcement, forensic, and judicial workflows. Hanafi et al. [31] and Guo et al. [32] also combine IPFS and Fabric for secure record storage and event auditing, demonstrating the architecture's viability for highly confidential, role-based document sharing. These systems move closer to the legal-consortium model considered in this paper, but their primary contribution remains evidence management and audit recording. To the best of the authors' knowledge, none of the prior legal and evidence-management systems in Table 1 report evaluating an access-control decision against chaincode on the critical path of every document download or measuring the latency cost of doing so in a REST-gateway legal-document workflow, leaving an open baseline against which future permissioned-blockchain document-access systems can calibrate their access-control overhead; the access-control-centric line of work reviewed next narrows, but does not close, this gap.

2.2.4. Access-Control-Centric Fabric and IPFS Systems

A fourth line of work makes the access decision itself the primary contribution. The architectural ancestor of this pattern is Steichen et al. [22], who gate IPFS retrieval on a smart contract so that a file is served only to an address the contract authorizes — the same structural idea as evaluating authorization on the release path. Four differences matter here. Their deployment is permissionless Ethereum, so principals are key-pair addresses rather than consortium identities with an organizational MSP; there is no measurement of behavior when the chain is unavailable; keys are not wrapped per recipient, so revocation cannot be expressed against an already-distributed document key; and the setting is generic file storage rather than a legal workflow with time-bounded grants and a regulator role. FileWallet [33] manages personal and organizational files on Fabric and IPFS with chaincode privilege checks, but validates them functionally rather than measuring their cost, and provides neither expiry nor per-recipient key distribution. Mukta et al. [21] verify zero-trust capability delegations on Fabric and benchmark the delegation chaincode with Hyperledger Caliper, but the architecture has no storage tier: no document release is ever gated by the verified capability. Notash et al. [35] enforce adaptive policies for e-health sharing over Fabric and IPFS and analyze them formally, reporting workflow-scale averages rather than isolating the enforcement decision on the release path. Closest to this work, the recent RBAC-IPFS scheme of Liu et al. [20] performs a single root-CID smart-contract authorization on the IPFS retrieval path and, notably, *does* measure it: contract execution in 1–2 ms and 146–156 ms end-to-end retrieval under an emulated 50 ms wide-area network. That system targets generic file sharing; what remains unmeasured in it, and in all of the systems above, is the behavior of the enforcement point when the ledger is unavailable, the end-to-end premium of on-path enforcement relative to the audit-log-only architecture it replaces, and the domain mechanisms a legal consortium requires (time-bounded grants, per-recipient key wrapping, user-level signature binding, and a regulator role).

2.3. Identified Research Gaps

The reviewed literature reveals four gaps that motivate the proposed framework. First, most blockchain-based evidence systems provide tamper-evident storage or timestamping but leave authorization to the application layer: Liu and Zheng [27] and Hanafi et al. [31] anchor or audit access events without evaluating the decision against ledger state before release,

so a compromised or overly privileged application server may still authorize access using state that is not independently verifiable at the time of access. Where a ledger-backed check does exist it is typically coarser than a per-document decision: Guo et al. [32] verify identity in chaincode and partition records across channels, and Onyeashie et al. [29] separate workflows by channel, so authorization is scoped by membership rather than evaluated per release. RBAC-IPFS [20] closes part of this gap for generic file sharing by pricing an on-path authorization; its remaining unmeasured aspects are those identified at the close of the previous subsection. Second, access-control models are often coarse-grained or static: capability delegation is verified on-ledger without gating any storage tier [21], and file-management ACLs lack expiry and revocation semantics [33], whereas legal collaboration requires operation-specific capabilities, expiry, revocation, and auditable sharing across institutions. Third, user-level non-repudiation is often incomplete: a ledger transaction may show that an organization submitted an event, but not that a specific user signed the document content or action being recorded [34]. Fourth, off-chain storage availability is frequently assumed rather than governed; IPFS content must be pinned and replicated deliberately.

The proposed framework addresses these gaps by moving the enforcement boundary onto the normal document-release path; the scope of that contribution, a measured relocation of the authorization boundary rather than a new access-control primitive or ordering protocol, is delimited in Section 1.3. The prototype still has limitations, including custodial Fabric identity handling and `localStorage`-based key storage, and those limitations are disclosed in Section 5 and Section 7.

Two related paradigms help clarify the boundary of the contribution. Capability systems such as Macaroons [36] and large-scale authorization systems such as Zanzibar [37] support expressive delegation and centralized policy evaluation at scale. The proposed framework does not replace those systems or claim to improve their policy expressiveness. Its difference is the audit and trust model: in the normal path, the policy state used for access evaluation is visible to consortium peers through Fabric rather than being only an internal service decision. Attribute-Based Access Control (ABAC) [48], including blockchain-assisted ABAC proposals such as Waheed et al. [49], can express fine-grained conditions. The present work is complementary: it focuses on where the decision is enforced and audited in a legal-document workflow, and on the measured overhead of placing that decision on the Fabric-backed access path.

Table 2 summarizes how the proposed architecture differs from these design classes across the properties most relevant to legal consortium deployments.

Table 2: Architectural Comparison of Access-Control Designs. This table compares the authorization trust boundary across four design classes. The centralized and append-only DB baselines are operational latency references, not security-equivalent alternatives.

Property	**centralized DB**	**Append-only DB**	**Fabric audit-only**	**This work**
Admin can silently alter access log	Yes	Hard	No (Fabric ledger; operational PostgreSQL log excluded)	No
Per-request ledger authorization on document-release path	No	No	No	Yes
Fail-closed on outage	No	No	No	Yes on the document-release path (empirically validated); prototype write paths are availability-first (Section 7)
Ciphertext integrity verifiable by client	No	No	Partial	Yes ($\mathcal{H}_D$ anchored on-chain)

Table 2 (continued)

Property	**centralized DB**	**Append-only DB**	**Fabric audit-only**	**This work**
On-chain metadata leakage	None	None	Medium	Medium (encrypted content is protected, but ledger metadata and access patterns remain visible to channel members. PDC-based reduction is future hardening.)
Per-user transaction attribution	Yes	Yes	Custodial (prototype)	Custodial (prototype); per-user X.509 is production path
Byzantine-fault-tolerant ordering	N/A	N/A	No (CFT)	No (CFT); documented SmartBFT production path [50, 51]
Production-ready	Yes	Yes	Partial	No (proof-of-concept prototype)

3. System Architecture and Design

The proposed framework separates three cryptographically distinct concerns: document confidentiality (AES-256-GCM client-side encryption), access policy enforcement (chaincode-authoritative ACL), and audit integrity (dual-store: Fabric ledger and PostgreSQL append-only log). This section first grounds the design in a concrete legal scenario (Section 3.1), then describes the operational workflows the scenario exercises, the cryptographic layer, the two-layer access control architecture, and the audit architecture. Governance, consensus, and the formal threat model are in Section 4.

3.1. Motivating Scenario: A Cross-Firm E-Discovery Exchange

The following scenario motivates every workflow in this section using only mechanisms the prototype implements. It follows a document production between opposing law firms: the setting in which the trust problem this paper addresses is sharpest, because the parties are professionally adverse yet legally obliged to exchange and safeguard each other's materials.

Setting. LawFirmA represents the producing party in civil litigation; LawFirmB represents the requesting party. Under discovery rules such as FRCP 26 and 34 in the United States, LawFirmA must produce responsive documents, many privileged-reviewed and confidential, subject to a protective order that limits who may view them and for how long; comparable disclosure duties arise in other jurisdictions, and cross-border variants increasingly involve electronic-evidence production regimes such as Regulation (EU) 2023/1543 [52]. A regulator (or court-appointed special master) may later audit how the production was handled. The EDRM model describes this lifecycle: identification through preservation, collection, review, and production [53]; the framework's role begins where reviewed documents leave the producing firm's custody.

Production. A LawFirmA associate uploads each reviewed document through the upload workflow (Section 3.2): the browser encrypts the plaintext under a fresh AES-256-GCM key before it leaves the machine (the lawyer's confidentiality duty under ABA Model Rule 1.6 and its "reasonable efforts" gloss in Formal Opinion 477R attaches to exactly this transmission step [54]), the ciphertext is stored in the private IPFS swarm, and `RegisterDocument` anchors the plaintext hash and content identifier on the consortium ledger. The anchored hash is what later authenticates the production: FRE 902(13)–(14) make electronically generated records and hash-verified copies self-authenticating by certification rather than live testimony, and the ledger supplies a process record and "digital identification" of the kind those rules contemplate [55].

Time-bounded access. LawFirmA then issues grants to the named LawFirmB reviewers with `ExpiresAt` set to the protective order's review window, wrapping the document key per recipient. Every LawFirmB download passes `CheckAccess` on the release path: the decision is evaluated against ledger state at request time (Experiment 2: 10.99 ms P50), so neither firm's application administrators can quietly widen or extend access. When the window lapses, expiry is enforced by the chaincode's deterministic timestamp comparison: no cleanup job, no trust in either firm's schedulers. If LawFirmA discovers an

inadvertently produced privileged document, the FRCP 26(b)(5)(B) clawback procedure maps to `RevokeAccess`: the revocation is committed as a ledger transaction both parties can see, and subsequent download attempts are denied and audited (Scenario S1, Section 4.1.4). If the ordering service is unreachable at revocation time, the revocation is recorded operationally and its ledger anchor is queued for durable re-submission, so the operational and ledger views reconverge automatically once ordering recovers, with no operator action (Experiment 16b: median 15.9 s from revocation to convergence for a short outage, $n = 5$). Clawback is therefore *eventually* ledger-verifiable rather than immediate. For the duration of the outage the release path continues to authorize from last-committed state, so a recipient who already holds the wrapped document key can still retrieve ciphertext—and that recipient is precisely the party clawback targets. The queued re-anchoring bounds this exposure to the outage rather than leaving it open indefinitely, but it does not eliminate it (Section 7).

Audit. When the regulator examines the production, or when the parties dispute whether a document was altered or accessed outside its window, `GetDocumentHistory` and the anchored audit events reconstruct the document's custody without trusting either firm's internal databases (Experiments 4 and 7). Should LawFirmB suffer an intrusion during the review window, ABA Formal Opinion 483's post-breach duties (stop the breach, assess what was accessed, notify affected clients) are materially supported by the same records: the ledger shows exactly which documents the compromised credentials could reach and which were actually released, and the fail-closed design (Experiment 9) means an attacker who disables the ledger gains denial, not access [56].

Every mechanism the scenario uses (browser-side encryption, per-recipient key wrapping, time-bounded grants, on-path `CheckAccess`, ledger-anchored revocation, and history reconstruction) is implemented and measured in the prototype; the scenario costs prose, not code. Table 3 generalizes the scenario into a requirement-by-requirement mapping from legal obligations to framework mechanisms and the evidence behind each.

3.2. Operational Workflows

Each workflow below is a step the scenario of Section 3.1 exercised: registration establishes the keys the production relies on, upload is the production step itself, grant and download implement the protective-order window, revocation implements clawback, and the audit workflow serves the regulator's

Table 3: Mapping from legal requirements to framework mechanisms and supporting evidence. US authorities are cited as concrete anchors; comparable duties arise in other jurisdictions.

Legal requirement	**Framework mechanism**	**Evidence / discussion**
Chain of custody and authentication of electronic records (FRE 901; FRE 902(13)–(14) self-authentication by certification [55])	Plaintext hash $\mathcal{H}_D$ and CID anchored on-chain at registration; full custody reconstruction via `GetDocumentHistory`	Experiments 4 and 7; Section 3.3
E-discovery production duties and lifecycle (FRCP 26/34; EDRM stages [53])	Time-bounded per-recipient grants (`ExpiresAt`), organization- and user-level ACL, exportable audit trail	Scenario walk-through (Section 3.1); Experiment 2
Clawback of inadvertently produced material (FRCP 26(b)(5)(B))	`RevokeAccess` as a ledger transaction visible to both parties; subsequent requests denied and audited	Threat-model Scenario S1 (Section 4.1.4)
Client confidentiality in transmission and storage (ABA Model Rule 1.6; Formal Opinion 477R [54])	AES-256-GCM encryption in the browser before transmission; hybrid per-recipient key wrapping; ciphertext-only off-chain storage	Section 3.3; Experiment 6
Post-breach duties: stop, assess, notify (ABA Formal Opinion 483 [56])	Ledger separates *reachable* from *released* documents per credential; fail-closed denial under infrastructure attack	Experiment 9; Section 4.1.4, S2
Retention limits and erasure (GDPR Art. 5(1)(e), Art. 17 [57]; legal-hold interplay)	Design decomposition: erasable off-chain ciphertext (unpin + garbage-collect) with minimal on-chain metadata (hash, CID, timestamps)	Section 7.1.9 (incl. residual pseudonymization caveat)
Regulator and court-supervised audit access	Regulator organization as consortium member with its own peer and ordering node; independent replica of all anchored events	Sections 3.2 and 4.3

review. The workflows are described as implemented and measured; the mapping from each to its legal anchor is Table 3.

3.2.1. User Registration

Fig. 2 shows the registration workflow. The user submits identity attributes, and the application server creates a profile in the off-chain PostgreSQL identity database. In the framework design, registration also enrolls the user with the Fabric CA, issuing an X.509 certificate under the organization's MSP [58, 39]; per-user enrollment is not implemented in the prototype (Table 7). In the framework design, each user also generates two keypairs in the browser at registration: a P-256 hybrid-wrap keypair (sk_{wrap}, pk_{wrap}) for document key wrapping and an ECDSA P-256 keypair (sk_{sign}, pk_{sign}) for document signing (Section 3.3). The two keypairs are intentionally dis-

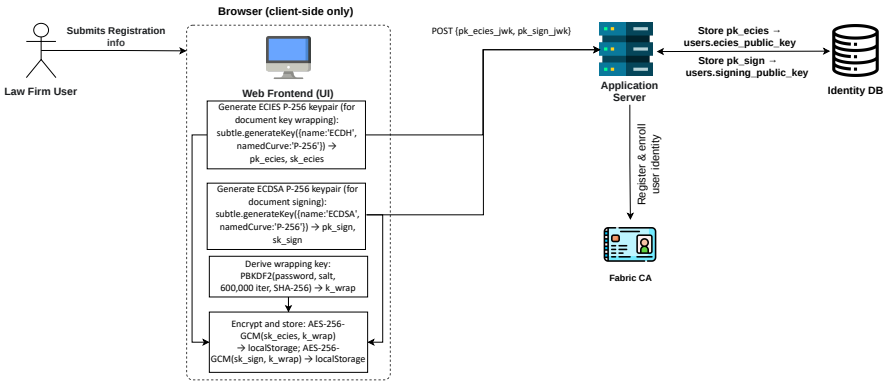


Figure 2: User registration workflow: browser-side P-256 keypair generation, PBKDF2-wrapped key storage in `localStorage`, public-key registration, and (in the framework design) Fabric CA enrollment; per-user enrollment is not implemented in the prototype.

tinct so that a key used for encryption is never used for signing, following key-separation practice [59]. In the measured prototype, Fabric transaction submission uses server-managed MSP credentials (custodial-identity caveat, Section 5.1).

3.2.2. Document Upload and Registration

Fig. 3 illustrates the upload workflow. The browser first generates a fresh AES-256-GCM document key k_{enc} and a 96-bit IV using `window.crypto.getRandomValues`. It computes the SHA-256 plaintext hash $\mathcal{H}_D$ *before* encryption so that $\mathcal{H}_D$ can serve as the document-integrity anchor.

After encryption, the browser prepends the IV to form the stored payload $D_{store} = IV \| D_{enc}$ and computes $\mathcal{H}_C = \text{SHA-256}(D_{store})$. This hash covers exactly the byte sequence from which the IPFS CID is derived. Strict client-side encryption ensures that the server receives only the IV-prepended ciphertext, not plaintext.

The signing key sk_{sign} then signs $\mathcal{H}_D \| \mathcal{H}_C \| \texttt{UTF-8}(id_D)$. This avoids a second round-trip for the CID while still binding the user to the exact plaintext content, the exact IPFS storage payload, and the document identifier. Because $\mathcal{H}_C$ hashes $IV \| D_{enc}$, any later attempt to swap the IPFS ciphertext invalidates the signature.

After IPFS returns the CID, the server invokes the `RegisterDocument` chaincode with $\mathcal{H}_D$, the CID, and metadata. In the prototype, σ is submitted and verified in a dedicated signing action (`POST /api/signatures/{docId}/sign`):

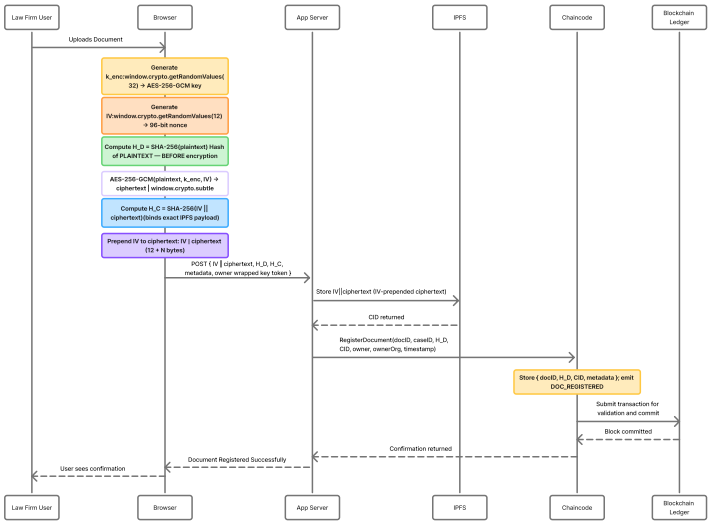


Figure 3: Document upload and on-chain registration, showing browser-side k_{enc} generation, plaintext hashing before encryption, IPFS upload, and CID anchoring. In the prototype, document signing is a separate verified action (Section 3.2) and is not part of this path.

the server verifies it over $\mathcal{H}_D \| \mathcal{H}_C \| \texttt{UTF-8}(id_D)$ and anchors the verified outcome on the ledger as an audit event; carrying σ inside the registration transaction itself is the framework design. The prototype signature does not bind upload timestamp or owner metadata; that binding is a production hardening item [16]. Transaction-level attribution remains custodial (Section 5.1).

3.2.3. Access Control and Document Retrieval

Access control is enforced in two layers. Fig. 4 shows Layer 1: Spring Security JWT RBAC validates the caller's role before the method body executes. Fig. 5 shows Layer 2: the `CheckAccess` chaincode evaluates the on-chain ACL at query time. Only if both layers authorize the request does the server retrieve the ciphertext from IPFS and the hybrid-wrapped key token from the requester's own access entry in the operational store (each grant also anchors the token on-chain). The client unwraps k_{enc} using sk_{wrap} and decrypts locally.

On the evaluated document-download path, Fabric unavailability causes

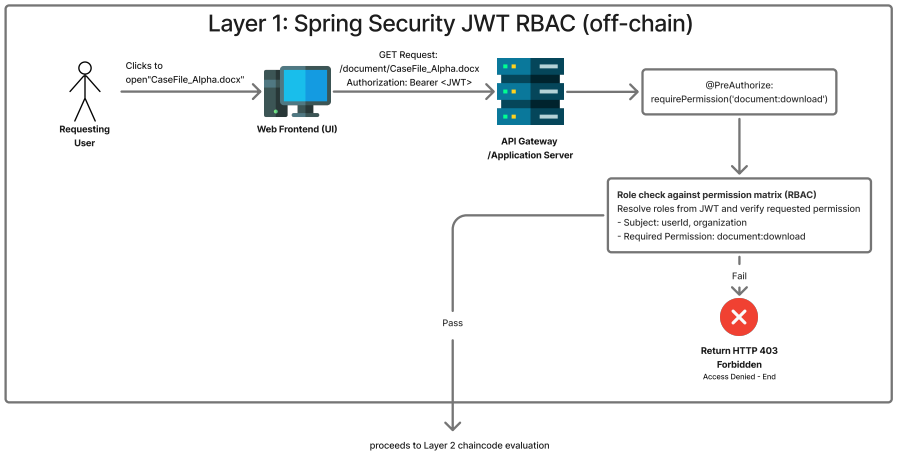


Figure 4: Layer 1 access control: Spring Security JWT RBAC at the API gateway. Requests failing role validation are rejected before reaching the blockchain layer.

HTTP 503 denial and no database ACL fallback. The service records a `FABRIC_OUTAGE_ACCESS_DENIED` event in the operational PostgreSQL audit log. In the released artifact this strict mode is the default; a deployment flag (`material-db-fallback-enabled`, default off) can opt in to an availability-first database-ACL fallback for the release path, which is audited separately and exercised by no measurement in this paper. Because Fabric is unavailable at the time of denial, these outage events are not synchronously committed to the ledger and therefore inherit operational-log trust limits rather than ledger-verifiability guarantees. During Fabric unavailability, no new documents are released through this path, and the chaincode authority cannot be bypassed [60, 10].

3.2.4. Audit and Integrity Verification

The system records every security-relevant action to two independent stores: the Fabric ledger and the PostgreSQL `audit_log` table. Fabric is the independently verifiable record because any consortium peer can query and validate ledger history. PostgreSQL is the operational audit index, configured as INSERT-only by triggers and used for low-latency dashboard queries. The two stores provide different trust guarantees: PostgreSQL improves

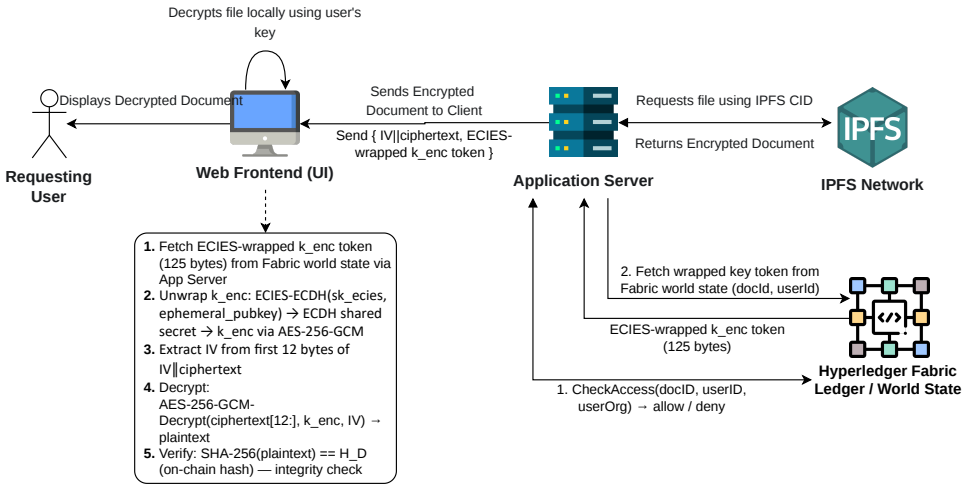


Figure 5: Layer 2 access control and document retrieval: `CheckAccess` chaincode evaluates the on-chain ACL at query time. The client decrypts locally; the server handles only ciphertext and hybrid-wrapped key tokens, anchored on-chain at grant time and served from the requester's own operational access entry.

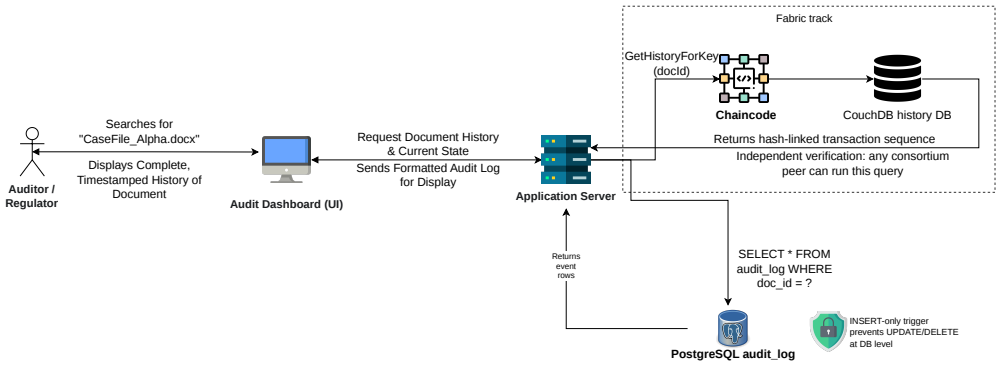


Figure 6: Audit trail retrieval. `GetHistoryForKey` returns the full transaction history for a document key and is independently verifiable by any consortium peer without trusting the application server.

operational visibility, while Fabric provides the stronger evidence against a malicious or compromised database administrator. Figs. 6 and 7 show the audit query and hash verification workflows.

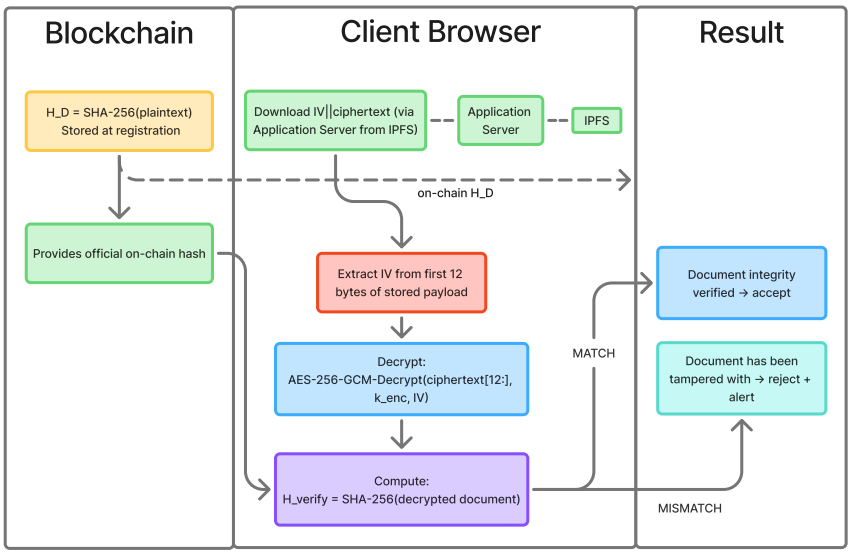


Figure 7: Document integrity verification by comparing on-chain $\mathcal{H}_D$ with the SHA-256 hash of the locally decrypted document.

3.3. Cryptographic Layer

The framework uses a three-tier hybrid cryptographic design. Symmetric encryption (AES-256-GCM) protects document content. Asymmetric key wrapping using an ECDH-HKDF-AES-GCM hybrid wrap over P-256 — a KEM/DEM construction assembled from WebCrypto primitives rather than a named scheme — distributes document keys to authorized recipients. Digital signatures (ECDSA P-256) bind each document action to the acting user's identity. Fabric MSP infrastructure uses ECDSA P-256 for transaction signing independently [39, 16]. In the measured prototype, user-level document signatures and Fabric transaction identity are therefore distinct (custodial-identity caveat, Section 5.1).

3.3.1. Notation

Algorithm 1 formalizes the registration process using the sign-then-store pattern: in the prototype, σ is computed over $\mathcal{H}_D \| \mathcal{H}_C \| \texttt{UTF-8}(id_D)$ before

Table 4: Notation

Symbol	Description
D, M, U	Document, Metadata, User
sk, pk	Private and public key
k_{enc}	Per-document AES-256-GCM symmetric key
sk_{wrap}, pk_{wrap}	P-256 hybrid-wrap keypair for key wrapping
sk_{sign}, pk_{sign}	ECDSA P-256 keypair for signing
$\mathcal{H}(\cdot)$	SHA-256 hash function
$\mathcal{H}_D$	SHA-256 hash of plaintext document (plaintext hash; integrity anchor)
$\mathcal{H}_C$	SHA-256 hash of the IV-prepended ciphertext payload, $\mathcal{H}_C = \text{SHA-256}(IV \Vert D_{enc})$; covers the exact byte sequence stored in IPFS
σ	ECDSA P-256 digital signature
M_{pre}	Prototype signing payload component: $\texttt{UTF-8}(id_D)$ only. Framework design goal: deterministically serialized full metadata $\{id_D, \text{owner}, \text{filename}, t_{\text{now}}\}$; deferred to future work.
$\Vert$	Concatenation
$\perp$	Failure or null return

the IPFS upload, avoiding the circular dependency of signing a value not yet known. The CID is submitted in the Fabric registration transaction, and σ is anchored through the signing action's ledger audit event (Section 3.2); any party verifying non-repudiation verifies σ over $\mathcal{H}_D \Vert \mathcal{H}_C \Vert \texttt{UTF-8}(id_D)$. With 96-bit IVs generated via a CSPRNG and a fresh AES-256-GCM key per document (Algorithm 1, lines 1–2), the birthday-bound IV collision probability for any single key is negligible below 2^{32} documents ($< 2^{-32}$ per NIST SP 800-38D [61]); per-document key generation eliminates cross-document IV-reuse risk entirely: because each document uses an independent k_{enc}, an IV collision in one document has no cryptographic effect on any other document [61]. Listing 2 (Appendix Appendix A) shows the browser implementation.

Protocol composition security. The prototype composes ECDH,

HKDF-SHA-256, and AES-256-GCM via WebCrypto. This is the standard KEM/DEM shape, and each primitive is used within its own standard assumptions, but the composition is not an instantiation of a named, analysed scheme and we claim no security theorem for it: no IND-CCA2 reduction is carried out here, and none is inherited. Adopting a named construction with an existing analysis — HPKE (RFC 9180) [62] being the obvious candidate — is a design target for a production implementation rather than a property of this prototype. Table 5 defines the 125-byte token format used in the prototype.

Table 5: Hybrid-Wrap 125-Byte Wrapped-Key Token Format.

Field	Content	Bytes
Ephemeral public key	Uncompressed P-256 point	65
AES-GCM nonce	96-bit random IV	12
Wrapped key ciphertext	AES-256-GCM encrypted k_{enc}	32
Authentication tag	AES-256-GCM tag	16
Total		**125**

KDF: HKDF-SHA-256 over the ECDH shared secret. AAD: the recipient's user identifier is bound as additional authenticated data, so a token minted for one principal fails authentication if presented under another. AAD is authenticated but not transmitted, so the token remains 125 bytes, and the binding costs nothing measurable (+0.0000 ms wrap, +0.0004 ms unwrap; 90 % CI within ±0.02 ms, n=300 per arm). The document identifier is *not* bound: on the upload path the owner's key is wrapped before the backend assigns the document id, so binding it would require client-generated identifiers. Tokens issued before this change carry no AAD and are byte-indistinguishable from bound ones, so unwrapping falls back to the unbound form until outstanding grants are reissued; that fallback is a transitional weakening of the guarantee.

ECDSA P-256 achieves EUF-CMA under the elliptic curve discrete logarithm (ECDLP) assumption [16]. In this setting, an adversary cannot forge a valid signature without the signing key.

The registration workflow uses a sign-then-store ordering. Before the IPFS upload, σ is computed over three values: the 32-byte plaintext hash ($\mathcal{H}_D$), the 32-byte IV-prepended ciphertext-payload hash ($\mathcal{H}_C = \text{SHA-256}(IV \| D_{enc})$), and $\texttt{UTF-8}(id_D)$. This ordering avoids a circular dependency: the CID is not known until after upload, so signing a payload that includes the CID would require either a second round-trip or a separate commitment scheme.

Signing $\mathcal{H}_C$ still commits the user to the exact IPFS storage payload, including the prepended IV from which the CID is derived. An attacker therefore cannot swap the stored ciphertext later without invalidating the

signature. The resulting construction binds both plaintext and ciphertext hashes, which is stronger than signing only plaintext while preserving the non-repudiation property of plaintext signing [16, 63].

The hybrid wrap protects k_{enc} in transit; AES-GCM protects document content at rest [14]; and ECDSA provides the user-key binding [16]. No citation is attached to the first clause because, as noted above, the wrap is a composition rather than an instantiation of a standardized scheme.

Algorithm 1 Document Registration (Sign-Then-Store Pattern)

Require: Document D, User U, Metadata M, keys sk_{sign}, pk_{wrap}
Ensure: Registration token R or $\perp$

$k_{enc} \leftarrow$ `getRandomValues`(32) {Fresh key per document}
$IV \leftarrow$ `getRandomValues`(12) {96-bit IV, unique per key}
$\mathcal{H}_D \leftarrow$ SHA-256(D) {Hash *plaintext before* encryption}
$D_{enc} \leftarrow$ AES-256-GCM(D, k_{enc}, IV)
$D_{store} \leftarrow IV \| D_{enc}$
$\mathcal{H}_C \leftarrow$ SHA-256(D_{store}) {Covers exact IPFS payload; IV included}
$M_{pre} \leftarrow$ `UTF-8`(id_D) {Prototype: document ID only. Framework goal: bind owner, filename, and t_{now} via deterministic serialization [64]}
$\sigma \leftarrow$ ECDSA-P256.Sign($\mathcal{H}_D \| \mathcal{H}_C \| M_{pre}$, sk_{sign}) {Sign plaintext AND ciphertext hashes} {Sign *before* IPFS upload}
$h_{IPFS} \leftarrow$ IPFS.Store($IV \| D_{enc}$) {Obtain CID after upload}
if $h_{IPFS} = \perp$ **then**
    return $\perp$ {Abort if IPFS unavailable}
end if
$M^* \leftarrow M \cup \{id_D, \mathcal{H}_D, \mathcal{H}_C, h_{IPFS}\}$ {Enrich with CID}
$\tau \leftarrow$ Blockchain.Submit($\langle M^*, \sigma, U \rangle$) {CID in tx; sig covers $\mathcal{H}_D \| \mathcal{H}_C \|$ `UTF-8`(id_D)}
if $\tau =$ success **then**
    emit `DOC_REGISTERED`($M^*.id$, U)
    return $R(M^*.id, h_{IPFS})$
else
    return $\perp$
end if

3.3.2. Key Derivation

User private keys are protected at rest using PBKDF2-SHA256 at 600,000 iterations, following current OWASP password-storage guidance [65] and exceeding the minimum work factor recommended by NIST SP 800-132 [66].

Listing 3 (Appendix Appendix A) shows the derivation function. Experiment 6 (Section 6.6) measures PBKDF2-SHA256 at 600,000 iterations with a P50 latency of 100.44 ms over 10 Node.js WebCrypto fallback trials. Browser automation was unavailable in the measurement environment, so this timing is reported as a same-API Node WebCrypto fallback result rather than a browser-performance claim.

3.3.3. ECDSA Document Signatures

At registration, the browser generates two separate P-256 keypairs using `window.crypto.subtle.generateKey`. The hybrid-wrap keypair is used for document key wrapping ($sk_{wrap} \neq sk_{sign}$), while the ECDSA keypair is used for signing. This follows key-separation practice [59].

When a user signs a document, the browser decrypts sk_{sign} in memory, imports it as a non-extractable signing key, and calls `window.crypto.subtle.sign({name:'ECDSA',hash:{name:'SHA-256'}})`. In the prototype, the ECDSA-P-256 signature is computed over the fixed-width concatenation

$$P = \mathcal{H}_D \parallel \mathcal{H}_C \parallel \texttt{UTF-8}(id_D), \tag{1}$$

where $\mathcal{H}_D$ is the 32-byte SHA-256 plaintext hash, $\mathcal{H}_C$ is the 32-byte SHA-256 hash of the IV-prepended ciphertext, and $\texttt{UTF-8}(id_D)$ is the document identifier.

This payload binds the signing key to the exact document content and IPFS storage payload. Timestamp, owner identity, filename, and case metadata are not included in the prototype signed payload. Binding full document metadata through deterministic canonicalization, such as JCS [64], is a framework design goal and production hardening item. For the timestamp specifically, a client-signed clock value would bind only the signer's *claimed* creation time; the trusted time in this design is the deterministic transaction timestamp assigned at commit, which cannot appear in a payload signed before submission without a second round trip: the same circular dependency that excludes the CID from the signed payload.

The resulting signature is a 64-byte IEEE P1363 raw signature ($r\|s$, 32 bytes each for P-256). The server verifies it with `SHA256withECDSAinP1363Format` (Java SunEC, standard in Java 17+), which consumes WebCrypto's raw output without format conversion [16]. Before verification, the backend recomputes $\mathcal{H}_C = \text{SHA-256}(IV \| D_{enc})$ from the IPFS-retrieved bytes and reconstructs the payload in (1). Listing 4 (Appendix Appendix A) shows the server-side verifier.

3.3.4. Key Management Lifecycle

The key lifecycle spans five phases, shown in Figs. 8–12. Figs. 8–11 depict mechanisms implemented in the prototype, except the Phase-3 signed-proposal validation shown in Fig. 10, which is framework design (see the caption); Fig. 12 (Phase 5, key-escrow recovery) depicts the framework design only and is not implemented in the current prototype (see Section 7.2 and Table 7).

Phase 1: key generation and storage. The browser derives a wrapping key from the user's password using PBKDF2. It uses that key to encrypt both sk_{wrap} and sk_{sign} before storing them in `localStorage`. The server never holds either private key in plaintext.

Phase 2: document encryption and registration. The browser generates k_{enc}, encrypts the document, and submits $\mathcal{H}_D$ and the CID to the `RegisterDocument` chaincode.

Phase 3: access grant. When an owner grants access, the browser wraps k_{enc} under the recipient's pk_{wrap} using an ECDH-HKDF-AES-GCM P-256 hybrid wrapping construction implemented with WebCrypto-supported ECDH/KDF/AES-GCM primitives. The resulting 125-byte wrapped token is submitted to `GrantAccess`; it is 51.2 % smaller than the 256-byte RSA-OAEP 2048 wrapped-key token measured in Experiment 6 [67].

Phase 4: access and decryption. The recipient unwraps k_{enc} using sk_{wrap}, extracts the IV from the first 12 bytes of the stored ciphertext, and decrypts locally.

Phase 5: revocation and rotation. On revocation, the system sets `key_rotation_pending` and notifies the document owner. The owner's browser must complete re-encryption because the server does not hold k_{enc}. An interim production-compatible path is proxy re-encryption, where the server holds a re-encryption key that transforms ciphertext for a revoked user into ciphertext for a newly authorized user without exposing plaintext [68, 69].

The `key_rotation_pending` window is a known limitation of client-managed key architectures. Once revocation is on the ledger, `CheckAccess` immediately denies new downloads for the revoked user. However, cached ciphertext remains outside cryptographic control until the owner completes re-encryption. Other authorized users can still be served the existing ciphertext under the original k_{enc} during this window. This limitation is disclosed in Table 7 and motivates the proxy re-encryption production path.

The two residual key-custody limitations (server-managed MSP wallets and `localStorage`-held signing keys) are cataloged with their production

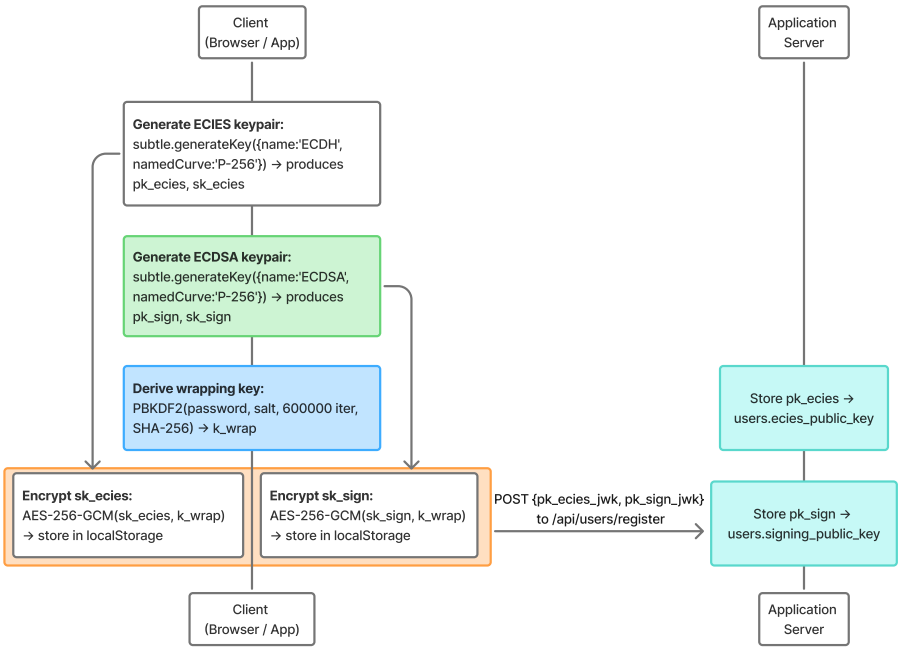


Figure 8: Phase 1: Client-side key generation. Hybrid-wrap and ECDSA keys wrapped via PBKDF2-derived key and stored in `localStorage`, per NIST SP 800-131A key-separation practice.

paths in Table 7 [58, 70, 71, 72].

3.4. Two-Layer Access Control Architecture

Access control is enforced in two independent layers, both of which must grant a request before any ciphertext is served.

Layer 1 is enforced by Spring Security at the API gateway using JWT-validated role permissions [73, 60]. The `@PreAuthorize` annotation on the service method rejects requests that fail role validation before any business logic executes.

Layer 2 is enforced by the `CheckAccess` chaincode invoked within the service method. The chaincode evaluates access through a single `ACL` map using two key conventions: per-user grants are stored under the user identifier (`ACL[userID]`), and organization-level grants are stored under a prefixed

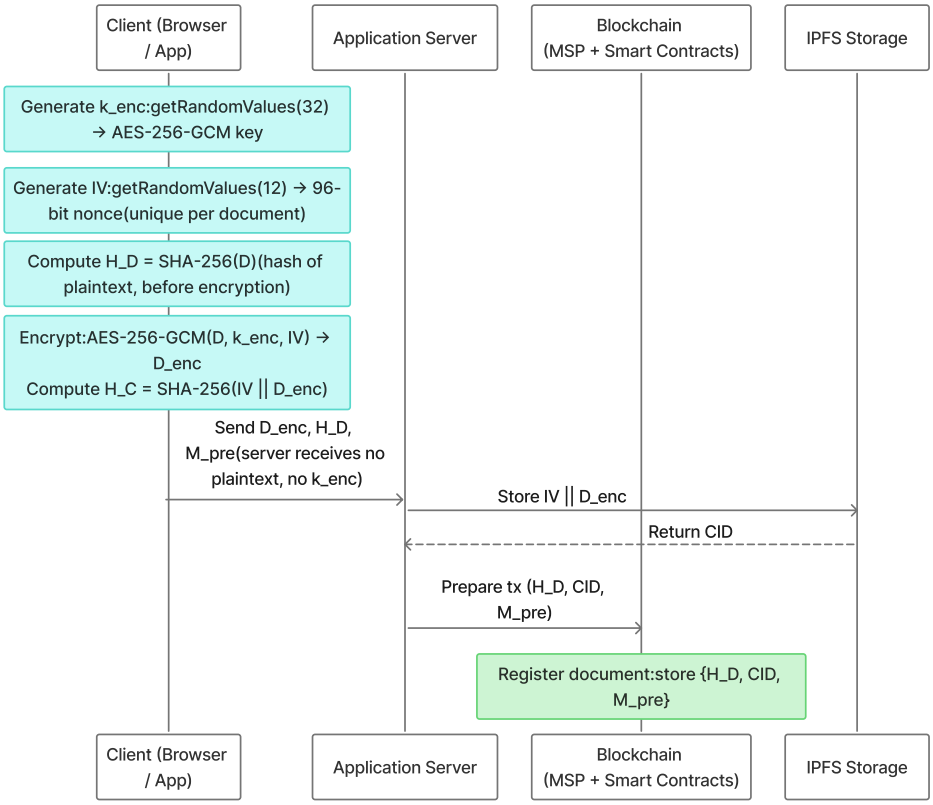


Figure 9: Phase 2 browser-side document encryption and registration with plaintext hashing before encryption and IV-prepended ciphertext. In the prototype, document signing is a separate action (Section 3.2) and is not shown here.

key (`ACL["ORG:"+userOrg]`). Both per-user and organization-level grants support time-bounded expiry evaluated at query time. The owner organization retains implicit read access via the `doc.OwnerOrg == userOrg` fallback. This fallback governs the ledger-side authorization decision only: at the release layer, a wrapped document key is issued solely against the requester's own per-recipient grant entry, so organization membership alone can yield at most ciphertext, never a document key (Section 7).

To prevent Multi-Version Concurrency Control (MVCC) conflicts and ledger bloat under high concurrent read loads, the chaincode dynamically

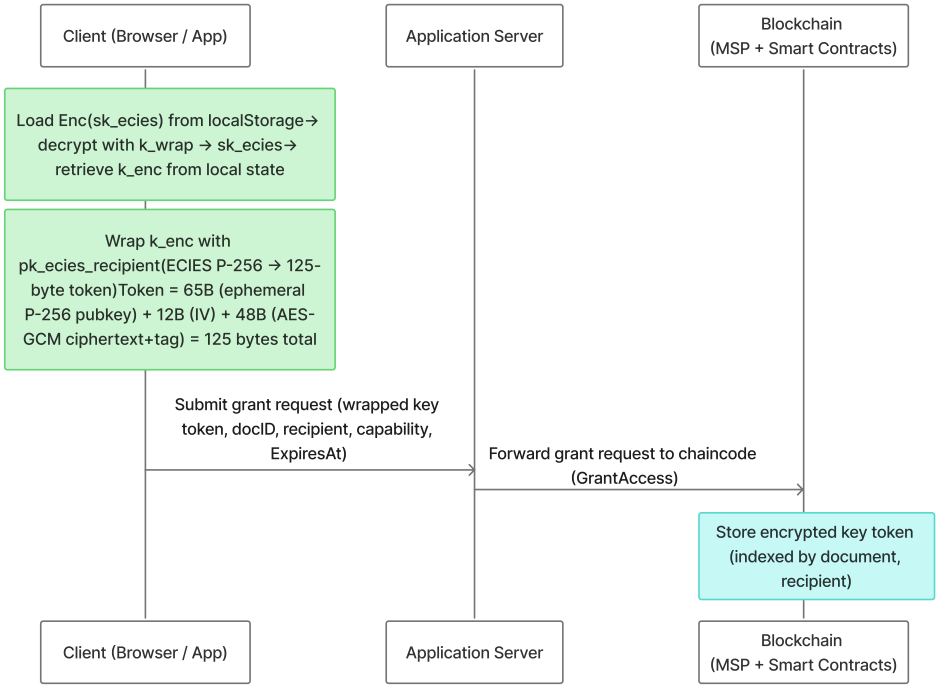


Figure 10: Phase 3 P-256 hybrid key wrapping, showing owner-wrapped k_{enc} and on-chain `GrantAccess` storage.

evaluates expiry but does not attempt a state write-back during the read path. To maintain strict audit fidelity in the CouchDB world state without introducing read-path latency, the system design relies on a dedicated, asynchronous administrative chaincode invocation (`CleanExpiredTokens`) executed periodically as a background cron-job to formally deprecate expired tokens. Listing 1 shows the chaincode implementation. Token deprecation therefore follows an *eventual consistency* model: expiry is enforced immediately at chaincode evaluation time, but the CouchDB world state reflects `StatusExpired` only after the next `CleanExpiredTokens` invocation; a direct world-state query between these two points will show the token as `StatusActive` despite being denied at runtime.

Listing 1: `CheckAccess` chaincode enforcing time-bounded capability tokens (see Section 3.4 for the eventual-consistency model). All five access branches shown: per-user grants

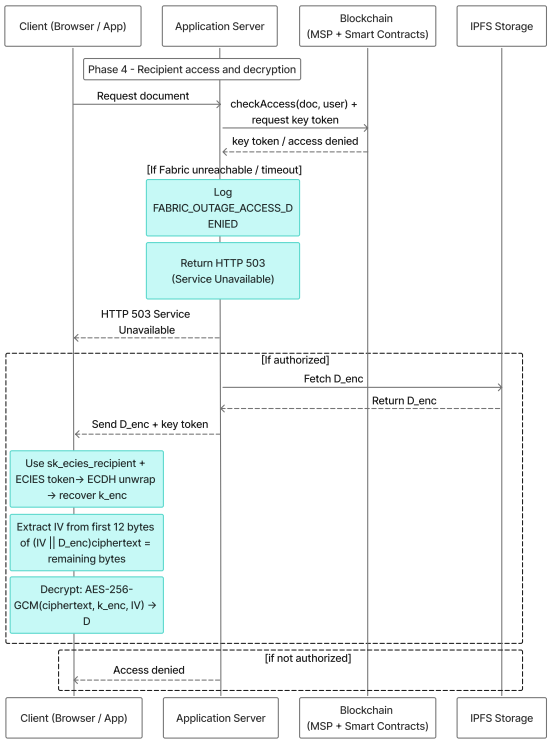


Figure 11: Phase 4: Recipient unwraps k_{enc} with sk_{wrap}, extracts IV from first 12 bytes of stored ciphertext, and decrypts locally. Server delivers only ciphertext and wrapped key token.

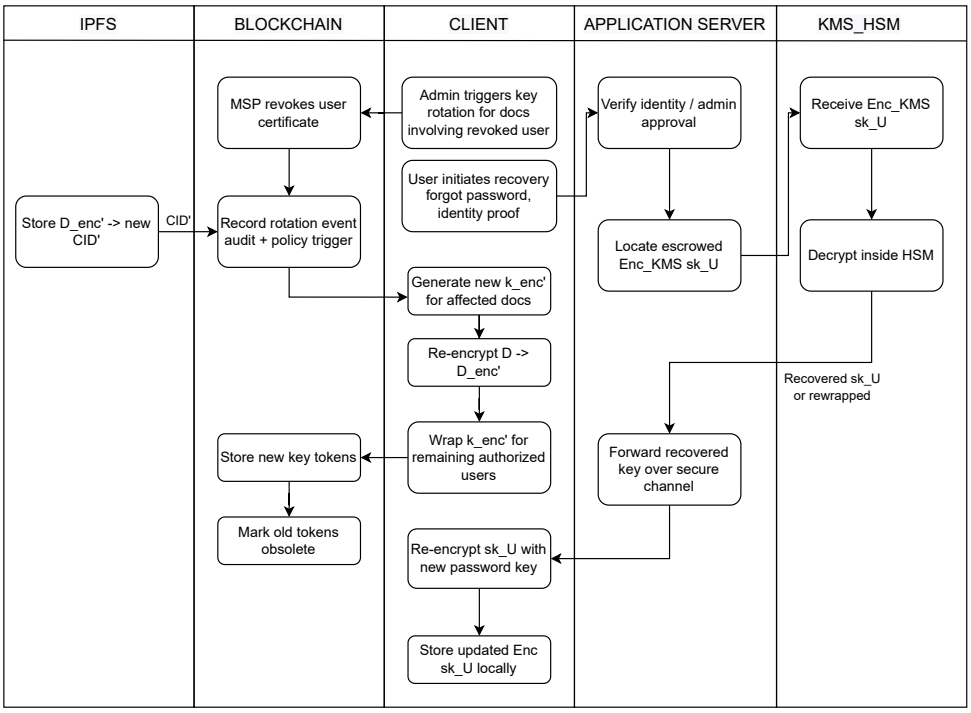


Figure 12: Phase 5: Key recovery via consortium KMS/HSM and revocation workflow (framework design, not implemented in prototype). Re-encryption requires the document owner's browser; the server does not hold k_{enc}.

(Branches 1–2), owner-organization fallback (Branch 3), and org-prefix grants (Branches 4–5).

```
// chaincode/legalcc/chaincode.go
func (c *LegalContract) CheckAccess(
    ctx contractapi.TransactionContextInterface,
    docID, userID, userOrg string,
) (string, error) {
  doc, err := getDocument(ctx, docID)
  if err != nil { return "false", err }
  if doc.Status != StatusActive {
    return "false", nil
  }
  txTimestamp, err := ctx.GetStub().
      GetTxTimestamp()
  if err != nil {
    return "", fmt.Errorf(
        "failed to get tx timestamp: %w", err)
  }
  now := time.Unix(txTimestamp.Seconds, 0).UTC()
```

```
  if grant, ok := doc.ACL[userID]; ok &&
      grant.Status == StatusActive {
    if grant.ExpiresAt == "" {
      return "true", nil
    }
    exp, err := time.Parse(time.RFC3339,
        grant.ExpiresAt)
    if err == nil && now.Before(exp) {
      return "true", nil
    }
    grant.Status = StatusExpired
    // NOTE: No state write-back occurs here. Expiry is
    // evaluated dynamically on every invocation to avoid
    // MVCC read-write conflicts under concurrent load;
    // StatusExpired is only persisted later by the
    // asynchronous CleanExpiredTokens background job.
    return "false", nil
  }
  if doc.OwnerOrg == userOrg {
    return "true", nil
  }
  // Branch 4-5: org-prefix grant
  if grant, ok := doc.ACL["ORG:"+userOrg]; ok &&
      grant.Status == StatusActive {
    if grant.ExpiresAt == "" {
      return "true", nil
    }
    if exp, err := time.Parse(time.RFC3339,
        grant.ExpiresAt); err == nil &&
        now.Before(exp) {
      return "true", nil
    }
  }
  return "false", nil
}
```

On the evaluated document-download path, Fabric unavailability causes HTTP 503 denial and no database ACL fallback. The service immediately logs a `FABRIC_OUTAGE_ACCESS_DENIED` audit event containing the failure reason and throws a `ServiceUnavailableException`, returning HTTP 503 to the caller. During Fabric unavailability, no new documents are released through this path, and a malicious actor cannot use a Fabric disruption to bypass the chaincode authorization layer. Listing 5 (Appendix Appendix A) shows the two-layer enforcement in the download path. If the requester has no firm or MSP binding, the service throws `AccessDeniedException` before invoking the Fabric gateway, ensuring the download path is fail-closed unconditionally for unauthenticated or incomplete principal bindings.

The complete two-layer enforcement path is summarized in Fig. 13, and Algorithm 2 formalizes the capability-grant construction.

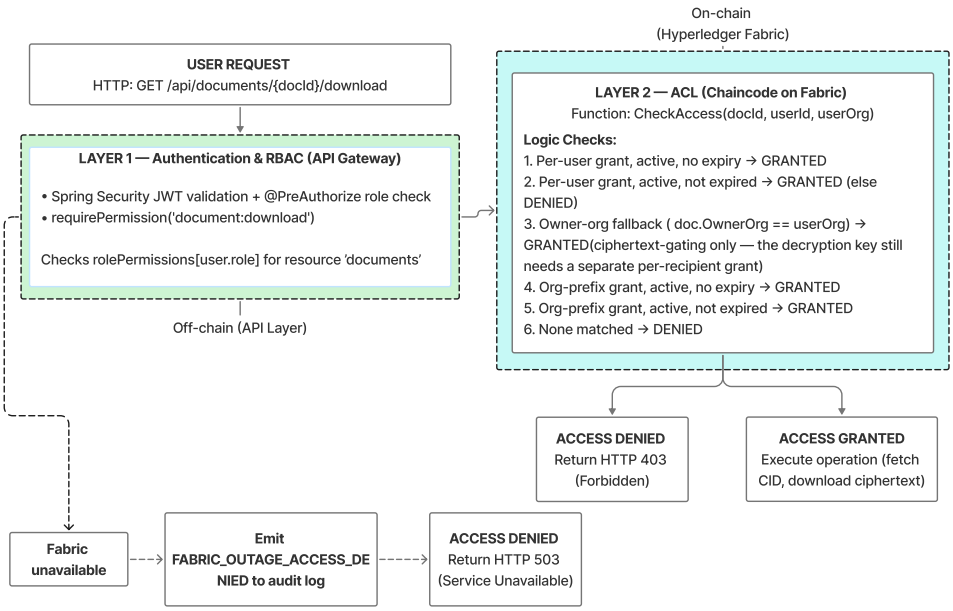


Figure 13: Two-layer access control pipeline: API-gateway JWT RBAC, on-chain `CheckAccess`, and fail-closed `FABRIC_OUTAGE_ACCESS_DENIED` blocking.

Algorithm 2 Capability-Based Access Grant

Require: Document ID d, subject s, permission p, conditions $\mathcal{C}$
Ensure: Capability token T or $\perp$
$t_{exp} \leftarrow t_{now} + \mathcal{C}.\Delta t$ $\{\Delta t = \infty$ if no expiry$\}$
$S_{scope} \leftarrow \text{Parse}(\mathcal{C}.scope)$
$T \leftarrow (d, s, p, t_{exp}, S_{scope})$
$\sigma_T \leftarrow \text{ECDSA-P256.Sign}(\mathcal{H}(T), sk_{grantor})$
$\mathcal{R} \leftarrow \{\mathcal{C}.\text{triggers}\}$
Blockchain.Store$(\langle T, \sigma_T, \mathcal{R} \rangle)$
emit `ACCESS_GRANTED`$(d, u_{grantor} \rightarrow s)$
return T

3.5. Audit Architecture

The system maintains two independent audit stores by design. Their independence is a deliberate security property, but the two stores provide different guarantees.

Fabric ledger (primary, cryptographic). Every significant event is recorded by the `LogAuditEvent` chaincode as an immutable ledger transaction. Any consortium peer can query the history of any document key

using `GetHistoryForKey`, which returns a chronologically ordered, hash-linked sequence of state transitions. The Fabric block hash chain provides cryptographic tamper-evidence natively; the application does not maintain a separate SHA-256 chain over on-chain records because doing so would be redundant with Fabric's own block structure.

PostgreSQL `audit_log` (secondary, operational). A separate append-only table records all events for fast dashboard queries. An INSERT-only enforcement trigger at the database level raises an exception on any UPDATE or DELETE attempt regardless of application privileges. A TRUNCATE-prevention trigger closes the PostgreSQL DDL bypass gap, and TRUNCATE privilege is revoked from all application roles at the schema level. Listing 6 (Appendix Appendix A) shows both triggers. These triggers are an *operational convenience* that raises the bar against accidental or application-layer modification; they are not a security defense against a database administrator with superuser access. A superuser can drop triggers, alter the schema, or bypass the SQL engine entirely via direct file access, rendering the trigger layer ineffective against the Malicious DB Administrator adversary class defined in Table 6. The security guarantee against that adversary relies entirely on the Fabric ledger, which the database administrator cannot access or modify: any consortium peer can independently verify the complete audit trail via `GetHistoryForKey` without trusting the application server or its database [10].

Experiment 4 (Section 6.4) measures PostgreSQL `audit_log` query P50 at 3.9 ms for 1,000 events on Linux `x86_64`, and Experiment 7 (Section 6.7) measures Fabric `GetHistoryForKey` at 135.5 ms P50 for a 107-entry document history. The two stores serve complementary roles: PostgreSQL supports low-latency operational dashboards, while Fabric provides cryptographic non-repudiation and independent verification by any consortium peer without trusting the application server.

4. Governance, Security, and Consensus

4.1. Operational Threat Model

This section defines the operational security assumptions under which the framework is evaluated. The goal is not to present a new cryptographic proof, but to make explicit which adversaries are in scope, which trust boundaries are enforced by Fabric, and which risks remain in the measured prototype. The analysis is organized in four parts: per-component trust assumptions

(Section 4.1.1), an adversary capability matrix (Section 4.1.2), stated security properties with attack-scenario walk-throughs traced to mechanisms and experiments (Sections 4.1.3–4.1.4), and explicitly out-of-scope attacks (Section 4.1.5).

The deployment model is a permissioned legal consortium consisting of known, legally bound institutions such as law firms and a regulator. Participants are authenticated through Fabric MSPs, and state-changing transactions are validated by endorsement policies before being ordered and committed. Under normal operation, document access is evaluated through the `CheckAccess` chaincode before protected material is released. On the evaluated document-download path, Fabric unavailability causes HTTP 503 denial and no database ACL fallback.

4.1.1. Trust Assumptions per Component

The security properties in Section 4.1.3 are argued under the following explicit assumptions. Each names the component whose honesty or integrity is presumed, so that every scenario in Section 4.1.4 can be read as the violation of a specific assumption.

A1 (Cryptographic primitives). The adversary may be adaptive but cannot break AES-256-GCM, ECDSA/ECDH P-256, or SHA-256 under their standard hardness assumptions.

A2 (Ordering service, CFT bound). With a 3-node Raft cluster (n=3), at most $f \leq \lfloor (n-1)/2 \rfloor = 1$ ordering node crash-fails; no orderer behaves Byzantinely [13]. Orderer quorum governs block formation only.

A3 (Endorsement quorum). No coalition of organizations sufficient to satisfy the channel endorsement policy colludes: e.g., under `AND('LawFirmAMSP.peer','RegulatorMSP.peer')`, a single organization cannot fabricate valid state writes, because committing peers independently validate endorsement signatures and read-write sets. A2 and A3 are independent bounds.

A4 (Storage-tier availability). At least one honest IPFS node pinning a document's ciphertext remains reachable. Integrity does *not* depend on A4 (hash anchoring detects tampering under A1); only availability does.

A5 (Identity-table integrity). The PostgreSQL identity table binding users to public keys (`pk_ecies`, `pk_sign`) has not been maliciously modified. This is deliberately the weakest assumption in the prototype (its violation is walked through as Scenario S3) and is discharged in the production design by ledger-anchoring key registrations.

A6 (Client endpoint). The user's browser is uncompromised while keys are unlocked and plaintext is present. Endpoint compromise is analyzed as out of scope (Section 4.1.5).

A7 (Bounded clock manipulation on the evaluate path). Grant expiry is evaluated inside `CheckAccess` against `GetTxTimestamp()`, which on an *evaluate* returns the timestamp the calling client placed in its proposal. Under custodial submission that client is the application gateway, so expiry enforcement does not rest on a clock independent of the gateway. The time-anchor mechanism (Section 6.19) constrains rather than removes this: an endorsed anchor cannot be moved backwards by the gateway, and a proposal more than a fixed tolerance behind that anchor is refused, so backdating is bounded by how stale the anchor is allowed to become. What remains assumed is that the anchor is refreshed — a gateway that withholds heartbeats widens the reachable window, and closing that requires the staleness ceiling whose availability cost is measured in Experiment 17. This is the assumption most specific to custodial deployment, and per-user enrolment does not discharge it, because evaluates are never ordered and therefore carry no ordering-service timestamp to substitute.

4.1.2. Adversary Capability Matrix

Table 6 identifies seven adversary classes relevant to the legal consortium deployment, the property each threatens, the mechanism that resists it, and the residual risk.

Table 6: Operational Threat Model: Adversary Classes and System Defenses. This is a structured adversary analysis, not a formal cryptographic security proof. Residual risk column identifies properties that require governance controls or hardware beyond the current prototype.

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Honest-but-curious peer	Can observe all transactions on joined channels; can read world state; cannot modify ledger or forge signatures	Confidentiality of document content and access patterns	Documents stored as AES-256-GCM ciphertext in private IPFS swarm; CIDs and hashes only on-chain; PDCs limit metadata to authorized peers [40]	Channel members still see CIDs, timestamps, accessor MSP identities, PDC hashes, and plaintext hash $\mathcal{H}_D$, leaking frequency patterns and enabling candidate-document confirmation via local `SHA-256`. The prototype uses organization-level membership (`doc.OwnerOrg == userOrg`) as first-pass control, gating ciphertext release only (document keys require an explicit per-recipient grant); per-user intra-organization ACLs and moving $\mathcal{H}_D$ to a PDC or keyed commitment $\text{HMAC}_k(D)$ remain production hardening.

Table 6 (continued)

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Malicious DB administrator	Full write access to PostgreSQL; can modify application tables; cannot access Fabric peers or IPFS directly	Integrity of access control records and audit log	Under normal operation, access control is evaluated by chaincode (Layer 2). During Fabric unavailability, the document-release path fails closed (returning HTTP 503). The `audit_log` row-level and statement-level triggers block accidental or application-layer modification and TRUNCATE attempts (Listing 6)	Triggers provide operational integrity only, not cryptographic protection from a PostgreSQL superuser. The DB admin cannot directly alter Fabric ACL or audit history, and Fabric outages deny access; this is measured rather than argued in Experiment 18, where a grant forged directly in `document_access` yields no release, and deleting a legitimate row likewise fails to revoke ciphertext access, so database writes are inert in both directions on the release path; however, replacing `pk_ecies` can make later grants wrap k_{enc} to an attacker key while the ledger records an apparently valid grant. A null firm/MSP raises `AccessDeniedException` before the Fabric gateway; no default organization is assumed (Listing 5).

Table 6 (continued)

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Compromised application server / API operator	Controls the Spring Boot service, API-layer RBAC middleware, REST gateway, and operational database access; may attempt to bypass chaincode, misuse server-managed Fabric MSP credentials, serve tampered frontend JavaScript to intercept client-side operations, cache or forward IPFS ciphertext after legitimate delivery, or abuse MSP identity bindings before ledger commitment	Authorization integrity, transaction attribution, confidentiality in prototype plaintext-handling paths	In the honest-code normal path, document release invokes `CheckAccess` against ledger-visible ACL state before protected material is returned. Fabric provides an independently verifiable record for ledger-mediated grants, revocations, and audit events; Fabric outage events log as `FABRIC_OUTAGE_ACCESS_DENIED`; user-level document signatures bind the signing key to $\mathcal{H}_D$, $\mathcal{H}_C$, and id_D	A fully compromised API operator can bypass application logic, misuse custodial MSP credentials, expose plaintext via malicious frontend delivery before browser encryption, abuse off-chain access paths, or substitute recipient keys before grant creation. Because it submits with custodial MSP credentials, it also controls the proposal timestamp that `CheckAccess` reads via `GetTxTimestamp()` on the evaluate path, and could otherwise backdate a query to satisfy an expired grant's `ExpiresAt` (Scenario S5). The endorsed time anchor of Experiment 17 bounds this rather than eliminating it: the gateway cannot rewind the anchor, but withholding heartbeats widens the reachable window unless the staleness ceiling is enabled, so grant-expiry enforcement assumes bounded rather than honest gateway clocks (Assumption A7). Fabric still makes ledger-mediated state changes auditable; production requires browser-only encryption, per-user Fabric identities, ledger-anchored public keys, committing each access decision with its evaluated timestamp, and strict service hardening.

Table 6 (continued)

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Compromised browser / XSS attacker	Executes malicious JavaScript in the user's browser session; can observe passwords or decrypted private keys during unlock/use; can initiate actions as the logged-in user; may tamper with frontend code before encryption or signing	Client-side key confidentiality, document confidentiality before encryption, and authenticity of user-initiated actions	Private keys are stored encrypted in browser `localStorage` under a PBKDF2-derived AES-GCM wrapping key; plaintext and signing operations are intended to occur in the browser; server-side audit records and Fabric logs make submitted actions traceable	Encrypted `localStorage` resists passive storage theft, not active browser compromise. XSS or malicious frontend code can capture passwords, plaintext, or keys during use; production requires CSP, supply-chain controls, WebAuthn/FIDO2 or HSM-backed keys, and hardened frontend delivery.
Compromised IPFS node	Can serve modified or stale ciphertext; cannot forge SHA-256 hashes	Document integrity and availability	SHA-256 hash of plaintext stored on-chain at registration; any tampered ciphertext produces a hash mismatch detectable by any authorized client on download; the design uses a private two-node swarm/cross-pinning configuration	Availability still depends on pinning policy and node health. If both IPFS nodes are compromised or unavailable, content may be unavailable; production requires $\geq$3 pinning nodes or an SLA-backed pinning service.
Orderer collusion (1-of-3 Raft)	A single compromised orderer can observe all unordered transactions; cannot unilaterally reorder or censor if the 2-of-3 quorum continues to function	Transaction ordering integrity and liveness	3-node Raft: any single orderer failure is tolerated while consensus continues; Fabric's execute-order-validate pipeline means peers independently validate every block regardless of orderer behavior [10, 12]	Two colluding orderers (2-of-3) can censor or delay transactions; CFT does not prevent Byzantine ordering. Fabric v3.0 SmartBFT [50, 51] is the production path for high-threat deployments.

Table 6 (continued)

Adversary	Capabilities	Property threatened	System defense	Residual risk
On-chain metadata leakage	Channel members can observe transaction timestamps, invoking MSP identity, and CIDs even for PDC-protected data	Access pattern privacy	PDCs store sensitive metadata in a private collection; only the collection hash is written to the main ledger [40]. A channel member observing transaction frequency and CID patterns over time can infer which documents are accessed frequently, potentially revealing case activity patterns without decrypting content	Traffic analysis can reveal access patterns without document content. Zero-knowledge proofs, batching, cover traffic, or private access protocols are future work to reduce leakage (Section 7.2).

Malicious Fabric peer clarification. Table 6 separates honest-but-curious peer observation from active peer compromise. A malicious Fabric peer may run altered local software, leak channel or world-state metadata, withhold endorsements, return misleading query responses, or collude with other organizations. It cannot by itself commit invalid ledger state when a transaction requires multi-organization endorsement, because committing peers validate endorsement signatures, read-write sets, MVCC versions, and chaincode-definition consistency before updating world state [10, 12]. Residual risks remain if enough endorsing organizations collude, if clients trust a single peer for query results, or if channel metadata itself is sensitive; production deployments therefore require multi-organization endorsement policies, peer monitoring, query-provenance checks where appropriate, and governance sanctions.

4.1.3. Security Properties

The architectural claims are stated as four properties. Each is argued informally under the assumptions of Section 4.1.1; machine-checked verification (e.g., Tamarin or ProVerif models of the wrapping and grant protocol) is future work, but the property set is fixed here so that such verification has a target.

P1 (No release without a ledger-evaluated grant). Under A1–A3 and A5–A7, no protected ciphertext or wrapped document key is returned on the evaluated download path unless a `CheckAccess` evaluation against committed ledger state authorizes the requester at request time. *Argument:* the release handler invokes chaincode before returning material (Experiment 2 measures this decision at 10.99 ms P50); the decision reads world-state grants committed under A3. Expiry is evaluated against the transaction timestamp, which is deterministic across endorsers but, on an evaluate, originates in the caller's proposal: the grant

membership test is ledger-evaluated in the full sense, whereas the *expiry* test is ledger-evaluated only up to the clock the caller supplies. The time anchor bounds that gap rather than closing it, so P1 is stated under A7 and its expiry component is correspondingly weaker than its membership component. In the prototype the release handler evaluates `CheckAccess` against a single peer (the gateway binds one peer endpoint), so P1 additionally assumes that queried peer returns honest world state; independence from a single malicious peer on the read path would require multi-peer query or endorsement-backed reads and is a production hardening item. *Evidence:* Experiments 9 and 14, and directly Experiment 18, in which a forged database grant inserted by an adversary with write access to the operational store does not yield release.

P2 (Fail-closed under ledger unavailability). Under A1, if the ledger cannot be consulted, the download path denies rather than falling back to database state. *Evidence:* Experiment 9: zero successful downloads and zero protected bytes over a 45 s outage under load, with denial audit rows; the storage tier behaves analogously (Experiment 10: total replica loss yields clean timeouts, never stale substitute content).

P3 (Tamper evidence for stored content). Under A1, any modification of off-chain ciphertext is detected before use: the plaintext hash anchored at registration cannot be matched by altered content without breaking SHA-256. Availability, by contrast, depends on A4.

P4 (Attributable ledger actions). Under A1–A3, every grant, revocation, and anchored audit event is attributable to the submitting organization via MSP signatures, and document commitments are additionally bound to user-level ECDSA keys, subject to the custodial-identity caveat of the prototype, and to A5 for the binding between users and their published keys.

4.1.4. Attack-Scenario Walk-Throughs

Four scenarios trace concrete attacks through the design, each identified with the assumption it targets, the mechanism that stops it (or the residual risk it exposes), and the experiment or argument that evidences the outcome.

S1: Revoked-credential replay.. A formerly authorized user (or an API client holding their bearer token) requests a document after the owner has revoked the grant or its `ExpiresAt` has passed. The application cannot serve from cached state: P1 places `CheckAccess` on the request itself, and the chaincode evaluates grant status and expiry against committed world state at the deterministic transaction timestamp. The request is denied and audited — the denial is recorded as an `ACCESS_DENIED` event and anchored to the ledger like any grant, so a refused attempt is independently verifiable rather than merely logged locally (Experiment 18) — *provided the revocation has committed to the ledger*. This qualification is load-bearing: because the prototype's write paths are availability-first, a revocation issued while the ordering service is unreachable updates the operational store but not the ledger, and `CheckAccess` continues to authorize from the stale on-chain grant until the revocation is re-anchored (Experiment 16; Section 7). *Traces to:* P1 mechanism; Experiment 2 (decision cost) and chaincode unit tests in the released artifact.

S2: Induced-outage fallback attack.. A malicious insider with infrastructure access (e.g., the database administrator of Table 6) deliberately disrupts Fabric connectivity, expecting the application to fall back to the PostgreSQL ACL that the insider can edit. The design refuses the trade: under P2 the path fails closed. *Traces to:* Experiment 9: 1,340 HTTP 503 denials, zero protected bytes released during the induced outage, denial events audited; the analogous storage-tier attack (disabling IPFS replicas) degrades availability but never substitutes content (Experiment 10, P3).

S3: Public-key substitution.. This is the sharpest residual risk in the prototype. A database or API adversary violating A5 replaces a legitimate user's `pk_ecies` before a grant is formed. Subsequent `GrantAccess` operations wrap the document key k_{enc} under the attacker's key, and the ledger commits a cryptographically valid transaction whose recipient binding is semantically wrong: P1 holds mechanically while its authorization is subverted upstream. This is the highest-priority hardening item precisely because no ledger mechanism currently witnesses the user-to-key binding. *Mitigation path:* anchor key registrations via a `RegisterIdentity` chaincode function at enrollment, so any post-registration substitution is detectable by any consortium peer; deferred to future work alongside per-user X.509 enrollment (Section 7.2).

The recipient-bound AAD described in Table 5 does *not* mitigate this scenario, and should not be read as doing so. AAD binds an identity *claim* to

the wrapped key, which defeats re-targeting an existing token to a different principal; but an adversary who has already substituted the public key holds the matching private key and can supply the legitimate recipient's identifier as AAD unchanged. Only a ledger-witnessed user-to-key binding closes S3. *Traces to:* violation of A5; residual-risk rows of Table 6.

S4: Storage-tier content substitution.. A compromised IPFS node (or an attacker who obtains swarm access) serves modified ciphertext for a known CID. Content addressing already makes this awkward (the CID is a hash of the stored ciphertext), and P3 backstops it end to end: the client verifies the decrypted plaintext against the on-chain anchored hash before accepting the document, so substitution is detected under A1 regardless of how many storage nodes collude. The attack degrades into denial of availability, which A4 and the replication measurements of Experiment 10 address (single-replica loss: no measurable penalty; total loss: clean timeout).

S5: Gateway-side backdating of an expired grant.. A compromised application gateway holds a document whose grant has expired. Rather than editing any stored state, it constructs a `CheckAccess` proposal carrying a timestamp from before the expiry instant. Because `CheckAccess` is an evaluate, the timestamp the chaincode reads is the one in that proposal, so the comparison against `ExpiresAt` succeeds and the release path authorizes a grant that has lapsed. Nothing in the ledger record is falsified; the adversary supplies the clock rather than the policy, which is why signature checks and endorsement do not detect it, and why per-user enrolment does not help — an evaluate is never ordered and so never receives an ordering-service timestamp.

The time anchor constrains this. `UpdateTimeAnchor` is a submit, so each endorsing peer independently checks the heartbeat's timestamp against its own clock before endorsing, and a backdated anchor cannot be committed without a colluding endorsement quorum (which is A3, already assumed). `CheckAccess` then refuses any proposal more than a fixed tolerance behind the committed anchor, so the reachable backdating window is bounded by the anchor's staleness rather than unbounded.

Two limits are worth stating plainly. First, the gateway is also what submits heartbeats: it cannot rewind the anchor, but it can stop advancing it, and the window then grows with the suppression interval unless the staleness ceiling of Experiment 17 is enabled — which costs availability during an ordering outage, one second of outage tolerance per second of backdating

tolerance. Second, this scenario is analyzed and its defence is measured for enforcement cost, but no backdated proposal was constructed against the deployment: the reported evidence covers the refusal logic and the staleness ceiling, not a demonstrated attack. *Traces to:* A7; residual risk is heartbeat suppression.

4.1.5. Out-of-Scope Attacks

The following are explicitly out of scope for the evaluated prototype, with the assumption they would violate noted: (i) *Byzantine ordering* (violates A2): two colluding Raft orderers can censor or reorder transactions; the SmartBFT migration path is discussed below. (ii) *Endorsement-quorum collusion* (violates A3): a coalition satisfying the endorsement policy can write attacker-chosen state; governance and policy breadth are the controls. (iii) *Endpoint compromise* (violates A6): XSS or malware in the user's browser can capture passwords, unlocked keys, or plaintext during use; mitigations (CSP, WebAuthn/FIDO2, supply-chain controls) are production hardening, not evaluated here. (iv) *Traffic analysis of on-chain metadata*: access frequency and CID patterns leak to channel members even with private data collections, as analyzed in Table 6; cryptographic pattern hiding is future work. (v) *Network-layer denial of service* against gateways, peers, or IPFS nodes beyond the measured outage scenarios. (vi) *Side-channel attacks* on the cryptographic implementations.

The framework uses Raft, a Crash Fault Tolerant (CFT) protocol. CFT tolerates node failures but not Byzantine orderer behavior. This choice is a *performance trade-off*, not a cryptographic security guarantee. Legal accountability reduces the probability of collusion among orderers operated by distinct, legally-bound institutions, but provides no cryptographic bound against Byzantine behavior: a sufficiently motivated or compromised orderer set can still violate CFT's safety assumptions. As noted in the threat model table, 2-of-3 orderer collusion remains a residual risk under CFT, and the current Raft configuration is a *prototype constraint* rather than a framework recommendation. For deployments in which Byzantine fault tolerance is required, the SmartBFT ordering service introduced in Hyperledger Fabric v3.0 is the recommended path [74, 50, 75]. The current prototype uses Fabric v2.4 Raft; migration to SmartBFT does not require changes to chaincode or application logic.

Cryptographic security properties. The individual primitives are used within standard design assumptions: AES-256-GCM provides authenticated

encryption under nonce-uniqueness assumptions, as specified in NIST SP 800-38D [61]; ECDSA P-256 provides digital-signature unforgeability under the elliptic curve discrete logarithm assumption, per FIPS 186-5 [16]; and the hybrid ECDH + KDF + AES-GCM construction follows the standard hybrid-encryption pattern for wrapping document keys [63].

4.2. Security Architecture

Security is implemented through a multi-layered, defense-in-depth approach that separates the intended framework boundary from the measured prototype boundary. The framework design places encryption, signing, and key unwrapping in the browser and uses Fabric as the independently verifiable authority for ACL and audit state. The measured prototype preserves the main ledger-visible checks and enforces strict fail-closed access control; its residual key-custody limitations are cataloged in Section 5.1.

4.2.1. Cryptographic Framework

The system employs a three-tier hybrid cryptographic approach. At the file encryption layer, documents are encrypted client-side in the framework design [14]. For secure key sharing, the AES document key is wrapped using P-256 hybrid wrap via the browser WebCrypto API [15], producing a 125-byte token that is 51.2 % smaller than RSA-OAEP 2048 (256 bytes), though not smaller than an HPKE (RFC 9180) token, as measured in Experiment 6 (Section 6.6). At the infrastructure layer, Hyperledger Fabric uses ECDSA (NIST P-256) for transaction signing and MSP identity validation [39, 16].

4.2.2. Identity Management

The system's identity management design is aligned with the NIST SP 800-63 series. NIST SP 800-63-4 supersedes SP 800-63-3 (withdrawn 2025-08-01) [71, 76]. In the current prototype, authentication is limited to username/password login combined with signed JWTs [73]. The architecture is compatible with migration toward SP 800-63-4-aligned AAL2/AAL3 deployments, but the prototype evaluates only username/password plus JWT authentication. A production deployment can integrate an external identity provider enforcing phishing-resistant MFA, WebAuthn/FIDO2, or hardware-backed authenticators while leaving the RBAC and ACL layers in Section 3 unchanged. This upgrade path requires no chaincode modification.

4.3. Consensus Mechanism and Validation

The framework employs Hyperledger Fabric's Raft consensus protocol [13, 12]. The Raft ordering service is configured as a cluster of three ordering nodes operated by distinct, audited consortium members: one node operated by the regulator and two nodes operated by separate law-firm organizations. This configuration is crash-fault tolerant: with $n = 3$, Raft tolerates one crash fault ($f = 1$) while maintaining consensus progress.

This governance claim refers to operational control of the ordering nodes. A production deployment should also avoid a single administrative trust point for orderer identity issuance, either by using member-controlled orderer MSP/CA administration or by placing any shared ordering CA under multi-party consortium control (the prototype's shared `OrdererOrg` CA is a deployment simplification; Section 5.3). In the current prototype, the Regulator organization participates as an ordering node and a passive channel peer; application-layer regulatory audit interfaces, dedicated read-only compliance views, and regulator-specific endorsement policies (e.g., `AND('LawFirmAMSP.peer','RegulatorMSP.peer')` for high-value asset transfers) are deferred to future work (Section 7.2). No single institution unilaterally controls transaction ordering under the stated consortium-governance assumption, provided that both orderer operation and orderer-identity administration are governed across consortium members.

Every transaction undergoes a multi-stage validation process before being committed to the ledger [10, 41]. For example, a deployment may require endorsement from one peer of the submitting organization for document upload (`OR('LawFirmAMSP.peer')`) and endorsements from both the owner's organization and the regulator for access grants (`AND('LawFirmAMSP.peer', 'RegulatorMSP.peer')`). These policies are illustrative unless matched by the deployed channel and chaincode configuration. A new chaincode definition can only be committed after a sufficient number of consortium member organizations have approved it to satisfy the channel's LifecycleEndorsement policy, ensuring smart-contract upgrades require multi-organization agreement [77].

5. Prototype Implementation

5.1. Prototype Scope

The paper presents a target hybrid framework; the current repository implements a proof-of-concept prototype. Table 7 documents the gaps between framework design goals and current prototype status.

In the user-facing frontend upload workflow, the prototype implements the confidentiality boundary: plaintext remains on the user device, browser-native cryptography performs strict client-side encryption [14, 15], and AES-256-GCM protects each payload before IPFS storage. The benchmark harness used in the performance experiments isolates the REST-gateway, Fabric, database, and IPFS paths by submitting opaque upload payloads that represent the IV-prepended ciphertext accepted by the server; therefore, Experiments 1–3 do not measure browser-side encryption time. Access control is also on the normal download path. Each download invokes the `CheckAccess` chaincode before protected material is released.

When Fabric is unreachable, the prototype fails closed. It records a `FABRIC_OUTAGE_ACCESS_DENIED` event, returns HTTP 503, and does not bypass the blockchain authorization layer [60]. The prototype implements a 2-node private IPFS swarm with cross-pinning and logical deletion [78, 79]. The upload path's parallel IPFS-storage and ledger-anchoring fork/join thread model is a framework-design detail not independently validated in the experiment campaign.

Fabric transaction-level attribution remains custodial because the prototype uses server-managed Fabric MSP credentials. User-level ECDSA document signatures still bind the signing key to the exact plaintext content ($\mathcal{H}_D$), the exact IPFS storage payload ($\mathcal{H}_C$), and the document identifier (id_D). Binding upload timestamp and owner metadata is a production hardening item, so these signatures should not be interpreted as per-user Fabric transaction non-repudiation. The custodial arrangement also does not flatter the reported latencies: migrating to per-user X.509 submission adds a client-side ECDSA P-256 signing operation, measured sub-millisecond in Experiment 6, and leaves the endorsement and ordering round trips unchanged.

Table 7: Framework Design vs. Prototype Implementation. Y = implemented; Y* = implemented with a known availability or governance caveat (see Production path column); P = partial; N = not yet implemented. Production path column identifies the engineering step required to close each gap. †The ECDSA signing/verification mechanism is implemented and binds the signing key to $\mathcal{H}_D$, $\mathcal{H}_C$, and id_D; in the prototype, verification occurs in a dedicated signing action anchored as a ledger audit event rather than inline in the registration transaction, and binding upload timestamp and owner metadata is a production hardening item. Fabric transaction-level attribution remains custodial because the prototype submits Fabric transactions through server-managed MSP credentials (Section 5.1).

Feature	**Framework design**	**Prototype**	**Production path**
Client-side encryption	Browser AES-256-GCM; plaintext never reaches server	Y	Already implemented
ECDSA document signatures	Browser ECDSA P-256 sign; server verifies before Fabric anchoring	Y†	Mechanism implemented and binds $\mathcal{H}_D$, $\mathcal{H}_C$, and id_D; Fabric transaction-level non-repudiation requires per-user Fabric identity (Section 5.1)
P-256 hybrid key wrapping	125-byte token; browser wraps k_{enc} under recipient pk_{wrap}	Y	Already implemented
Fail-closed Fabric invocation	`CheckAccess` evaluated on every download; service denies access on Fabric outage	Y	Implemented

Table 7 (continued)

Feature	**Framework design**	**Prototype**	**Production path**
Per-user and org-prefix ACL evaluation	`CheckAccess` evaluates per-user `ACL[userID]` grants (Branches 1–2), falls back to `doc.OwnerOrg == userOrg` for owner-organization access (Branch 3), and evaluates org-prefix grants under `ACL["ORG:"+org]` (Branches 4–5), as shown in Listing 1.	Y	Implemented
Fine-grained intra-organization per-user ACL	Remove `doc.OwnerOrg == userOrg` owner-org fallback; require explicit per-user grants for all principals, including intra-organization peers	P	Prototype currently authorizes ledger-level read (ciphertext release) for any member of the document owner's organization; document keys remain per-recipient-granted in all cases, and per-user ledger grants are enforced only for cross-organization access. Production hardening item.

Table 7 (continued)

Feature	**Framework design**	**Prototype**	**Production path**
Time-bounded capability tokens	On-chain expiry enforced in chaincode; expired grants denied by `ExpiresAt`; expiry write-back omitted on read path to prevent MVCC conflicts; asynchronous `CleanExpiredTokens` cron-job handles ledger cleanup	Y	Implemented; concurrent expiry write-back behavior treated as production hardening
3-node Raft ordering	Cluster across 3 distinct consortium members (2 firms + regulator)	Y	Already implemented; CFT property follows from Raft configuration
2-node IPFS swarm	Replicated private pinning across 2 nodes; availability depends on pinning policy, node health, and network reachability	Y*	Implemented as private two-node replication; $\geq$3 nodes or managed pinning recommended for production SLA. Experiment 10 measures 3-node replication cost and single-replica-failure behavior on a bench topology
Per-user Fabric identity	Per-user X.509 certificate issued by Fabric CA; HSM-backed	N	Per-user Fabric CA enrollment; HSM wallet integration
Hardware-backed signing key	WebAuthn/FIDO2 or TPM; signing key never in localStorage	N	WebAuthn/FIDO2 integration replacing localStorage

Table 7 (continued)

Feature	**Framework design**	**Prototype**	**Production path**
Key rotation on revocation	Browser re-encrypts k_{enc} for remaining authorized users	P	Notification implemented; browser re-encryption requires owner action; proxy re-encryption is the interim path
TOTP MFA recovery codes	Recovery codes generated at enrollment; admin re-enrollment flow	N	TOTP recovery code generation at enrollment
Hardware TLS attestation	TPM/HSM-backed TLS certificates on Fabric peers	N	Replace software certificates with hardware-attested TPM certs
Fabric PDC storage for wrapped-key tokens	Wrapped-key tokens may be stored in PDCs to reduce key-token metadata exposure	N	Prototype stores hybrid-wrapped tokens as encrypted Fabric world-state values indexed by document and recipient, so every channel member can enumerate which principals hold keys to which documents. The litigation-specific consequences, and the tension with external verifiability that makes PDC migration a design trade rather than a pure improvement, are set out in Section 7.1.5.

The "Per-user and org-prefix ACL evaluation" row refers to implemented cross-organization grant evaluation in the access-control logic, whereas "Intra-organization access control" refers to finer-grained per-user enforcement within the same owner organization. The latter remains partial because owner-organization fallback access can bypass strict intra-organization per-user granularity at the ledger-authorization layer; wrapped document keys remain gated by explicit per-recipient grants in all cases.

5.2. Technology Stack

The technology stack was selected to satisfy enterprise-grade requirements for security, scalability, and modularity. Hyperledger Fabric v2.4 provides the permissioned blockchain core, including private data collections and pluggable consensus [10]. CouchDB 3.2 serves as the peer state database for rich JSON queries over document metadata and ACL structures [80]. Chaincode is implemented in Go for native Fabric ecosystem support [77], and IPFS (Kubo, Dockerized) provides content-addressed off-chain encrypted storage [11].

The application backend is a Spring Boot 3.2.5 REST API running on Java 21 (OpenJDK). This stack was selected for `@PreAuthorize` Spring Security integration on the Layer 1 RBAC path and for native Hyperledger Fabric Java SDK support [81]. Off-chain identity and audit data are stored in PostgreSQL 16 [82], and the web frontend is implemented in React. The evaluation network is containerized with Docker and Docker Compose.

Experiment 6 used Node.js WebCrypto as a same-API fallback because browser automation was unavailable. PBKDF2-SHA256 at 600,000 iterations measured 100.44 ms P50 over 10 trials, and P-256 hybrid wrap reduced the wrapped-key token from 256 bytes under RSA-OAEP 2048 to 125 bytes. The production stack uses Java 21.

5.3. Blockchain Network Topology

The repository contains configuration artifacts for a larger six-peer-organization topology: five representing law firms (Firm-A to Firm-E) and one representing a regulator, plus a separate ordering organization, as depicted in Fig. 14. The measured prototype deployment used a shared `OrdererOrg` CA to issue the three orderer identities for the evaluated three-organization configuration (LawFirmA, LawFirmB, and RegulatorOrg; three orderers, three peers), a repository-level simplification. The production governance model (Section 4.3) instead requires member-controlled orderer-identity administration or multi-party control of any shared ordering CA, which otherwise remains a residual administrative trust point. The ordering service is a 3-node EtcdRaft cluster operated by three distinct consortium members (one regulator node and two law-firm nodes), consistent with the governance configuration described in Section 4. This configuration is crash-fault tolerant under Raft assumptions, tolerating one crash fault in a three-node cluster [12]. Encrypted document payloads are stored off-chain in a private 2-node IPFS swarm with cross-pinning; this configuration provides replicated private pinning, but availability

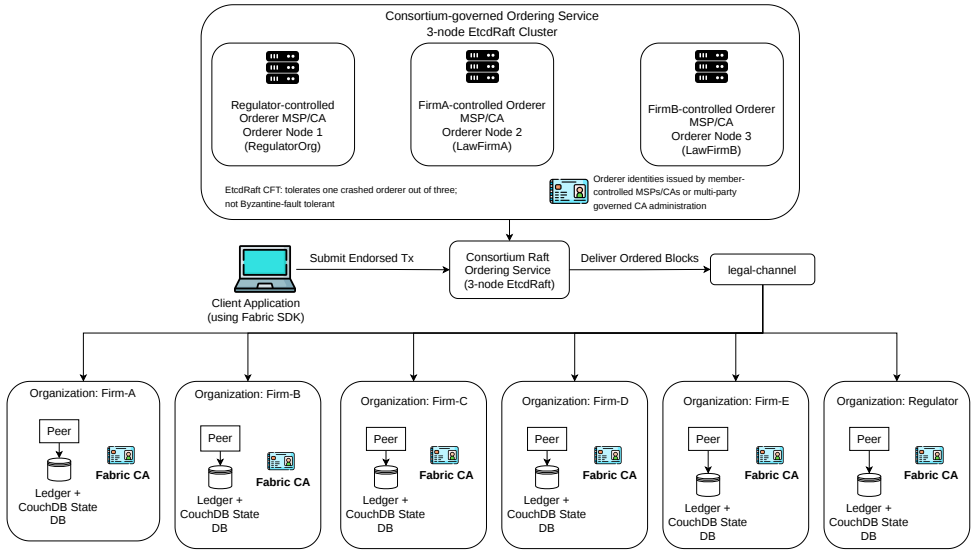


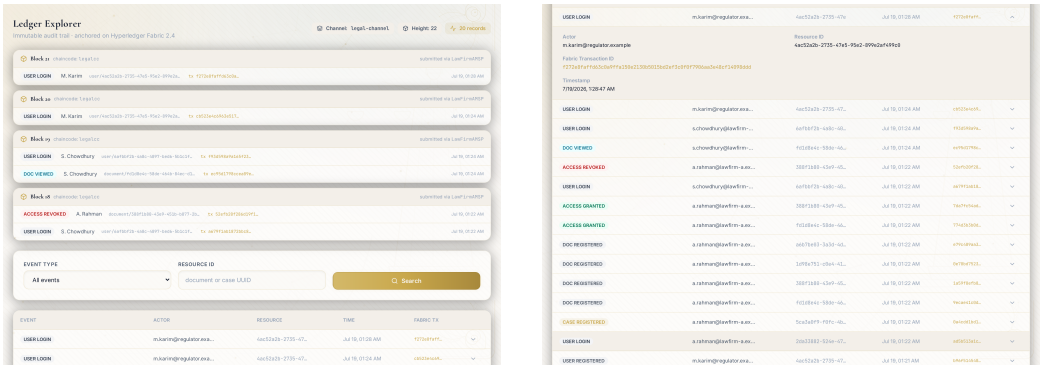
Figure 14: Six-peer-organization Fabric reference topology with a consortium-governed three-node EtcdRaft ordering service. The orderer nodes are operated by the regulator, LawFirmA, and LawFirmB under member-controlled orderer identity administration or multi-party CA governance. Raft provides crash-fault tolerance for one orderer failure; Byzantine-fault-tolerant ordering is outside the prototype scope.

still depends on pinning policy, node health, and network reachability. It should not be interpreted as broad public-IPFS decentralization [83].

Consequently, all performance claims in Section 6 refer to the measured three-organization configuration, not the larger repository topology. This subset understates realistic gossip and endorsement overhead compared to the full deployment; throughput numbers should be treated as an upper bound for the measured 3-org configuration.

5.4. Chaincode Design and Functionality

The core business logic is encapsulated in the `legalcc` chaincode, which implements: `RegisterDocument` (anchors document hash, CID, and ECDSA signature as a ledger asset); `GrantAccess` and `RevokeAccess` (manage capability tokens with optional expiry in the on-chain ACL map); `CheckAccess` (evaluates the on-chain ACL at query time with time-bounded token enforcement; Listing 1); `GetHistoryForKey` (returns the full hash-linked transaction history for any document key); and `CleanExpiredTokens` (asynchronous



(a) Ledger explorer (b) Audit log

Figure 15: Audit and transparency views of the prototype interface: (a) the ledger explorer groups committed entries by block and shows transaction identifiers, channel, timestamp, transaction type, and document references; (b) the operational audit-log view lists application events with timestamps and expandable metadata. The remaining interface screens are provided as supplementary material.

world-state cleanup of expired grants; the eventual-consistency model is described in Section 3.4) [84]. Concurrent write-back behavior for the same expired grant was not isolated in the experiment campaign and is treated as a production hardening issue.

5.5. User Interface and Functional Showcase

The prototype exposes the full workflow through a web interface: registration with role selection, a navigation dashboard, card-based case management, and document dialogs that surface the content identifier, Fabric transaction identifier, integrity hash, and wrapped-key reference for each file, together with controls for downloading the encrypted object or a locally decrypted copy. Because auditability is the central claim of this work, Fig. 15 shows the two transparency views. The ledger explorer (Fig. 15a) groups recent entries by block and displays transaction IDs, the channel, timestamps, transaction types, and document references. The audit-log view (Fig. 15b) lists application events with timestamps and expandable metadata, each event linked to its anchoring Fabric transaction identifier, so administrators can review user activity from within the prototype interface. The remaining interface screenshots are provided as supplementary material.

6. Performance Evaluation

Sixteen experiments evaluate RQ1–RQ3. Experiments 1–9 form the core campaign; Experiments 10–15 extend it along axes the core campaign leaves open: off-chain storage cost, standard-instrument comparability, world-state scale, network size, an architectural baseline, and consolidated statistics. Experiment 16 isolates the orderer-only outage geometry that Experiment 9 (all-peer outage) cannot reach (Section 6.17). Measurements were collected on three platforms: a Windows 11 workstation (Ryzen 7 5700X, Java 17 Temurin, Node.js 24), an Apple M1 MacBook under Rosetta 2 emulation (macOS, Java 21 Temurin, Node.js 22), and a native Linux x86_64 server (Intel Core Ultra 5 125H, Ubuntu 22.04, 8 GB RAM, NVMe SSD, Java 21 OpenJDK, Node.js 18). All authoritative figures are from Linux; M1 and Windows results are auxiliary cross-platform checks where reported. Experiments 10–15 were collected on the same Linux workstation at a later system image and code revision (Docker 29, Node.js 20); a per-run machine-readable environment record accompanies each released dataset.

The evaluated Fabric network used three peer organizations (LawFirmA, LawFirmB, RegulatorOrg) on a shared `legal-channel`, each backed by CouchDB 3.2. The ordering service was a 3-node EtcdRaft cluster across three distinct consortium members. This configuration tolerates one crash fault under CFT assumptions, but the experiment campaign did not evaluate Byzantine ordering behavior [12].

Unless otherwise stated, `BatchTimeout` was set to 2 s and `BatchSize.MaxMessageCount` to 500. Sustained throughput was measured with a 60 s fixed-duration closed-loop load generator; a fixed-count harness was used as a latency cross-check. A custom Node.js harness targeted the Spring Boot REST gateway with a 20 % write / 80 % read mix. Each data point is the mean of five independent runs after a discarded warm-up round, following standard Hyperledger Fabric benchmarking practice [85, 86]. The load generator ran co-located with the network containers on the single evaluation host; generator CPU was recorded to detect client-side saturation and remained far below it in the sensitivity runs (2–14 %, Experiment 8). A post-campaign two-host validation additionally repeated both reference configurations with the same closed-loop harness on a separate consumer laptop (Intel i5-1035G1) over campus Wi-Fi: cross-host throughput measured 228.0 TPS [222.7, 233.3] at the 500 ms `BatchTimeout` (n=10; vs. 193.0 [182.8, 203.2] co-located) and 70.5 TPS [67.8, 73.2] at 2 s (n=5; vs. 66.3 [63.7, 68.9]), with zero transaction

errors and client CPU at 4–12 %. Cross-host throughput therefore exceeds the co-located values reported throughout this paper (co-location depressed rather than inflated the measured numbers), and the co-located figures are retained as the conservative baseline. Table 8 summarizes the configurations.

Direct numerical comparison of the REST-gateway workload with prior systems is not possible because, to the best of the authors' knowledge, the systems in Table 1 do not report application-gateway throughput under concurrent load with a documented traffic mix [27, 28, 29, 30, 31, 33, 34]. The PostgreSQL-only baseline is therefore used as a reproducible upper-bound reference [87]. Instrument-level comparison *is* possible where prior systems report Hyperledger Caliper results [20, 21]: Experiment 11 benchmarks the chaincode with Caliper for exactly this purpose, and Section 6.16 tabulates the reported numbers side by side with the usual hardware caveats.

For upload-path experiments, the load generator submits opaque byte payloads representing the IV-prepended ciphertext that the server receives in the browser workflow. Consequently, Experiments 1– 3 isolate REST-gateway, Fabric, database, and IPFS costs; browser-side encryption is measured separately in Experiment 6.

6.1. Experiment 1: Scalability Under Concurrent Load

This experiment varies concurrent client load from 50 to 600 and measures committed throughput in both Fabric mode and a PostgreSQL-only baseline, to answer RQ3. Writes were issued as `POST /documents/upload` requests

Table 8: Experimental Hardware Configurations

Parameter	**Windows**	**M1 (Rosetta)**	**Linux (auth.)**
OS	Win 11 Pro	macOS amd64 emu.	Ubuntu 22.04
CPU	Ryzen 7 5700X	Apple M1	Core Ultra 5 125H
RAM	16 GB	16 GB	8 GB
Java	17 Temurin	21 Temurin	21 OpenJDK
Fabric	2.4.x	2.4.x	2.4.x
Orderer	3-node Raft	3-node Raft	3-node Raft
Role	Auxiliary	Exps. 1–4	Exps. 1–15

and reads as `GET /wrapped-key`, with a 10 s per-request timeout and five repetitions per concurrency level.

The framework REST gateway with Fabric sustained approximately 66–70 TPS in the 60-second duration runs at the canonical setting; the fixed-count cross-check sweep spans 62–70 TPS per point across the measured 50–600 client range, with zero transaction errors up to 400 clients. The sustained rate is about 4.0 to 4.2× above the 16.7 TPS baseline of RQ3, an illustrative capacity-planning assumption for a 1,000-user firm performing one document action per minute per user [1, 19]. Throughput is invariant with respect to concurrency: as load rises, per-request latency grows while committed throughput holds flat, the signature of a system gated by the fixed 2 s Raft batch window rather than by compute or concurrency. CPU utilization stayed below 23 % throughout, confirming that the bottleneck is the Raft orderer `BatchTimeout=2s`, not compute. The PostgreSQL-only baseline peaked near 279 TPS in the stable region. Beyond that point, the closed-loop load generator saturates, so the shaded region in Fig. 16 is treated as a client-side limit rather than a database-server failure.

Three considerations bound this interpretation. First, the experiment used the measured 3-organization deployment rather than the larger repository topology. Larger deployments will incur additional gossip, endorsement, and resource overhead, so the measured throughput is an upper bound for the evaluated configuration rather than a guarantee for all consortium sizes.

Second, `BatchTimeout` is tunable. Reducing it from 2 s to 500 ms raises the highest measured throughput to 193.0 TPS under the evaluated three-organization deployment, an 11.6× margin over the 16.7 TPS baseline without architectural change [12].

Third, enterprise traffic is bursty. An aggressive 10× burst factor raises the target to ≈167 TPS; Experiment 8 (Section 6.8) shows that the 500 ms setting reaches 193.0 TPS, a 15.6% margin above that peak target [88]. The throughput cost of blockchain consensus is therefore a configuration trade-off, not an architectural barrier.

6.2. Experiment 2: Function-Level Operation Latency

This experiment isolates per-operation latency for `CheckAccess` (read) and `RegisterDocument` (write) in Fabric and DB-only modes, providing the direct answer to RQ2.

`CheckAccess` latency was measured with a Node.js `performance.now()` timer under sequential single-client load. Twenty warm-up calls were discarded

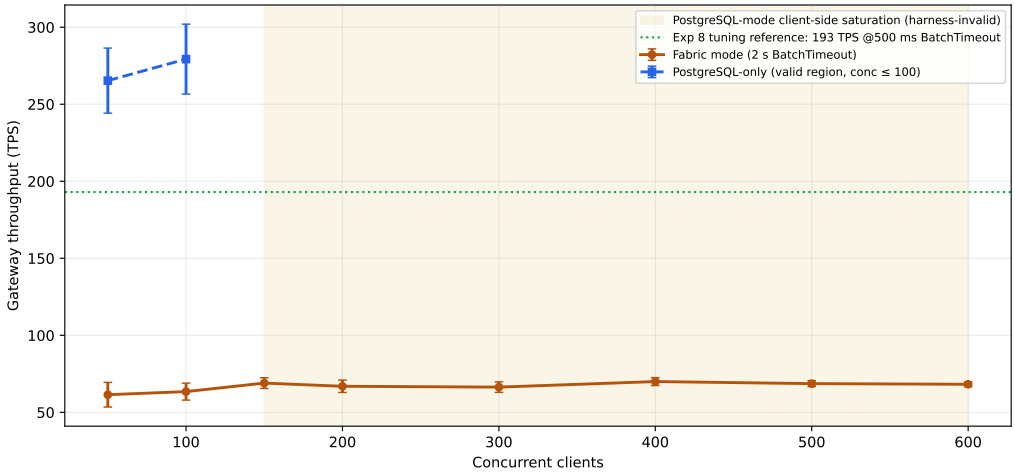


Figure 16: Experiment 1 scalability under concurrent load on Linux `x86_64` (computed from the released raw data; mean with 95 % t confidence intervals, n=5 per point): Fabric REST-gateway throughput compared with the PostgreSQL-only baseline restricted to its harness-valid region (conc $\leq$ 100); the shaded region marks PostgreSQL-mode client-side load-generator saturation (self-flagged invalid rows, excluded from analysis), and the dotted line is the Experiment 8 `BatchTimeout` tuning reference (193 TPS at 500 ms).

before 100 measured samples; the same protocol was applied to the DB-only baseline on the same hardware. Warm-up matters because JIT compilation can modestly overstate framework cost before the JVM has stabilized.

`RegisterDocument` latency was measured end-to-end through the Spring Boot Fabric gateway. The write path was verified to commit on-chain: each call returned a real `fabricTxId`, and `GetDocumentHistory` confirmed the same identifier. As with the read-path test, 100 measured samples followed 20 warm-up calls.

The write path is dominated by ordering latency. `BatchTimeout` accounts for ≈96 % of the 2,084 ms total, and the independent Experiment 3 commit-latency constant (2,132 ± 20 ms across all file sizes) is consistent with orderer processing dominating the path. Experiment 2 and Experiment 3 use independent sample sets and different harness boundaries, so their values are reported as consistent observations rather than additive components.

On native Linux x86_64 with JVM warm-up, the ledger access check costs a measurable amount more than the DB-only path: `CheckAccess` runs at 10.99 ms P50 [10.71, 11.25] against 6.44 ms [6.21, 6.70] for the PostgreSQL

ACL lookup, a difference of +4.37 ms (90 % CI [3.81, 4.93]; two-tailed Mann-Whitney $p = 1.0 \times 10^{-31}$), with the full distribution in Table 9. Both paths remain far below the 50 ms interactive threshold of RQ2 at every measured percentile, and a two one-sided test against that pre-specified margin confirms equivalence within it ($p < 10^{-15}$). The cost is therefore real but not consequential for interactivity, which is the claim RQ2 actually makes.

We report this difference as a cost rather than as equivalence. An earlier version of this experiment reported the two paths as statistically indistinguishable on the basis of a non-significant Mann-Whitney result; that inference was unsound — failure to reject is not evidence of equivalence — and it does not survive re-measurement on the current build.

The difference is *not* attributable to the TimeAnchor freshness read introduced in Experiment 17. Deploying chaincode with that read removed and restored, around the measurement above (sequences 10 and 11, $n = 200$ per configuration), changes `CheckAccess` by -0.10 ms (90 % CI $[-0.63, +0.42]$, $p = 0.75$) — no detectable effect, with the spread between two identically configured runs exceeding the on-versus-off difference. The ≈4 ms is the standing cost of the on-path ledger check itself: a gRPC round trip to the peer and chaincode execution against the state database, against a single local SQL query in the comparison arm. Measured across three chaincode sequences the difference is stable at +4.0–4.4 ms.

A standalone PostgreSQL-only write baseline was not collected because the framework has no write path that bypasses Fabric. Measuring isolated PostgreSQL inserts would omit the Fabric SDK endorsement round-trip and orderer batch costs that are structurally part of every document registration. The absolute `RegisterDocument` latency (2,084 ms P50) is therefore evaluated directly against the 5 s SLO. Experiment 1 corroborates this interpretation: under the 66–70 TPS Fabric load, PostgreSQL operated at approximately 25% of its independently observed write-capacity reference (≈279 TPS), and CPU utilization remained below 23%. Cross-platform results also support BatchTimeout dominance: the Apple M1 session measured 2,147 ms, within about 3 % of the Linux figure, as expected when the orderer timeout accounts for ≈96 % of total write latency.

The 0.65 ms median difference is well within HTTP round-trip noise and meets the RQ2 50 ms read-overhead threshold with substantial headroom. The write-latency SLO is met in absolute terms (2,084 ms P50 vs. the 5 s threshold). Experiment 1 confirms the latency is orderer-dominated rather than

Table 9: Read-path latency distribution: `CheckAccess` via Fabric gateway vs. DB-only ACL lookup, warmed runs (Linux x86_64). The Fabric arm was measured twice, once on either side of the DB-only arm, so host drift is observed rather than assumed; both brackets are pooled here.

	Fabric ($n = 200$)	**DB-only** ($n = 100$)
Mean (ms)	11.651	7.284
Std dev	2.846	2.746
P25 (ms)	10.04	5.83
P50 (ms)	10.99	6.44
P75 (ms)	12.46	7.70
P95 (ms)	17.30	13.38
P99 (ms)	28.84	19.22
Mann-Whitney $p = 1.0 \times 10^{-31}$ (two-tailed); difference +4.37 ms, 90 % CI [3.81, 4.93]; TOST vs. ±50 ms: equivalent ($p < 10^{-15}$).		

database-dominated (PostgreSQL operating at ≈25% of its write-capacity reference, CPU below 23%), so the SLO headroom is structurally grounded, as summarized in Fig. 17.

6.3. Experiment 3: File-Size Independence of Ledger Latency

Because the ledger stores only a fixed-length SHA-256 hash and IPFS CID regardless of document size, Fabric commit latency should be invariant with respect to payload size. This experiment confirms that property across a 1–50 MB range. IPFS upload time was measured directly via the Kubo HTTP API (`curl` multipart, `pin=false`), isolating the IPFS-node upload cost from the secondary-pinning overhead present in the full REST path (≈234 ms on M1). Ten samples per file size were collected; total *server-side* latency is Fabric constant plus IPFS upload. This excludes client browser encryption time and network transmission from the client to the REST gateway, which are workload- and bandwidth-dependent and not modelled in this experiment.

Fabric commit latency was $2{,}132 \pm 20$ ms across all file sizes (1–50 MB), flat within run-to-run Raft variance. IPFS upload grew from 10 ms at 1 MB to 95 ms at 50 MB, consistent with near-linear Kubo chunk hashing. Total server-side P50 ranged from 2,142 ms to 2,226 ms at 50 MB, an ≈85 ms increase over

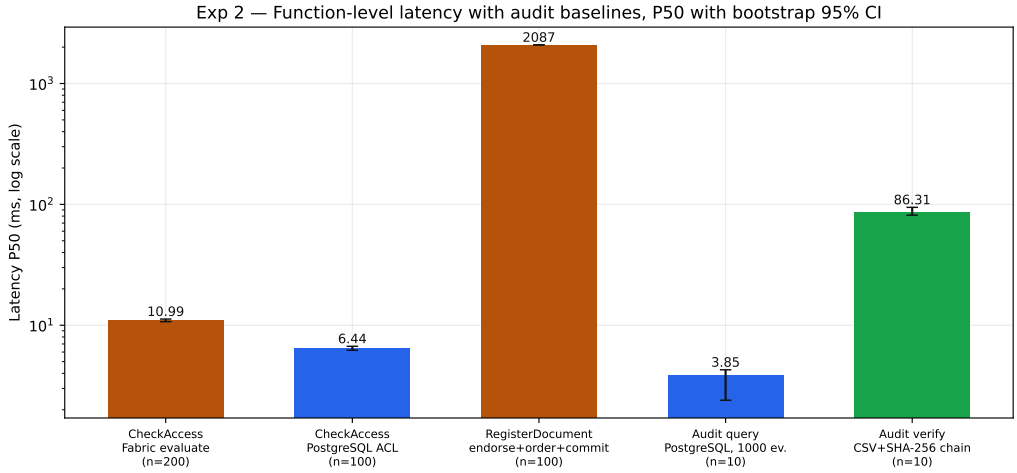


Figure 17: Experiment 2 function-level latency (P50, log scale; Linux, JVM-warmed; bootstrap 95 % confidence intervals): `CheckAccess` in both modes (Fabric n=200 pooled across the two brackets, PostgreSQL n=100; *disjoint* CIs), `RegisterDocument` (n=100), and the Experiment 4 audit trust/latency baselines (n=10 each).

a 50× size range, entirely attributable to IPFS. All points remain below the 5 s RQ2 threshold by more than 2×, confirming that even large legal evidence files register without exceeding acceptable latency for a non-interactive background operation (Fig. 18).

6.4. Experiment 4: Audit Verification Efficiency

This experiment quantifies operational audit-query latency for PostgreSQL and CSV-based verification over 1,000 events, and compares those results with the Fabric `GetHistoryForKey` measurement from Experiment 7 at a 107-entry document history. These paths provide different trust guarantees and are not identical workloads: PostgreSQL is the low-latency operational index, CSV+SHA-256 is a manual verification baseline, and Fabric provides ledger-verifiable history.

A PostgreSQL `audit_log` table was seeded with 1,000 real Fabric-anchored events. Automated query time was measured via `GET /api/audit?size=1000` over five runs. Manual verification time was the wall-clock cost of a Python script that fetches the CSV export and recomputes the SHA-256 digest of each of the 1,000 exported records and cross-checks them against the on-chain validation anchors. This is a computational lower bound, since operational

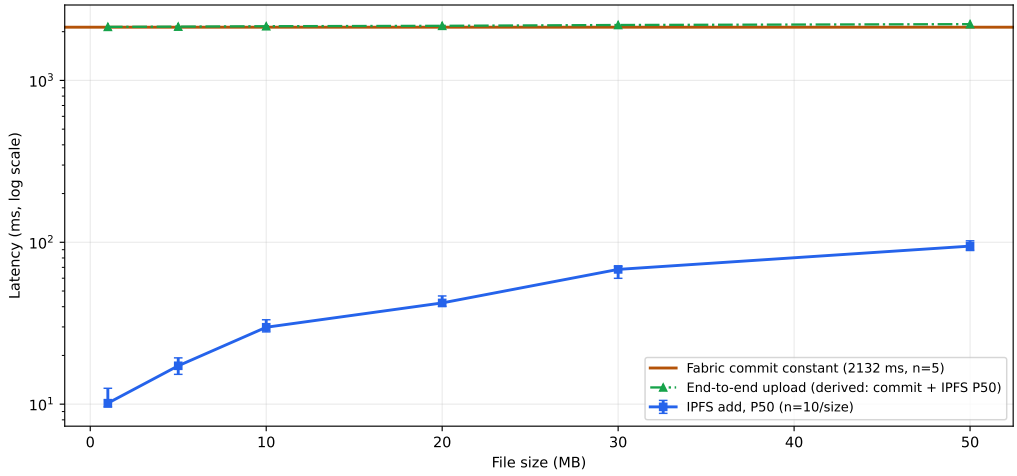


Figure 18: Experiment 3 file-size independence of ledger latency (Linux, direct Kubo API; bootstrap 95 % CIs, n=10 per size): IPFS upload P50 (10–95 ms across 1–50 MB), the constant Fabric commit latency (2,132 ms band), and the derived end-to-end upload cost (commit + IPFS P50), matching the 2,142–2,226 ms range reported in the text.

manual audit additionally requires human download, inspection, and cross-referencing.

The automated PostgreSQL query completed in 3.9 ms median (mean 3.45 ms; n=10 trials, conventional median as the average of the 5th and 6th sorted values). Manual script verification took 86.3 ms, 22× slower in raw compute, and orders of magnitude slower in practice. The raw speedup, however, is not the point. The architectural contribution is the *complementary independence* of the two stores. PostgreSQL provides sub-5 ms operational dashboards with application/database-trigger-level append-only protection: row-level triggers block UPDATE and DELETE; a statement-level trigger blocks TRUNCATE (which bypasses row-level triggers in PostgreSQL); and TRUNCATE privilege is revoked from all application roles (Listing 6). The Fabric ledger provides cryptographic non-repudiation that PostgreSQL cannot replicate: any consortium peer can independently verify the complete audit trail via `GetHistoryForKey` without trusting the application server or its administrator [10, 12]. Compromise of one store does not affect the other. A malicious DB administrator who truncates the PostgreSQL log leaves the on-chain Fabric record intact and independently verifiable, as compared in Fig. 19.

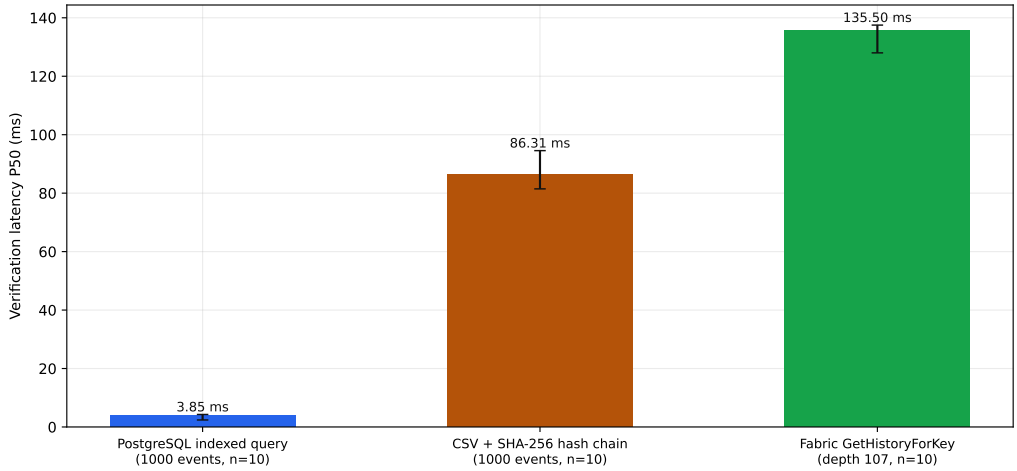


Figure 19: Experiment 4 audit verification efficiency (P50 with bootstrap 95 % CIs, n=10 each): PostgreSQL indexed queries, manual CSV + SHA-256 chain verification, and Fabric `GetHistoryForKey` ledger-verifiable history (depth 107, from Experiment 7).

6.5. Experiment 5: Resilience Under WAN Latency

WAN conditions were simulated in two configurations using Linux `tc netem`. The first configuration injected symmetric delay on the Docker bridge interface (`br-29a499b93a83`), affecting host-to-container HTTP connections at 0, 50, 100, and 150 ms RTT. The second configuration additionally applied `tc netem` delay to the `veth` interface of each orderer container, so that inter-orderer Raft traffic also experienced the injected latency; this configuration was measured at all four RTT levels. Throughput was measured at 200 concurrent clients (five 60-second trials per RTT level); commit latency was measured via 20 CLI samples of `RegisterDocument -waitForEvent` per level.

Bridge-only results (0–150 ms RTT). Throughput degraded from 68.2 TPS at 0 ms to 60.8 TPS at 150 ms RTT, an 11.0 % reduction. It remained at least 3.6× above the 16.7 TPS legal synthetic steady-state baseline at every tested level.

Commit latency grew from 2,140 ms to 2,457 ms (+317 ms at 150 ms RTT), consistent with ≈2 HTTP round-trips through the injected delay. The intermediate points (+100 ms at 50 ms RTT and +217 ms at 100 ms RTT) confirm predictable growth. This configuration isolates HTTP-layer gateway queuing because the injected delay does not reach container-to-container

Table 10: Experiment 5 WAN Resilience: bridge-only delay vs. bridge plus per-orderer-container `veth` delay (200 clients, 5 trials per RTT level).

RTT	**Mean TPS (200 clients)**		**P50 Commit (ms)**	
	Bridge	Bridge + veth	Bridge	Bridge + veth
0 ms	68.2	69.5	2,140	2,141
50 ms	64.9	50.9	2,240	2,919
100 ms	62.3	47.0	2,357	3,132
150 ms	60.8	38.3	2,457	3,588

Fabric peer traffic (Fig. 20).

Bridge plus per-orderer-container results (0–150 ms RTT). With delay also applied to each orderer container's `veth` interface, the 0 ms baseline was 69.5 TPS and 2,141 ms P50 commit latency. These values match the Fabric commit constant of $2{,}132 \pm 20$ ms from Experiment 3.

Throughput declined to 50.9 TPS at 50 ms RTT, 47.0 TPS at 100 ms RTT, and 38.3 TPS at 150 ms RTT, a 44.8 % total reduction from baseline. P50 commit latency grew to 2,919 ms (+778 ms), 3,132 ms (+991 ms), and 3,588 ms (+1,447 ms), respectively. The larger degradation relative to bridge-only delay reflects Raft AppendEntries round-trip latency, which the per-container delay injection captures.

At the highest-RTT, most-delayed point, some requests exceeded the 10 s client timeout; reported TPS is therefore successful-commit throughput. Even there, 38.3 TPS remains 2.3× above the 16.7 TPS baseline, and 3,588 ms remains 1.4 s below the 5 s write budget. The threshold is not approached anywhere in the tested range [12, 89]. Table 10 summarizes both configurations.

6.6. Experiment 6: Cryptographic Primitive Benchmark

All cryptographic operations in the framework are intended to execute client-side in the browser. Browser automation was unavailable in the measurement environment, so this experiment uses Node.js WebCrypto as a same-API fallback rather than claiming browser-performance results. The benchmark was executed on Linux with Node.js WebCrypto and produced 170 CSV rows: 17 measured operation/configuration entries with 10 trials each. Fig. 21 reports the operations most relevant to the paper's document workflow. The

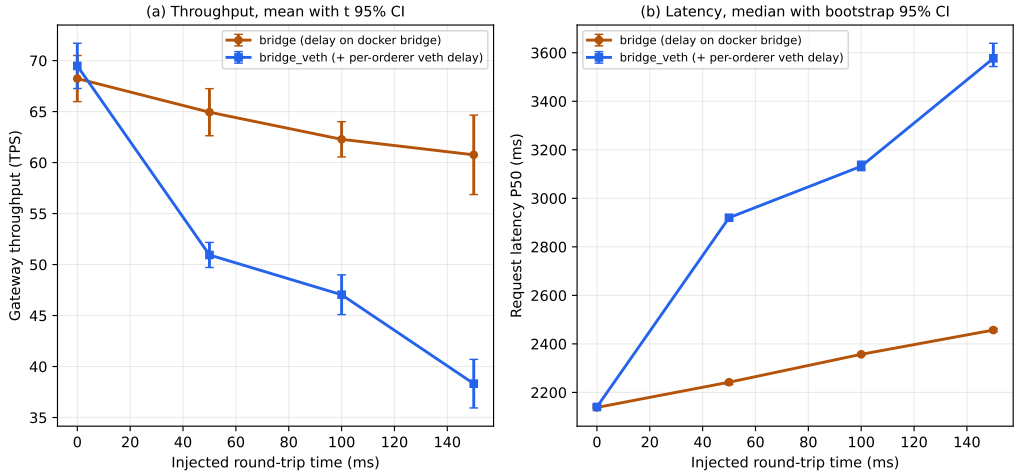


Figure 20: Experiment 5 WAN resilience (`tc netem`, Linux `x86_64`, 200 clients; full 8-point sweep, n=5 per point): (a) throughput (mean, t 95 % CI) and (b) request latency (P50, bootstrap 95 % CI) for bridge-only vs. bridge + `veth` delay injection.

goal is to verify that password-based key derivation, document encryption, hashing, signing, verification, and key wrapping are not the dominant costs in the measured system.

PBKDF2-SHA256 at 600,000 iterations completed with a P50 latency of 100.44 ms over 10 trials, within the 1,000 ms interaction budget commonly used for preserving user flow and consistent with the work-factor guidance used in the framework [17, 66]. For 50 MB inputs, AES-256-GCM encryption measured 44.42 ms P50, AES-256-GCM decryption measured 61.94 ms P50, and SHA-256 hashing measured 44.43 ms P50. These values confirm that cryptographic processing is small relative to the Fabric write latency dominated by the 2 s `BatchTimeout`.

ECDSA P-256 document signing and verification were sub-millisecond in the same Node WebCrypto benchmark: signing measured 0.183 ms P50 and verification measured 0.097 ms P50. P-256 hybrid key wrapping measured 0.403 ms P50 and unwrapping measured 0.603 ms P50. The hybrid wrap is used primarily for compact per-recipient token storage rather than for a speed advantage: both Hybrid-wrap and RSA wrapping costs are sub-millisecond in the Node WebCrypto benchmark. The hybrid-wrap token occupies 125 bytes, compared with 256 bytes for RSA-OAEP 2048, a 51.2 % reduction that reduces ledger and network storage per access-grant event.

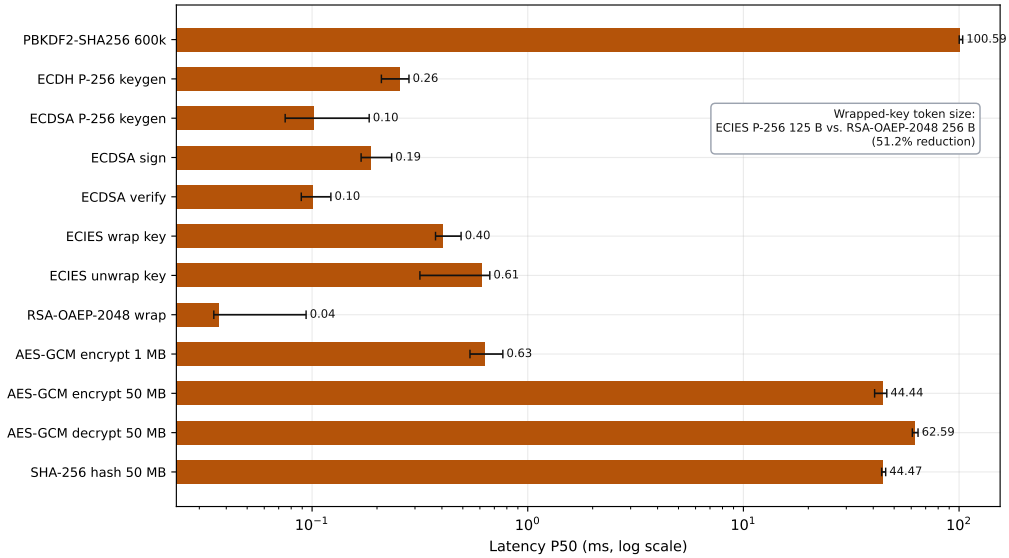


Figure 21: Experiment 6 cryptographic primitive benchmark using Node.js WebCrypto fallback (Linux; P50 with bootstrap 95 % CIs, n=10, log scale): PBKDF2-SHA256, ECDH/ECDSA operations, hybrid-wrap and RSA-OAEP key wrapping, AES-GCM, and SHA-256, with the wrapped-key token-size comparison (hybrid wrap 125 B vs. RSA-OAEP-2048 256 B) annotated.

That comparison should not be read as a claim of state-of-the-art compactness, because RSA-OAEP 2048 is a weak baseline to beat in 2026. An HPKE token under DHKEM(X25519, HKDF-SHA-256) with AES-128-GCM (RFC 9180) [62] would occupy roughly 80 bytes for the same payload — about 36 % *smaller* than the construction measured here. Most of the difference is encoding rather than cryptography: this prototype serialises an uncompressed P-256 point (65 bytes, reducible to 33 compressed) and an explicit 12-byte nonce that HPKE derives from the key schedule. The defensible statement is therefore that per-recipient token overhead is small in absolute terms and materially smaller than the RSA baseline it replaced, not that it is minimal.

6.7. Experiment 7: Blockchain Audit Trail Query Depth

Experiment 4 showed that PostgreSQL serves operational dashboards at 3.9 ms P50. The Fabric ledger's `GetHistoryForKey` path is slower, but it is the only measured path that provides cryptographic non-repudiation verifiable by any consortium peer. This experiment therefore measures the

cost of ledger-verifiable history queries at a realistic workflow depth.

A benchmark document was seeded with 107 committed history entries: one `RegisterDocument` transaction followed by 106 `GrantAccess` invocations, each committed in a separate block. This depth represents an actively shared document with periodic access-grant renewals across multiple firms. Ten trials were executed with `peer chaincode query` (CLI), measuring the full round trip from peer request, to CouchDB history scan, to response.

`GetHistoryForKey` completed in 135.5 ms P50 (mean 134.0 ms, range 123–145 ms, $\sigma = 6.4$ ms). The coefficient of variation is $<5\,\%$, indicating stable performance with no observed index-scan anomalies. Because the audit-history measurement was obtained through a peer chaincode query CLI path, whereas the main latency and throughput experiments use the REST-gateway workload path, these measurements are reported as distinct evaluation paths and should not be interpreted as identical workloads. Only the 107-entry depth was directly measured.

CouchDB evaluates `GetHistoryForKey` with a full sequential scan and no secondary index, so latency is expected to grow approximately linearly with history depth. Fig. 22 therefore shows only the measured depth: all ten trials at 107 entries with the median and its bootstrap 95 % confidence interval. As a back-of-envelope linear extrapolation (stated in text, not plotted), 135.5 ms at 107 entries implies ≈1,266 ms at 1,000 entries: suitable for background forensic reports, but past the 200 ms interactive threshold, which would be crossed at approximately 158 entries. Multi-depth validation across deeper histories is left as future work.

For documents with deeper histories, operational queries should use the PostgreSQL `audit_log` path measured in Experiment 4 (3.9 ms P50). The Fabric path is reserved for contexts requiring cryptographic non-repudiation that no application-layer log can provide, and is the core answer to RQ1 [4, 84].

6.8. Experiment 8: BatchTimeout Sensitivity

Experiment 1 established that throughput at the default `BatchTimeout=2s` is gated by the Raft batch window rather than by concurrency. This experiment characterizes the throughput–latency trade-off as `BatchTimeout` is reduced, and tests whether the load generator, not the orderer, becomes the limiting factor at higher rates. At 50 concurrent clients, the network was rebuilt at each of 2 s, 500 ms, and 250 ms (`MaxMessageCount` fixed at 500), and ten 60-second sustained-throughput trials were taken per setting using

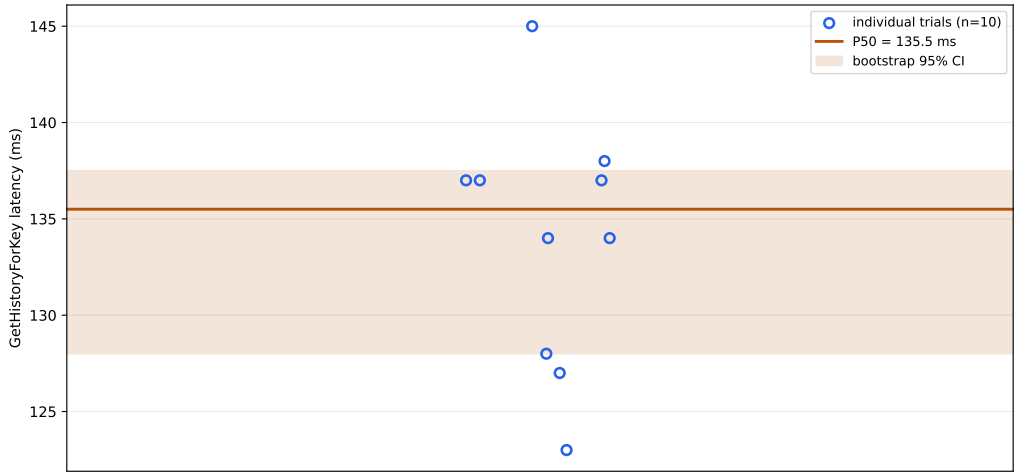


Figure 22: Experiment 7 Fabric `GetHistoryForKey` latency at the measured 107-entry history depth: all ten individual trials (n=10) with the median (135.5 ms) and its bootstrap 95 % confidence interval. Only measured data are shown; no extrapolation to deeper histories is plotted. Multi-depth validation is future work.

the closed-loop load generator. The load generator's own CPU utilization was recorded each run to detect client-side saturation.

Sustained throughput was 67.7 TPS at 2 s (95 % CI [66.3, 69.0]), rose to 193.0 TPS at 500 ms, and then fell to 180 TPS at 250 ms. The 500 ms setting is therefore the sustained-throughput sweet spot, providing an 11.6× margin over the 16.7 TPS synthetic steady-state baseline without architectural change.

The regression at 250 ms is reproducible across all ten trials and is *not* a load-generator artifact. Client CPU remained far from saturation: approximately 2 %, 11 %, and 14 % for the 2 s, 500 ms, and 250 ms settings, respectively. The likely cause is internal to the orderer: a 250 ms batch window cuts roughly twice as many small blocks as a 500 ms window, increasing per-block commit and validation overhead. The 250 ms setting also shows higher variance (137–206 TPS across trials) than 500 ms (174–217 TPS), slightly lowering net sustained throughput.

This resolves an apparent tool-dependent discrepancy. A fixed-count harness ranks 250 ms above 500 ms because it favors per-request latency and batch completion time. The sustained closed-loop measurement reported here is the representative throughput metric and identifies 500 ms as the

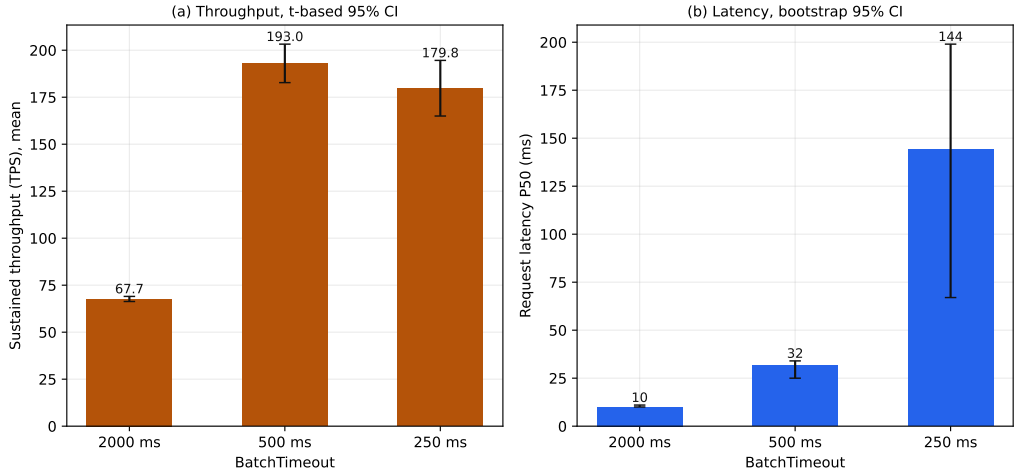


Figure 23: Experiment 8 `BatchTimeout` sensitivity (Linux `x86_64`, 50 clients; n=10 per setting, zero-error runs): (a) sustained throughput (mean, t 95 % CI) and (b) request latency (P50, bootstrap 95 % CI) at 2 s, 500 ms, and 250 ms, showing the reproducible 500 ms optimum.

operational optimum shown in Fig. 23.

6.9. Experiment 9: Fail-Closed Security Under Fabric Peer Outage

The threat model (Section 4.1) specifies a strict *fail-closed* architecture for protected document release during network partitions: if the Fabric peers become unreachable, the evaluated download path denies access rather than falling back to a database ACL. This experiment validates that behavior empirically. Under a steady 50-client ciphertext-download load, all three Fabric peers were stopped at $t = 30$ s and restarted at $t = 75$ s, a 45-second total Fabric peer outage. Fig. 24 shows the per-second timeline.

Successful downloads dropped to exactly zero requests per second for the entire outage window; peers were restarted at $t = 75$ s, with the first successful download at $t = 95$ s, an approximately 20-second recovery lag whose dominant component is the access-path circuit breaker (resilience4j, 30 s open-state window with automatic half-open transition) completing its open interval before probe calls readmit Fabric, with gRPC reconnection contributing the remainder.

During the outage, the service issued HTTP 503 fail-closed denials at approximately 30 per second on average (peaking at 44), securely blocking all

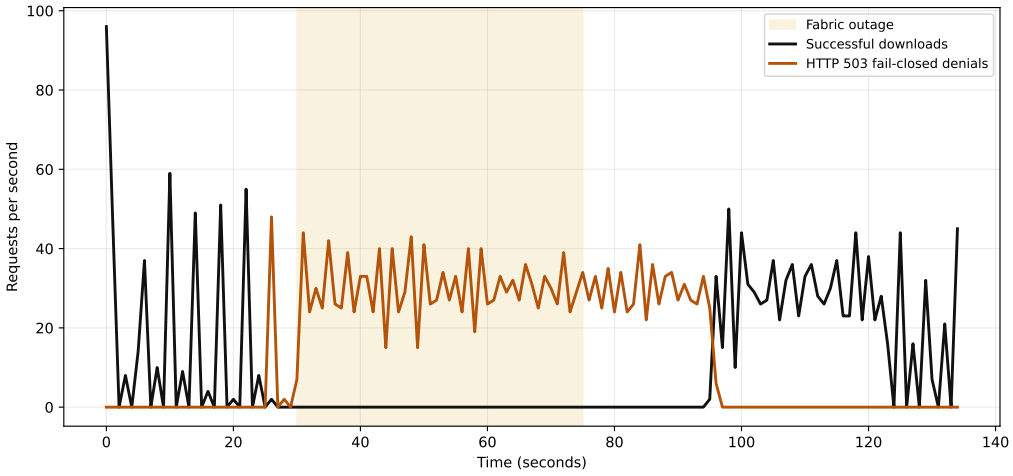


Figure 24: Experiment 9 fail-closed Fabric peer outage (computed from the released raw data): zero successful downloads during the 45 s outage window; 1,340 HTTP 503 denial responses and 2,013 PostgreSQL audit rows (the difference reflects additional system-level audit events per denial). Denials visibly persist ≈20 s past peer restart ($t = 75$–95 s): the access-path circuit breaker completes its 30 s open-state window before half-open probes readmit Fabric calls, after which normal operation resumes without intervention.

tested download requests (1,340 denial responses and 2,013 PostgreSQL audit rows; the difference reflects additional system-level audit events recorded per denial beyond the HTTP response itself). `FABRIC_OUTAGE_ACCESS_DENIED` audit rows were written to the operational PostgreSQL log capturing the gRPC failure reason. On peer restart, normal Fabric evaluation resumed cleanly with no manual intervention.

This result substantiates the fail-closed behavior for the evaluated document-download path: a peer outage degrades to a secure denial state, prioritizing confidentiality over availability. A malicious DB administrator who deliberately induces an outage cannot force a fallback to the database ACL to bypass chaincode authorization on this path.

6.10. Experiment 10: IPFS Storage and Retrieval Cost

Experiments 1–9 exercise the storage tier only on the upload path (Experiment 3). This experiment completes the storage-tier picture on a three-node private Kubo v0.27 swarm: retrieval latency versus file size (1–50 MB, matching Experiment 3's range) under 1–24 concurrent cold fetches, replication cost for a second and third pinned replica, storage overhead, and behavior under

node failure (Fig. 25). Each retrieval targets a distinct content identifier so concurrent fetches are independent transfers; all latencies are LAN-scale (docker bridge) and compose with the synthetic WAN model of Experiment 5 for wide-area estimates.

Retrieval latency is linear in file size, with a 2–3.5× premium when the serving node must fetch blocks over the swarm rather than reading locally (50 MB: 254 ms local vs. 875 ms remote P50 at concurrency 1, rising to 7.0 s at 24 concurrent cold transfers with zero failures across 576 requests). Replicas are independent: pinning a third copy costs approximately the same transfer time as the second, so k-node replication scales linearly in total transfer cost. The UnixFS DAG imposes a constant 0.024 % storage overhead at every measured size (256 KiB chunking), plus a 46-byte content identifier per document. Most importantly for availability governance: with a document pinned on two of three nodes, stopping one replica produced no measurable retrieval penalty (P50 206 ms vs. 204 ms baseline, 10/10 served), stopping all replicas produced clean timeouts with zero bytes served, and service resumed without manual repair after restart: the storage-tier counterpart of Experiment 9's fail-closed result on the ledger tier.

6.11. Experiment 11: Standard-Instrument Benchmark (Hyperledger Caliper)

The custom REST-gateway generator measures the application path users actually traverse, but most published permissioned-blockchain numbers are produced by Hyperledger Caliper [85]. To anchor comparability, this experiment benchmarks the deployed chaincode directly with Caliper 0.6 through the Fabric peer-gateway API (the same client path as the Spring gateway), sweeping fixed loads of 50–600 in-flight transactions for the two release-path functions (Fig. 26).

`CheckAccess` (evaluate, read path) reaches 875 TPS at load 600 with a flat 20 ms average latency and zero failures, confirming that the ledger-side access decision is not the release-path bottleneck. The 20 ms Caliper average and Experiment 2's 10.99 ms P50 measure different regimes (600 concurrent in-flight transactions on the direct peer-gateway path versus a single sequential client through the REST gateway) and are complementary rather than conflicting. `RegisterDocument` (submit, write path) climbs to 172 TPS, approaching the ≈193 TPS ordering ceiling of Experiment 8, with write latency pinned at 1.7–2.0 s by the 2 s `BatchTimeout` at every load, an independent standard-instrument corroboration of the custom generator's saturation analysis. At loads ≥500 the peer gateway's default admission

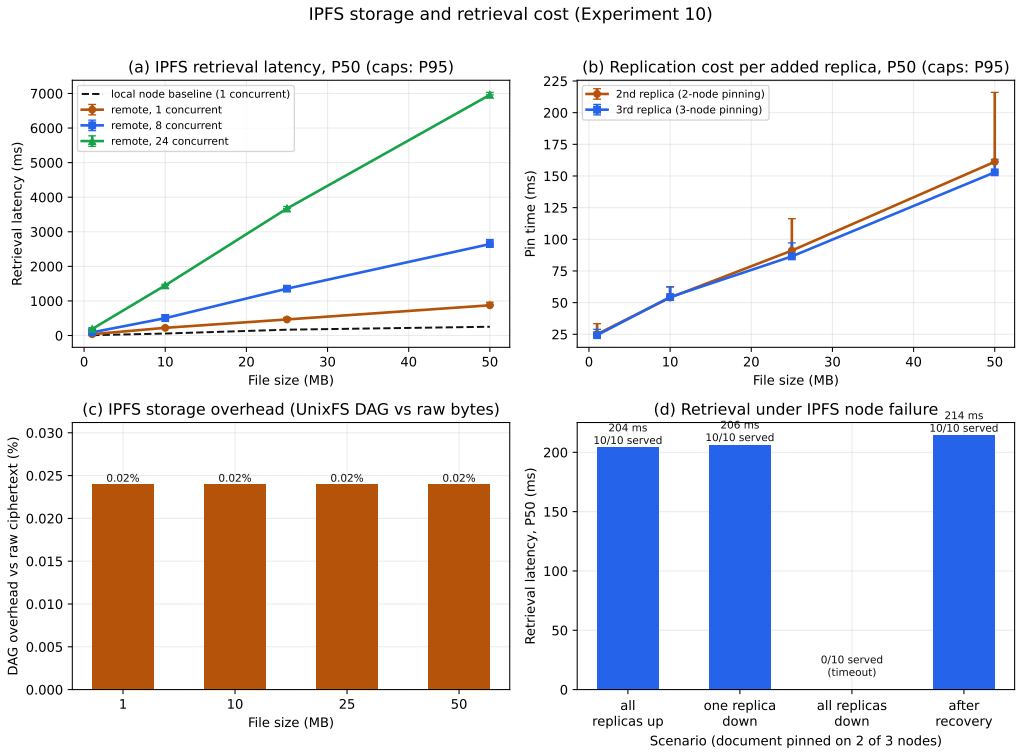


Figure 25: Experiment 10 IPFS storage/retrieval cost: (a) retrieval latency vs. file size and concurrency (local vs. cross-node); (b) replication cost per added replica; (c) constant 0.024 % UnixFS DAG overhead; (d) retrieval under node failure with a document pinned on 2 of 3 nodes.

control (`peer.limits.concurrency.gatewayService`=500) rejects excess in-flight submissions cleanly (3.7 % at load 500, 27 % at load 600); the limit is a documented, tunable Fabric default that was deliberately left at the value used throughout Experiments 1–9.

6.12. Experiment 12: Document-Volume and Ledger-Growth Scaling

Experiment 7 measured history depth on a single key; this experiment measures the orthogonal axis: how the release-path queries and storage footprint behave as the *number of documents* grows. Starting from a fresh ledger, 1,000,000 documents were registered through the peer gateway (200 closed-loop submitters, 70–76 TPS sustained, zero failed transactions over the 3.6 h campaign), with `CheckAccess` and `GetDocumentHistory` latency

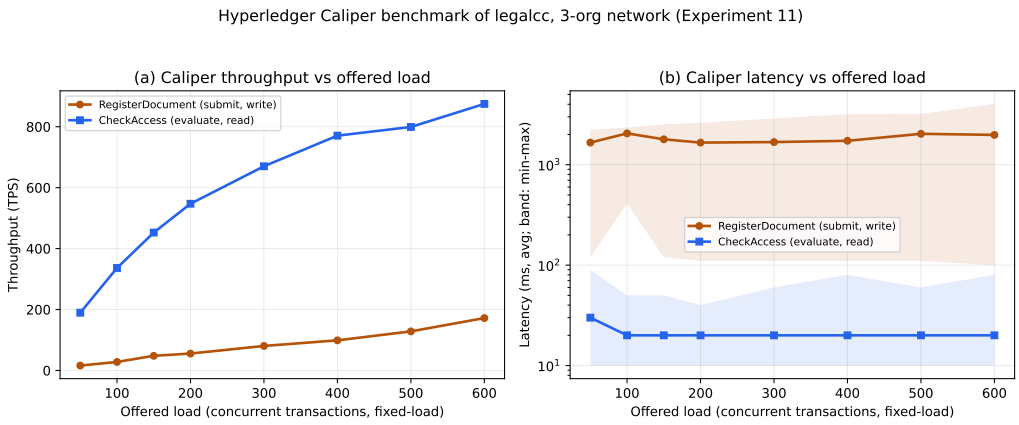


Figure 26: Experiment 11 Hyperledger Caliper benchmark of the deployed chaincode via the peer-gateway API: (a) throughput and (b) latency vs. offered fixed load for `CheckAccess` (evaluate) and `RegisterDocument` (submit). Caliper reports average/min/max latency, the convention used by comparable published systems.

sampled (n=100 each, random keys) and per-peer disk usage recorded at the 10^3, 10^4, 10^5, and 10^6 checkpoints (Fig. 27).

Both queries are effectively flat across three orders of magnitude of world-state growth: `CheckAccess` P50 rises from 7.96 ms at 10^3 documents to 8.60 ms at 10^6 (+8 % for a 1000× state increase). These checkpoints were measured on the build current at the time, whose function-level equivalent was Experiment 2's then-reported 6.51 ms on a near-empty ledger; the read path has since been re-measured at 10.99 ms P50, so the flatness result should be read as an elasticity claim about world-state size rather than as an absolute that carries to the current build. `GetDocumentHistory` rises from 5.68 to 7.12 ms over the same growth, and `GetDocumentHistory` rises from 5.68 to 7.12 ms. Storage grows linearly at ≈5,618 bytes of block store plus 1,389 bytes of CouchDB state per document per peer (≈7 KB/document; the 10^6 measurement lands within 0.1 % of the linear extrapolation from 10^5), i.e., one million documents occupy roughly 7 GB per peer, commodity-hardware territory for a realistic legal archive.

Document count is not the only axis of growth. The TimeAnchor introduced in Experiment 17 refreshes on a fixed schedule, and each refresh is a *submit*, so it occupies a block permanently whether or not any document is registered. Re-measuring on the current build (chaincode v1.19) confirms the per-document constant is unchanged — 5,751 bytes of block store per

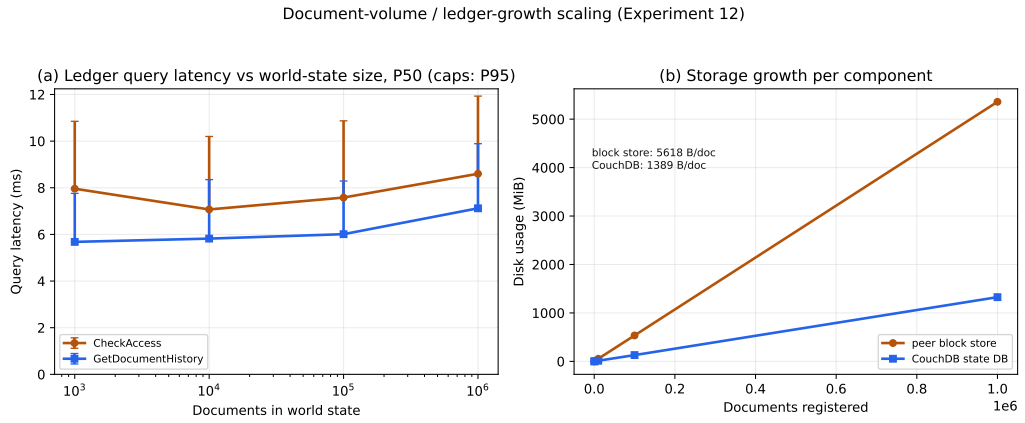


Figure 27: Experiment 12 ledger-growth scaling from 10^3 to 10^6 registered documents: (a) release-path query latency vs. world-state size (P50, caps P95); (b) linear per-peer storage growth (≈7 KB/document across block store and CouchDB).

document, within 2.4 % of the figure above — but with document activity held at zero the ledger still grows at 7.84 MB/day per peer (90.7 B/s, $R^2 = 1.0000$ over 20 min; exactly one block per minute at 5,470 bytes, matching the 60 s default refresh interval). Storage therefore scales as $a \cdot \text{documents} + b \cdot \text{time}$ rather than in documents alone. The time term is not negligible over a retention horizon: at the default interval it contributes ≈2.9 GB per peer per year, so a decade of custody costs roughly 29 GB per peer — about four times what a million documents cost — for an archive that receives nothing further. This is the standing cost of a trustworthy clock rather than a defect, since an anchor that is never refreshed bounds nothing, and it is directly tunable: the interval is a configuration parameter, and lengthening it lowers the storage rate proportionally while loosening the backdating bound that Experiment 17 quantifies. We report the block store only, which is append-only; the CouchDB state database is compacted, so its size is a sawtooth from which no meaningful idle rate can be fitted.

6.13. Experiment 13: Scaling with Network Size and Endorsement Policy

Experiments 1–12 share the reference three-organization topology. This experiment regenerates complete networks at 2, 3, 5, and 7 organizations (one peer each, plus a 3-organization/two-peer point), each with per-peer CouchDB, a 3-node Raft ordering service, and identical batch parameters, and benchmarks every point under both a single-organization endorsement

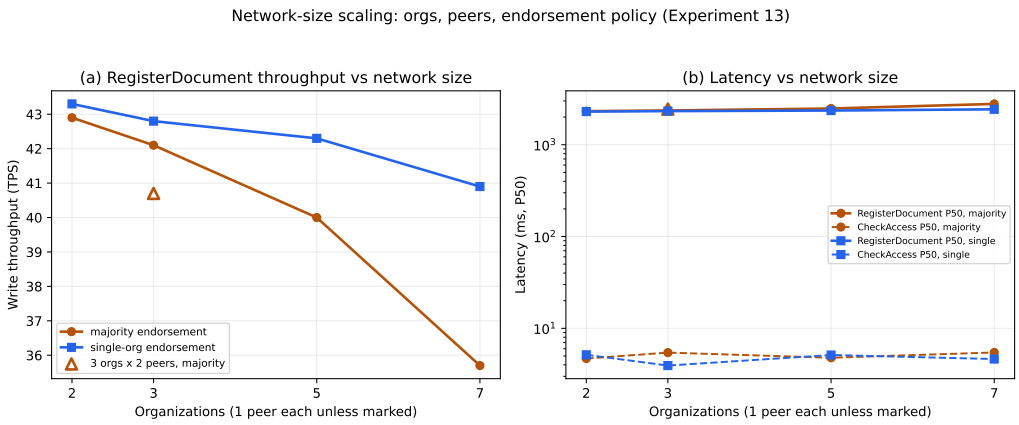


Figure 28: Experiment 13 network-size scaling across 2–7 organizations and endorsement policies: (a) write throughput; (b) write and read latency (log scale). Single-host deployment; inter-point ratios, not absolute TPS, are the meaningful result.

policy and the channel-default majority policy (Fig. 28; 2,000 writes and n=100 reads per point, zero failed transactions at all nine points).

Write throughput is ordering-dominated, not size-dominated: under single-organization endorsement it declines only 5.5 % from 2 to 7 organizations (43.3 to 40.9 TPS) with write latency pinned near the 2 s batch window throughout. The majority-endorsement premium grows gently with consortium size (−0.9 % at 2 organizations, −5.4 % at 5, and −12.7 % at 7, i.e., 35.7 TPS), reflecting parallel endorsement collection that tracks the slowest endorser rather than the sum. Doubling peers per organization costs 3.3 %. Read-path `CheckAccess` latency is size-independent (3.9–5.5 ms P50 at every topology and policy), extending Experiment 12's flatness result from the volume axis to the topology axis. One measurement artifact is documented in the released data: the first 7-organization/majority run produced an implausible read-latency inflation attributable to transient host contention and was re-run in isolation; both runs are preserved, and the verified rerun is reported here.

6.14. Experiment 14: Price of On-Path Enforcement vs. a Passive-Audit-Log Baseline

The architectural claim of this paper is that the ledger belongs *on* the document-release path rather than beside it as a passive audit log. This experiment prices that claim end to end. A default-off Spring profile (`audit-log-only`) implements the opposing architecture in the same

codebase: the PostgreSQL ACL decides, and the decision is recorded through the existing asynchronous audit pipeline, which anchors a `LogAuditEvent` transaction to Fabric off the request path. The identical ciphertext-download workload (closed loop, 10–200 concurrent clients, 2,000 requests per level) was run against both modes; 16,000 requests completed with zero failures (Fig. 29).

On-path enforcement costs +6.5 to +9.1 ms P50 per release at 10–100 concurrent clients (e.g., 21.6 vs. 15.1 ms at concurrency 10) and at most 10 % throughput at saturation; at 200 clients the on-path P95 is actually *below* the baseline's. The end-to-end premium independently corroborated Experiment 2's then-current function-level figure on a different build five weeks later. These figures characterise the build measured at the time; the release path has since acquired the time-anchor freshness read (Experiment 17), lost the organization fallback, and begun anchoring denials, so they should be read as belonging to that configuration rather than to the current one.

Preparing this baseline exposed a resource-management defect in the prototype, disclosed here rather than in a footnote because it bears on how the comparison should be read: Spring's `@Async` audit anchoring had silently fallen back to an unbounded thread-per-task executor, capping the gateway at ≈14 requests/s under sustained download load. The defect was fixed (bounded executor) before either mode was measured. Experiments 1–9 are unaffected, because their workload never triggers the pathological path: `GET /wrapped-key` writes no audit event, and upload audits carry a transaction identifier that skips ledger anchoring.

The baseline as measured above nonetheless anchors *fire and forget*. Its surplus anchors accumulate in an in-memory queue and are lost when the process stops, which is a property of that implementation rather than of the audit-log-only architecture: any serious deployment of it would use a durable queue and batched anchoring. Pricing the architecture against an implementation that discards its own audit trail would not be a fair comparison, so the baseline was rebuilt with a durable, batched anchor pipeline and the workload re-run against all three modes (Experiment 14b).

6.15. Experiment 14b: Against a Baseline That Keeps Its Audit Promise

The audit-log-only mode of Experiment 14 anchors fire and forget, so its audit trail is only as complete as its queue happens to drain. Comparing on-path enforcement against that is comparing against an implementation that does not deliver what its architecture promises. This experiment rebuilds

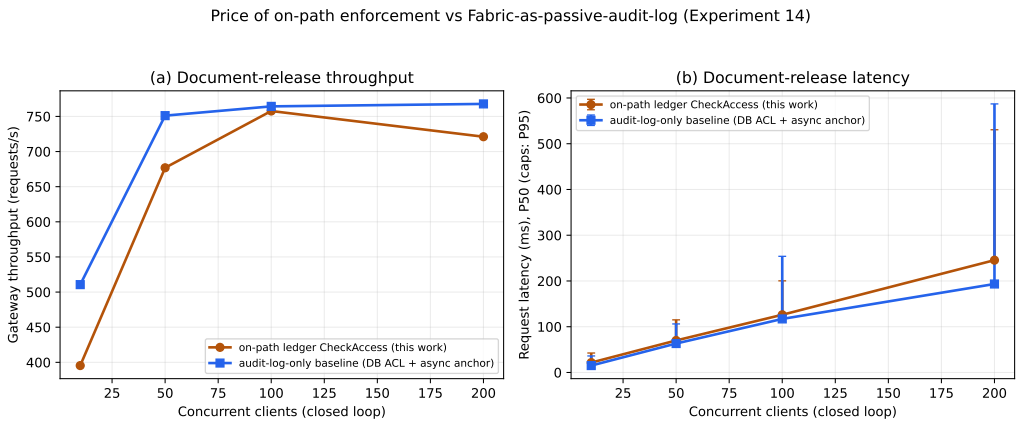


Figure 29: Experiment 14 on-path enforcement vs. the audit-log-only baseline on the identical download workload: (a) gateway throughput; (b) request latency (P50, caps P95). The architectural premium of on-path enforcement is single-digit milliseconds on the build measured here; as with the body text, these values predate the time-anchor freshness read, the removal of the organization fallback, and denial anchoring, and belong to that configuration rather than the current one.

the baseline with a durable, batched anchor pipeline and re-runs the same workload against three modes.

Durability here is cheap to obtain and does not require a new store. Audit rows are committed to PostgreSQL before any anchoring is attempted, so the backlog is already on disk; a checkpoint table records which contiguous range of audit records a given ledger transaction covers, and the highest committed range is a watermark above which rows are simply not yet anchored. Anchoring is batched — a digest over a batch is anchored in one transaction rather than one transaction per event, since per-event anchoring would cap throughput at the commit latency — and tamper evidence is preserved, because altering any record in an anchored batch changes the digest. The checkpoint is kept beside the audit log rather than inside it, since the log is append-only at the database level and relaxing that constraint to record anchoring status would trade away the very property being protected.

On a reduced sweep (concurrency 10 and 50, 1,000 requests per level, zero failures in every mode; Fig. 30):

	concurrency 10		concurrency 50	
Mode	**TPS**	**P50**	**TPS**	**P50**
On-path enforcement	469.5	16.6 ms	789.6	59.2 ms
Audit-log-only, fire and forget	393.4	15.6 ms	557.9	57.3 ms
Audit-log-only, durable batched	142.3	32.8 ms	226.9	209.3 ms

The comparison inverts. Experiment 14 found the baseline faster than on-path enforcement and framed the difference as a premium paid for a stronger guarantee. Against a baseline that actually retains its audit trail, on-path enforcement instead delivers 3.3× the throughput and half the median latency at concurrency 10, and 3.5× the throughput at concurrency 50.

The reason is structural rather than an artefact of tuning. In the audit-log-only architecture the anchor *is* the security mechanism: if it does not reach the ledger there is no ledger-verifiable record of the decision, so the pipeline must be durable and must sustain the request rate. In the on-path design the ledger decision is taken before release, so the anchor is telemetry rather than the enforcement mechanism and need not be durable for the security property to hold. One architecture pays for durability on every decision; the other does not have to. Anchor retention follows the same split: after the run the fire-and-forget mode left 69 records unanchored and lost them at process exit, while the durable mode left 16 in flight, on disk, across 318 committed batches.

Two qualifications. The sweep is reduced, so the saturation behaviour Experiment 14 reports at 100–200 clients is not re-validated here. And the on-path mode's own audit events are deliberately *not* durably anchored, which is precisely the asymmetry being argued: the comparison is between architectures as each must be built to keep its own promise, not between identical pipelines.

6.16. Experiment 15: Consolidated Evidence and Positioning Against Reported Numbers

To counter point estimates masking variance, the released dataset consolidates 37 headline metrics across all fifteen measurement experiments (Experiments 1–15; Experiments 14b, 16, 16b, 17, and 18 are behavioral and contribute no headline metric) with 95 % confidence intervals: bootstrap percentile intervals (10,000 resamples) for medians computed from per-sample

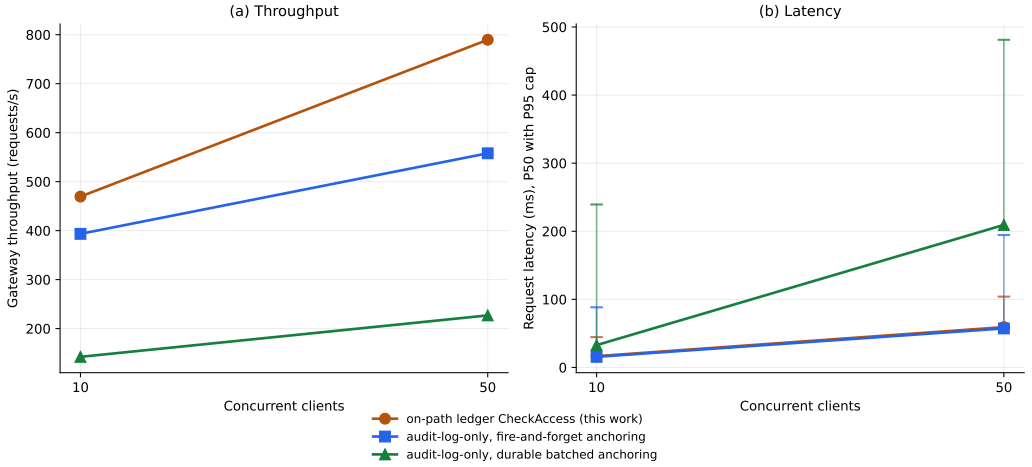


Figure 30: Experiment 14b: on-path enforcement against both audit-log-only baselines on the identical download workload. (a) gateway throughput; (b) request latency, P50 with P95 cap. Making the baseline durable — so that it actually retains the audit trail its architecture promises — reverses the comparison reported in Experiment 14: the fire-and-forget baseline is competitive with on-path enforcement, while the durable one is roughly a third of its throughput. Axes are ticked only at the two measured concurrency levels; this is a reduced sweep and does not re-validate the saturation behaviour reported at 100–200 clients.

data, and Student-t intervals for means of trial-level throughput.[1] Two intervals carry the paper's central claims. First, the function-level ACL comparison of Experiment 2 yields *disjoint* intervals (Fabric `CheckAccess` 10.99 ms [10.71, 11.25], n=200, vs. PostgreSQL 6.44 ms [6.21, 6.70], n=100), so the on-path cost is interval-supported rather than merely asserted, and so is its containment well inside the 50 ms budget. Second, Experiment 14's end-to-end premium is significant but small: 21.62 ms [21.39, 21.84] vs. 15.09 ms [14.84, 15.36] at concurrency 10 (n=2,000 each). Across the independent setups of Experiments 12, 13, and 14 the ledger access decision lands consistently at 4–9 ms P50; Experiment 2's re-measurement on the current build sits above that

[1]Medians in the consolidated dataset and in the figure annotations are computed with the central-pair-averaging convention (`statistics.median`) over the released per-sample data, whereas P50 values quoted in the prose retain the floor-index percentile convention of the per-experiment run summaries in the released dataset. The two conventions can differ marginally (e.g., 100.59 vs. 100.44 ms for the PBKDF2 annotation in Fig. 21); the prose uses the run-summary convention consistently throughout.

Table 11: Reported performance of the closest published systems versus this work. Values are each paper's own published figures on its own testbed (no hardware normalization); statistic conventions differ as noted. Pre-proof values for [20, 35] are subject to change in the versions of record.

System	Platform / instrument	Read path	Write path	Testbed
RBAC-IPFS [20]	Fabric 2.4.9 + IPFS, 10 peers; Caliper	queryCid 159.8 ms client mean (50 ms emulated WAN); on-path retrieval authorization 146–156 ms e2e	≈356–378 TPS saturated; ≈450 TPS mixed workload	Xeon Gold, 128 GB RAM
Mukta et al. [21]	Fabric, 4 peers; Caliper	verifyDC peak 29.4 TPS @75 users	createDC 25.0 TPS, delegateDC 24.2 TPS peaks	4-core, 8 GB
Liu and Zheng [27]	FISCO BCOS + IPFS; java-sdk bench	not reported (send/confirm times tabulated)	peak 247 TPS	2-core, 4 GB
Peelam et al. [34]	Private Ethereum (PoS) + IPFS + fog; custom simulation	not reported	transaction delays avg 24.5 s (±3.6–52.7 s at 95 %)	Core i5 (8 GB) and i9 (32 GB) hosts
This work (gateway path)	Fabric 2.4 + IPFS, 3 orgs; custom REST loadgen	`CheckAccess` 10.99 ms P50 [10.71, 11.25], n=200	66–70 TPS @2 s batch; 193.0 TPS [182.8, 203.2] @500 ms batch	Core Ultra 5, 8 GB
This work (chaincode direct)	same network; **Caliper 0.6**	`CheckAccess` 874.9 TPS @load 600, 20 ms avg	`RegisterDocument` 172.0 TPS @load 600, ≈2 s avg	same

band at 11.0 ms. The three older figures were taken on earlier builds, so the spread reflects build and harness differences rather than variance within one configuration, and only the paired within-run differences should be compared across them.

Table 11 positions the measured numbers against the figures reported by the closest published systems. The numbers are *not* hardware-normalized: each row reports the system's own published values on its own testbed, so order-of-magnitude positioning and measurement coverage, not a throughput race, are the comparison targets. Where prior systems used Caliper, the instrument-matched row is Experiment 11.

6.17. Experiment 16: Divergence Under Orderer-Only Outage

Experiment 9 stops all peers, so both the read and write paths fail and the download path denies. That geometry cannot reach the asymmetric failure

the architecture is exposed to: an outage of the *ordering service* while peers stay reachable. `CheckAccess` is a Fabric evaluate answered by a peer, whereas `RevokeAccess` is a submit ordered by the orderer. This experiment isolates that case. Against a fresh cross-organization grant, the three orderers were stopped (peers left running), an owner revocation was issued, and the release path was re-exercised.

The revocation returned HTTP 204 to the caller yet never reached the ledger: the prototype's write paths are availability-first (Section 7), so `RevokeAccess` caught the submit failure, committed the PostgreSQL revocation, and returned success. The ledger grant therefore remained `StatusActive`, and `CheckAccess` (evaluated against the still-reachable peer) continued to authorize the revoked user: the ciphertext download returned HTTP 200 and served the document bytes after the revocation, while the wrapped-key endpoint returned HTTP 403 because it is gated by the operational access row, which the revocation did update. The exposure under this geometry is thus bounded to ciphertext, not the document key; but a prior grantee already holds the wrapped key from legitimate access, so the revocation fails against exactly the party it targets. Restarting the orderers did not reconcile the state: with no automatic re-anchoring, the ledger continued to authorize the revoked user until the revocation was manually re-issued and committed while Fabric was reachable. This confirms the write-path divergence documented in Section 7 as an empirical result and delimits its scope; the raw sequence is in the released dataset. The permanence of that divergence—the property that distinguishes a recovery lag from an indefinite failure—motivated the durable re-anchoring evaluated next.

6.18. Experiment 16b: Bounded Reconciliation After Durable Re-Anchoring

Experiment 16 shows that the divergence it measures is permanent rather than window-bounded. This experiment re-runs the identical geometry—same fixture generator, same three-orderer outage with peers left running, same fresh cross-organization grant—against a revocation path modified to anchor durably. `RevokeAccess` now writes the operational revocation and an outbox row in one database transaction, a scheduled worker re-submits queued anchors with capped exponential backoff, and the endpoint returns HTTP 202 with an explicit pending ledger-sync status while an anchor is outstanding, reserving HTTP 204 for anchors that have committed. The procedural difference is in the recovery step: rather than re-issuing the revocation manually, the run

restarts the orderers and then waits, taking no further action, until both the chaincode decision and the outbox row indicate convergence.

Five runs were executed (Fig. 31). In every run the revocation returned HTTP 202 with a pending status during the outage; the ledger, database, and outbox were observed to disagree as expected mid-outage; and after the orderers were restarted the ledger reconverged with no operator action, after which the release path denied the revoked user (HTTP 403, where Experiment 16 remained at HTTP 200 indefinitely). Measured from the outbox row's own timestamps, convergence took a median of 15.9 s (mean 15.85 s, SD 0.69 s, range 14.86–16.51 s, $n = 5$), with every run committing on its first retry.

This figure should be read as a recovery lag, not as an exposure bound. The divergence window is the sum of the outage duration and the reconciliation lag, and only the latter is under the system's control; because these runs restart the orderers within seconds, the reported median is dominated by Raft leader re-election. A longer outage yields a proportionally longer window. What the result establishes is narrower and more specific: the divergence no longer outlives the outage. The exposure *during* the outage—ciphertext release to a revoked holder of a previously distributed wrapped key—is unchanged from Experiment 16, and the retry schedule is not exercised near its bound here, since no run required more than one retry.

6.19. Experiment 17: Cost of Bounding Clock Trust on the Evaluate Path

Grant expiry is evaluated inside `CheckAccess` against `GetTxTimestamp()`. On an evaluate that value originates in the caller's proposal, and under custodial submission the caller is the gateway (Assumption A7, Scenario S5). This experiment prices the mechanism that bounds the resulting exposure: a *time anchor*, refreshed by a periodic `UpdateTimeAnchor` submit. Because the heartbeat is a submit, each endorsing peer independently validates its timestamp against that peer's own clock before endorsing, so the committed anchor is a reference the gateway cannot rewind; `CheckAccess` then refuses any proposal more than a fixed tolerance behind it. Only the accept-or-reject decision consults an endorser's local clock — the value written to state is the proposal timestamp, identical across endorsers — so the read-write set stays deterministic and endorsement is unaffected.

Latency cost. The check adds one world-state read to every access decision, on the path carrying this paper's headline enforcement-cost claim, so it was measured rather than assumed. Two chaincode deployments differing only

in whether that read executes were compared back to back on the same fixture and harness. The added read costs **0.78 ms at the median** (7.25 to 8.04 ms; means 7.34 and 8.37 ms; $n = 150$ per arm), a difference that is statistically clear (Mann-Whitney, $p < 10^{-15}$) on the build measured here, and it remains far inside the 50 ms interactivity threshold of RQ2. These absolute figures come from a different endpoint than Experiment 2 and are not directly comparable to it; only the paired difference transfers.

This cost does not reproduce on the current chaincode. Repeating the same paired toggle on sequences 10 and 11 (n=200 per arm) finds no detectable difference (-0.10 ms, 90 % CI $[-0.63, +0.42]$, $p = 0.75$), placing the $+0.78$ ms above outside the interval. The two runs differ in chaincode generation and world state, so they are not strictly contradictory, but only the newer figure describes the shipped build, for which the honest statement is that the freshness read's cost is undetectable and bounded above by $\approx$0.6 ms. A plausible mechanism, which we did not measure, is that the anchor key is now hot: the heartbeat rewrites it every 60 s and every `CheckAccess` reads it, so it stays resident in the state-database cache.

Security-availability trade-off. The bound is only as good as the anchor's freshness, and the gateway is also what refreshes it. It cannot move the anchor backwards — non-advancing heartbeats are refused — but it can stop advancing it, after which the reachable backdating window grows with the suppression interval. A staleness ceiling caps that window by refusing decisions taken against an anchor that has stopped advancing, at the cost of refusing honest decisions during a genuine ordering outage. Sweeping the ceiling and measuring when refusal begins gives:

Ceiling	**Availability window**	**Backdating bound**
30 s	31 s	151 s
60 s	59 s	179 s
120 s	121 s	241 s
0 (disabled)	no refusal within 200 s	bounded only by anchor staleness

Enforcement tracks the configured ceiling to within the 3 s polling granularity. The more useful observation is that the two columns are the same quantity read from opposite ends: every second of backdating tolerance purchased is a second of outage tolerance surrendered. The trade-off is a straight line of unit slope, and no setting improves both. The prototype ships with the ceiling disabled, preferring availability and accepting the weaker bound.

What this does and does not establish. The enforcement of the ceiling and the cost of the check are measured; the refusal logic is additionally covered by chaincode unit tests. The load-bearing claim — that a compromised gateway cannot commit a *backdated anchor* — is argued from endorsement rather than demonstrated: it depends on multiple peers independently validating the heartbeat, which the single-stub unit tests cannot exercise, and no backdated proposal was constructed against the deployment, since neither the peer CLI nor the gateway SDK exposes the proposal timestamp. Establishing it empirically requires a client that assembles and signs proposals directly, and is left to future work.

6.20. Experiment 18: Adversarial Database Mutation

The claim that motivates the whole architecture is that an adversary holding write access to the operational database cannot subvert authorization, because release is decided from committed ledger state rather than from the PostgreSQL access-control list. That claim is argued throughout Section 4.1; this experiment measures it. Each mutation is applied directly with `psql`, never through an application code path, and is reverted before the next case.

A forged grant does not release material. Inserting a `document_access` row granting read capability to a user who holds no ledger grant left the two stores disagreeing — the database reported the capability, `CheckAccess` reported `false` — and both the ciphertext and wrapped-key endpoints denied the request (HTTP 403), exactly as they had before the mutation. An attacker with full write access to the operational store gains nothing on the release path.

A deleted grant does not revoke. The mirror case is the more operationally consequential one. Deleting the row of a user who *does* hold a ledger grant did not stop ciphertext release: the ledger still authorized, and the download continued to succeed. The wrapped-key endpoint returned HTTP 403, because it is gated on the operational row rather than on chaincode. Database writes are therefore inert in both directions on the ciphertext path, which is the intended property, but the asymmetry is worth stating plainly: an operator who deletes a row and observes the wrapped-key denial may reasonably but wrongly conclude that access has been revoked. Revocation must go through the ledger path (Experiments 16 and 16b).

The organization fallback is confirmed by measurement. A member of the owning organization holding no grant in either store received the ciphertext (HTTP 200), because `CheckAccess` falls through to an ownership check on the

requesting organization. The wrapped key was refused, since no per-recipient grant exists. This bounds how far the first result generalizes: the ledger path is authoritative, but its policy is permissive within the owning organization, and the exposure is real rather than hypothetical. It is the same limitation recorded in Table 7 and in the answer to RQ1, now evidenced end to end rather than inferred from the chaincode.

Denials are anchored. Building this experiment surfaced a gap in the audit trail: a request refused by chaincode policy produced no audit record at all, so a defeated probe left no trace, and only successful reads and outage-driven denials were recorded. Policy denials are now emitted as `ACCESS_DENIED` events carrying the refusal reason and the requester's organization, and anchored to the ledger like grants; a document that previously accumulated no denial records across this sequence now accumulates three. The property that matters is not the count but the anchoring: a refused attempt is independently verifiable by consortium peers rather than resting on the honesty of the server that refused it, which is what makes the audit claim in Scenario S1 checkable.

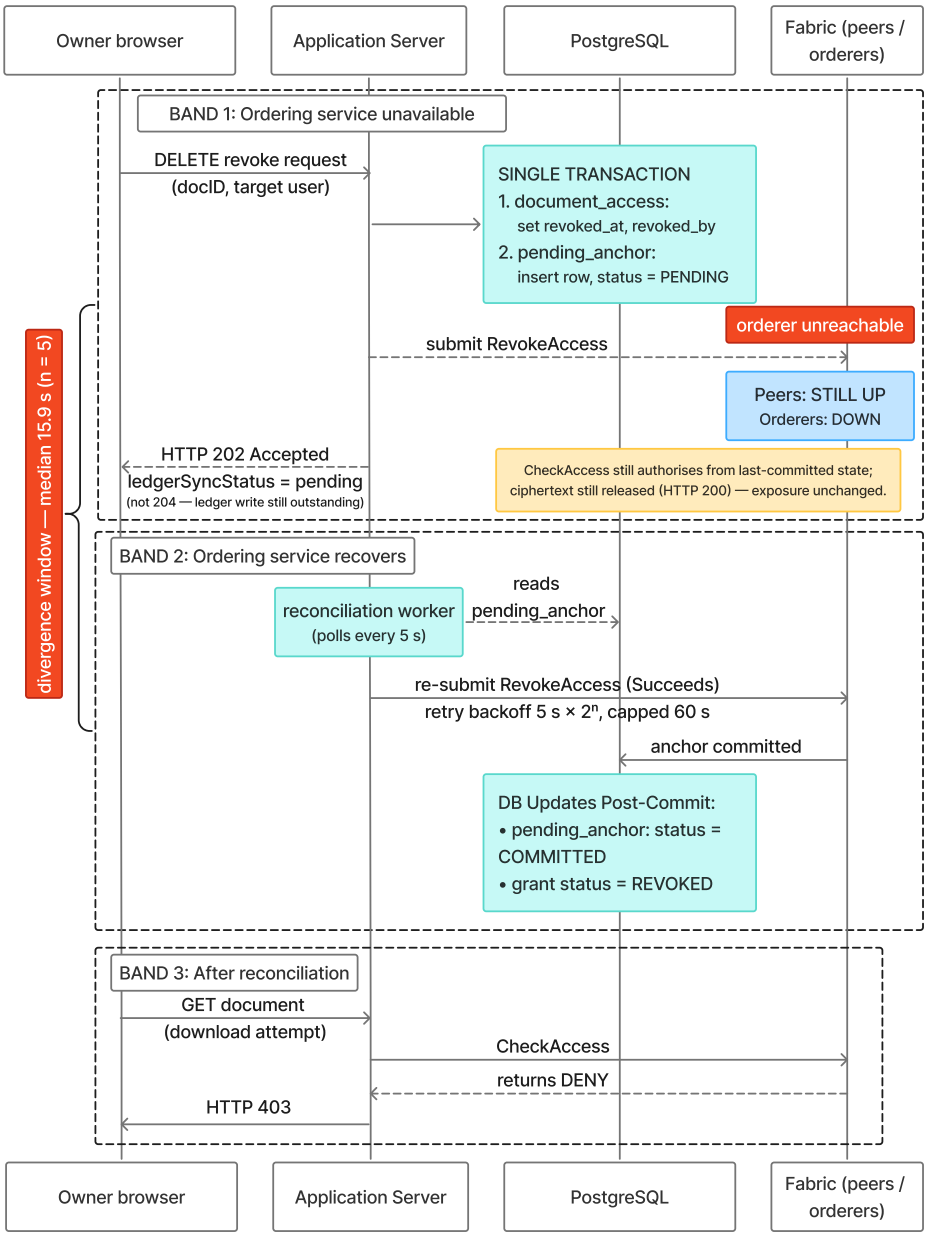


Figure 31: Durable re-anchoring of a revocation issued during an ordering-service outage. The operational revocation and its outbox row are written in a single database transaction, so the ledger write is deferred rather than lost, and the endpoint reports a pending ledger-sync status instead of unqualified success. A background worker re-submits the queued anchor once ordering recovers, after which the release path denies the revoked user. Note that the release path continues to authorize from last-committed state for the duration of the outage: the mechanism bounds how long the divergence outlives the outage, not the exposure during it.

7. Limitations and Future Work

7.1. Limitations

Each limitation is accompanied by a concrete mitigation path.

- **Coarse-grained intra-organization access control.** The prototype `CheckAccess` chaincode (Listing 1, lines 37–39) authorizes read access for any peer whose MSP organization matches the document owner's organization (`doc.OwnerOrg == userOrg`), without requiring an individual ACL entry. The service layer bounds what this authorization can release: the wrapped-key endpoint issues a document key only against the requester's own per-recipient grant entry, k_{enc} is never wrapped for ungranted principals, and document listings are likewise scoped to explicit grants. An ungranted member of `LawFirmA` who learns a document identifier can therefore retrieve the AES-256-GCM ciphertext of any `LawFirmA` document, but not its key or plaintext. The exposure is bounded to ciphertext, metadata, and document existence rather than plaintext: still a least-privilege violation within a firm and a harvest-now-decrypt-later surface. The framework design specifies per-user grants for all principals; closing this gap in production requires removing the organization-level fallback and enforcing explicit grants for intra-organization peers, consistent with a zero-trust access model.

- **Access-decision replay granularity.** The `CheckAccess` chaincode uses the Fabric transaction timestamp for grant-expiry evaluation, read via `ctx.GetStub().GetTxTimestamp()`, ensuring all endorsing peers assess the same proposal-embedded timestamp regardless of local clock offset. This guarantees determinism across peers, not trustworthiness of the clock: on an evaluate the timestamp originates in the client's proposal, so under the prototype's custodial submission a compromised gateway can supply a backdated value to resurrect an expired grant (residual-risk row, Table 6). Exact historical replay of individual access decisions would additionally require committing each access-decision event on-chain with its evaluated timestamp and result; this is identified as a production hardening item and deferred to future work. Expired-grant cleanup (as opposed to evaluation) follows a separate eventual-consistency model via the asynchronous `CleanExpiredTokens` job (Section 5.4).

7.1.1. Geographic Distribution and WAN Latency

Experiment 5 evaluated WAN conditions up to 150 ms RTT under two delay configurations: bridge-only delay at the HTTP layer, and bridge plus per-orderer-container `veth` delay, which also captures inter-orderer Raft AppendEntries latency.

Under bridge-only delay, throughput degraded by 11.0 % (68.2 to 60.8 TPS at 200 clients), and commit latency grew from 2,140 ms to 2,457 ms. When inter-orderer Raft traffic was also delayed, throughput degraded by 44.8 % (69.5 to 38.3 TPS), and commit latency grew from 2,141 ms to 3,588 ms. In both configurations, the 16.7 TPS synthetic steady-state baseline was cleared by at least 2.3×, and commit latency remained below the 5 s operational threshold.

Performance in globally distributed, multi-continent deployments has not been validated at production scale. Mitigations include reducing `BatchTimeout` (500 ms reached 193.0 TPS and 250 ms reached 180 TPS in Experiment 8, both upper bounds for the evaluated three-organization deployment) or placing orderer nodes with regional awareness. Hyperledger Fabric's modular ordering service supports such channel and placement choices [12, 86].

The WAN experiment also does not independently test orderer-node failover, multi-peer failure combinations, or recovery during concurrent production-like legal workloads. Likewise, the 16.7 TPS workload model is a synthetic capacity-planning assumption rather than a replay of real legal transaction logs. Future empirical work should replay anonymized legal document-management transaction logs and test peer failure, orderer failover, and recovery under geographically distributed load.

7.1.2. Crash-Fault Tolerance vs. Byzantine Fault Tolerance

The 3-node EtcdRaft ordering service is crash-fault tolerant only; Byzantine orderer behavior is out of scope, as analyzed in Section 4.1.5. For deployments requiring a Byzantine threat model, SmartBFT ordering (Fabric v3.0) is the production path: migration changes the ordering service and channel configuration but not the document-management chaincode [50, 51, 75].

7.1.3. Key Management and Browser Dependency

Three key management limitations arise. First, key rotation is owner-initiated: re-encryption of all affected document keys after a compromise requires the document owner's browser session. An interim production-compatible direction is proxy re-encryption (PRE), where an owner authorizes

a proxy to transform a wrapped document key from one recipient key to another without revealing the document key to the proxy [90, 91, 6]. This paper does not implement PRE; it is identified as a future key-rotation mechanism that would require a careful construction, threshold governance, and implementation-level security analysis. Second, private keys are stored in browser `localStorage` under PBKDF2-derived AES-256-GCM protection; key loss through browser data clearing is permanent without a backup procedure. Third, all cryptographic operations use the browser WebCrypto API [15], unavailable in headless or server-side contexts. The production resolution for both the second and third limitations is WebAuthn/FIDO2 or HSM-backed key protection, deployed in a manner consistent with SP 800-63-4 AAL2/AAL3 requirements [71, 72, 59, 70].

7.1.4. On-Chain Metadata Privacy

Document content is protected by AES-256-GCM and is never stored on the ledger. Metadata remains visible to channel members, including timestamps, invoking MSP identity, CIDs, and access-event sequences. A channel member can infer case activity patterns by observing transaction frequency and CID patterns over time, even without decrypting content.

Hyperledger Fabric PDCs restrict sensitive fields to authorized subsets of channel members [40, 92]. However, each collection hash is still written to the main ledger [78]. Possible mitigations include narrower channels or PDC scopes, metadata minimization, batching, cover traffic, private access protocols, and zero-knowledge proofs for selected policy checks [93].

The plaintext hash $\mathcal{H}_D$ is also stored in main channel state. Any peer holding a candidate document can therefore confirm its registration without access to the ciphertext or decryption key. This leak is intentional for the prototype's integrity-verification workflow, but production systems should move $\mathcal{H}_D$ into a Fabric Private Data Collection or replace it with a keyed commitment available only to authorized auditors.

7.1.5. Access-Graph Disclosure to Channel Members

The preceding discussion applies to metadata generally. One instance of it is specific enough to the litigation setting to warrant naming, because it is a disclosure that the architecture creates rather than merely fails to prevent.

Wrapped-key tokens and the document ACL are ordinary world-state values on the main channel, indexed by document and recipient. Every channel member can therefore enumerate, for every document, the exact

set of principals holding a key to it, together with the time each grant was issued and revoked. Document *content* remains encrypted throughout; what is exposed is the *access graph*.

In litigation that graph is itself sensitive, and often more immediately actionable than the documents. Granting a key to an outside expert discloses that the expert has been retained, potentially before any disclosure obligation has attached. The number and identity of reviewers on a matter signal staffing level, which correlates with a party's own assessment of exposure. The volume and timing of grants to an opposing firm reveal production scope and its pace. Bursts of grant activity align with case events — a filing deadline, a deposition — so even pseudonymous identifiers leak through timing. Where ethical walls are in force, the grant graph reveals precisely which matters a given lawyer has been screened away from, which is the opposite of what screening is meant to achieve.

This exposure is in direct tension with the paper's central design choice. Authorization is externally verifiable *because* the decision and the state it reads are on the ledger, visible to every channel member; the same visibility is what discloses the access graph. Moving wrapped-key tokens and the ACL into a Fabric Private Data Collection (Table 7) would confine them to a member subset, but it narrows verification to that same subset — a regulator outside the collection could no longer independently check an access decision — and it changes the endorsement requirements for `CheckAccess`. We therefore record this as an unresolved design tension rather than a straightforward hardening step. It is not addressed in the measured prototype, and the sizing implications of PDC migration are not evaluated in any experiment reported here.

7.1.6. Authentication Assurance Level

The prototype implements AAL1 under NIST SP 800-63-4 [71]: username/password with signed JSON Web Tokens [73]. AAL1 is adequate for the prototype evaluation of normal-path chaincode access checks and audit mechanisms, but it is insufficient for production legal consortium deployments. The authentication gateway is explicitly designed for AAL2 and AAL3 upgrade without chaincode modification.

7.1.7. Identity Database Public Key Integrity

The PostgreSQL identity table binding users to their public keys (`pk_ecies`, `pk_sign`) is not cryptographically protected the way the `audit_log` table is.

The resulting public-key substitution attack, its consequences, and the ledger-anchored-registration mitigation are analyzed as Scenario S3 (Section 4.1.4); it remains the highest-priority hardening item (Section 7.2).

7.1.8. IPFS Retrieval Timeout and Thread Management

The prototype does not enforce an explicit timeout on IPFS `GetFile` calls in the document download path. Under normal operation, the Fabric `CheckAccess` invocation completes before the IPFS retrieval begins. However, if the IPFS swarm experiences a node failure, disk fault, or network partition *after* chaincode authorization succeeds, the Spring Boot HTTP thread handling the request may block indefinitely waiting for the ciphertext. Under sustained load, this could exhaust the application server's thread pool, causing a cascading failure that degrades or disables endpoints unrelated to IPFS. The production mitigation is a circuit-breaker pattern with a hard timeout (e.g., 5 s) on all IPFS retrieval calls, returning HTTP 504 to the caller and logging an `IPFS_RETRIEVAL_TIMEOUT` audit event. This ensures IPFS unavailability degrades gracefully rather than propagating to a full service outage.

Experiment 9 validates fail-closed behavior only for the document download path (`downloadCiphertext()`). The write paths are *not* fail-closed: upload, grant, and the signing action's audit anchoring each catch Fabric submission failures, log a warning, and complete the operational write, prioritizing availability. During a ledger outage this produces operational-versus-ledger divergence, and the sharpest case is revocation—a revocation recorded in PostgreSQL while the stale on-chain grant remains chaincode-authorized continues to authorize the revoked user on the release path.

Revocation is the one write path where this divergence is now bounded rather than open-ended. `RevokeAccess` writes the operational revocation and a durable outbox row in a single database transaction; a background worker re-submits the queued anchor with capped exponential backoff until it commits, and the API returns HTTP 202 with an explicit pending status—rather than an unqualified success—while the anchor is outstanding. Experiment 16b measures the result against the identical outage geometry: the ledger reconverges automatically, with no operator action, in a median of 15.9 s ($n = 5$, all runs committing on the first retry), after which the release path correctly denies the revoked user. This addresses the permanence of the divergence, not the divergence itself: the window decomposes into outage duration plus reconciliation lag, and only the second term is under

the system's control. For the duration of the outage the exposure measured in Experiment 16 is unchanged—ciphertext is still released to the revoked user, bounded to ciphertext because the wrapped-key endpoint is gated by the operational access row.

Two limitations remain. First, the outbox covers revocation only; grants issued during an outage are still anchored on a best-effort basis. Second, making reads fail closed under an orderer-only outage would convert every ordering outage into a total read outage, which is not a favorable trade. The principled alternative—refusing to authorize from world state older than a threshold relative to an ordered time reference—is implemented as the staleness ceiling of Experiment 17, which is the same mechanism that bounds clock trust for grant expiry. Its cost is now measured rather than conjectured, and it is not free: the ceiling trades one second of outage tolerance for each second of exposure it removes, which is why the prototype ships with it disabled. Choosing a non-zero ceiling is a deployment decision about which failure is less acceptable, not a strict improvement.

7.1.9. GDPR and Regulatory Compliance

The framework presents a structural tension with the European General Data Protection Regulation (GDPR) [57]. Article 17 grants data subjects a right to erasure, requiring personal data to be deleted upon request. Hyperledger Fabric's append-only ledger provides tamper-evidence precisely because committed transactions cannot be deleted.

The proposed framework partially resolves this tension through design decomposition: document ciphertext is stored off-chain in IPFS and can be unpinned and garbage-collected, while the ledger retains only SHA-256 hashes, CIDs, timestamps, and MSP identity references [42, 43]. Whether those residual values constitute personal data under GDPR Article 4(1) [57] is context-dependent and legally contested: data-protection analysis often treats cryptographic hashes of personal data as *pseudonymised* rather than anonymous, especially when linked to persistent institutional or user identities [79, 78]. Article 17 compliance is therefore not fully solved by the prototype: production use in jurisdictions applying GDPR erasure obligations requires a deployment-specific legal basis and governance process, together with technical hardening such as metadata minimization, narrower channels, private data collections, off-chain redaction workflows, or redactable-ledger mechanisms where legally required. Cross-jurisdiction data-residency obligations must likewise be resolved at deployment time, since legal exposure

depends on where IPFS pinning nodes, Fabric peers, and orderers are operated.

Accordingly, the framework treats the ledger as evidentiary rather than dispositive: the consortium agreement, not the chaincode, remains the legal instrument governing access and erasure obligations. If on-chain metadata is treated as personal data in a deployment jurisdiction, full compliance may require a legal determination specific to the consortium or a technical mechanism for selective record modification under governance. Redactable-blockchain mechanisms based on chameleon hashes are a possible research direction, but adopting them in Fabric would require modifying the block-hash construction and defining threshold governance for trapdoor use [90, 94]. Without such advanced cryptographic deletion methodologies or explicit off-chain zero-knowledge verification patterns, the system may face significant compliance barriers to deployment in strict European jurisdictions handling personally identifiable legal records; this risk assessment is consistent with EDPB guidance on pseudonymization and blockchain data governance [95, 96].

7.1.10. Integration with Legacy Legal Systems

The prototype operates as a self-contained system. Production deployment in an active legal consortium requires connectors to existing practice management software, court filing platforms, and document repositories. The following outlines the four architectural concerns a production integration must address.

(a) API surface and capability exposure. The Spring Boot REST gateway exposes four capability groups relevant to legacy connectors:

- document registration: `POST /documents/upload`;
- document retrieval: `GET /documents/{id}/download`;
- access grant and revocation: `POST /documents/{id}/grant` and `POST /documents/{id}/revoke`;
- audit trail query: `GET /api/audit`.

A legacy system with read-only audit requirements needs only the last endpoint and does not interact with the Fabric layer directly [81].

(b) Identity boundary: OAuth2 tokens to Fabric X.509 identities. Legacy practice management systems typically issue OAuth2 bearer tokens for authenticated sessions. At the integration boundary, a token exchange service must map a validated OAuth2 identity to a Fabric X.509 certificate

issued by the consortium Fabric CA. In the prototype this mapping is handled under the shared administrative MSP identity; in production each legacy user identity must be enrolled with the Fabric CA and issued a certificate so that chaincode invocations are attributable to a specific principal [58, 73].

(c) Boundary security controls. The integration boundary must enforce: TLS mutual authentication (mTLS) between the legacy connector and the Spring Boot gateway to prevent credential interception; rate limiting on write endpoints to prevent batch flooding of the Raft orderer beyond its `BatchTimeout` capacity; and payload inspection to reject uploads that exceed the IPFS node's configured maximum object size before a Fabric transaction is attempted [6].

(d) Phased migration path. A four-phase migration is recommended. Phase 1: read-only audit access, in which the legacy system queries `GET /api/audit` to surface blockchain-verified audit records alongside its existing log. Phase 2: read-write with fallback, in which document registration and retrieval route through the gateway but the legacy system retains its own ACL as a parallel authority. Phase 2 is a pre-enforcement migration phase for non-Fabric-classified documents only; once a document is classified as Fabric-governed, its release path must not fall back to legacy ACL authority. During Phase 2, legacy ACLs operate in shadow or advisory mode alongside Fabric enforcement, not as an availability substitute. Phase 3: Fabric-governed ACL enforcement with fallback policy disabled, in which the `CheckAccess` chaincode becomes the access-control authority for protected document release and the legacy ACL is retired. Phase 4: full per-user X.509 enrollment, in which the shared administrative MSP identity is replaced by per-user certificates and hardware-backed signing keys. Each phase is independently deployable and requires no chaincode modification [77, 6].

7.2. Future Work

Three directions dominate the path to production. First, per-user Fabric CA enrollment with HSM-backed or WebAuthn/FIDO2 key storage [70, 58, 72] would replace the shared administrative MSP identity and the password-protected `localStorage` key store, closing the custodial-attribution gap identified in Section 5.1; the same infrastructure supports a consortium-governed key-recovery workflow with KMS/HSM-backed escrow, threshold administrative approval, and auditable re-wrapping of recovered keys, so that no single administrator gains unilateral access to user private keys. Second, formal verification of the `CheckAccess` and `RegisterDocument` chaincode functions

would provide machine-checked guarantees of access-control completeness and absence of chaincode-level bypasses under the modeled policy [97]. Third, privacy-preserving access verification, including zero-knowledge proofs for selected policy checks and batching or cover-traffic mechanisms to reduce timing and frequency leakage, would address the on-chain metadata exposure discussed in Section 7 [93, 40].

Further directions include regulator-facing compliance interfaces (read-only audit dashboards and joint endorsement policies such as `AND('LawFirmAMSP.peer', 'RegulatorMSP.peer')`), real-time anomaly detection over the low-latency audit feed [98], decentralized identity integration bridged to Fabric MSP enrollment [99], cross-network evidence verification via relay mechanisms or verifiable credentials [100], cross-version CID chaining for document-version provenance, formal incentive mechanisms for private-swarm pinning, and a comparative complexity and cost model spanning application-only, append-only database, audit-log, and on-path authorization architectures.

8. Discussion

This section interprets the experimental results in relation to the three research questions and clarifies the boundaries of the measured prototype. The framework combines Hyperledger Fabric, encrypted IPFS storage, and a REST-based application gateway. The work evaluates the practical effect of moving the document-release authorization boundary from application-only trust to ledger-verifiable access-control state on the normal release path, with fail-closed behavior empirically confirmed for the download path. The prototype still has explicit boundaries, including server-managed Fabric MSP credentials and a two-node private IPFS swarm, but it implements strict client-side encryption and, on the evaluated document-download path, a fail-closed security policy during Fabric unavailability.

RQ1 asked whether a permissioned blockchain architecture can provide ledger-verifiable access-control state to consortium peers under normal operation while enforcing a strict fail-closed policy on the evaluated document-download path during Fabric unavailability. The answer is affirmative, with two boundaries. Access-control *state*, namely grants, revocations, and document registrations, is independently verifiable by consortium peers. `CheckAccess` derives each download decision from that state at request time, although individual query results are not themselves committed as transactions. The evaluated decision is also answered by a single peer in the

prototype; independence from a single malicious peer on the read path is a production hardening item, consistent with the malicious-peer analysis in Section 4.1.2.

First, ledger transactions remain attributable to the prototype's server-managed MSP identity until per-user Fabric CA enrollment is implemented (Table 7). Second, per-user `CheckAccess` enforcement was demonstrated for cross-organization access. Intra-organization requests still authorize by organization membership at the ledger layer (an exposure bounded to ciphertext release, since document keys are issued only against per-recipient grants, and measured directly in Experiment 18: an organization member with no grant in either store receives ciphertext but is refused the wrapped key), and removing that fallback is the primary production hardening step.

Third, and specific to time-bounded access, the answer is affirmative for which grants exist but qualified for when they lapse. `CheckAccess` evaluates expiry against a timestamp that, on an evaluate, originates in the caller's proposal, and under custodial submission the caller is the gateway this architecture otherwise removes from the authorization path. Expiry is therefore deterministic across endorsers without being independent of the gateway's clock. The time anchor of Experiment 17 bounds the resulting exposure by committing an endorsed time reference the gateway cannot rewind, at a measured cost of 0.78 ms on the release path; enforcing a tighter bound in the face of a gateway that simply withholds heartbeats additionally requires a staleness ceiling, which trades one second of outage tolerance for each second of backdating tolerance. Expiry enforcement is thus bounded rather than removed from application trust, and that distinction is a property of custodial submission, not of the prototype's maturity: an evaluate is never ordered, so no ordering-service timestamp exists to substitute.

The dual-store audit design separates operational speed from independent verifiability. PostgreSQL provides low-latency operational audit queries (3.9 ms P50 for 1,000 events, Experiment 4). Fabric `GetHistoryForKey` provides ledger-verifiable history (135.5 ms P50 at a 107-entry document history, Experiment 7). The 107-entry history is shallow relative to production legal lifecycles, so deeper-history scaling remains future work.

Under the stated threat model, a malicious database administrator cannot silently rewrite Fabric history. A compromised IPFS node also cannot undetectably modify document ciphertext because $\mathcal{H}_D$ is anchored on-chain at registration. During Fabric peer outages, the evaluated download path denied all downloads and logged `FABRIC_OUTAGE_ACCESS_DENIED` events to

the operational PostgreSQL audit log. Anchoring those outage denials on Fabric would require queue-and-replay support.

RQ2 asked whether chaincode-based access evaluation adds acceptable overhead for interactive legal workflows. The read-path threshold is met. Fabric `CheckAccess` costs +4.4 ms over the PostgreSQL-only ACL baseline (10.99 vs. 6.44 ms P50, Mann-Whitney $p = 1.0 \times 10^{-31}$), a difference that is statistically clear but an order of magnitude inside the 50 ms threshold; a two one-sided test against that pre-specified margin confirms equivalence within it. The cost is the on-path ledger check itself rather than the time-anchor freshness read, which is not detectable on this build.

The write-latency SLO is also met in absolute terms: `RegisterDocument` measured 2,084 ms P50 against the 5 s threshold. The incremental overhead over a PostgreSQL-only write was not measured because the framework has no database-only registration path. Experiment 1 indicates that latency is orderer-dominated rather than database-dominated, with PostgreSQL at ≈25% of its write-capacity reference and CPU below 23%.

File-size experiments show that Fabric commit latency is independent of payload size because the ledger stores only metadata, hashes, and CIDs. WAN experiments further show that, even with 150 ms RTT applied to inter-orderer Raft traffic, throughput remains above the estimated workload and write latency remains below 5 s. The file-size experiments report server-side P50; client-side AES-256-GCM encryption adds approximately 44 ms P50 for a 50 MB input, as measured in Experiment 6.

RQ3 asked whether the framework sustains throughput sufficient for realistic legal workload peaks. The measured REST-gateway Fabric deployment sustains approximately 66–70 TPS under concurrent load up to 600 clients, about 4× above the estimated 16.7 TPS workload for a 1,000-user organization issuing one document action per minute per user. Reducing `BatchTimeout` to 500 ms produced the highest measured throughput, 193.0 TPS, under the evaluated three-organization deployment, an 11.6× margin without changing the application architecture.

Experiment 9 confirms strict fail-closed enforcement on the document-download path. Successful downloads fell to exactly zero for the full 45-second Fabric unavailability window, while HTTP 503 denials continued at approximately 30 per second on average. Every denied attempt was recorded in the operational PostgreSQL audit log, consistent with the outage-anchoring limitation noted under RQ1.

Normal operation resumed automatically after an approximately 20-second

recovery lag (circuit-breaker open-state completion plus gRPC reconnection). The write paths remain availability-first in the prototype (Section 7).

The evaluation also shows that the cryptographic layer is not the dominant performance cost. Using Node.js WebCrypto as a same-API fallback, PBKDF2-SHA256 at 600,000 iterations measured 100.44 ms P50, AES-GCM encryption of a 50 MB document measured 44.42 ms P50, and AES-GCM decryption measured 61.94 ms P50. ECDSA signing and verification, and hybrid-wrap wrapping and unwrapping, were sub-millisecond in the measured environment. P-256 hybrid wrap is used primarily for compact per-recipient key distribution: its 125-byte wrapped-key token is 51.2 % smaller than the 256-byte RSA-OAEP 2048 token it replaced, though larger than the ≈80-byte HPKE (RFC 9180) equivalent, mostly for encoding reasons.

9. Conclusion

This paper presented a hybrid framework for legal document management that combines Hyperledger Fabric, encrypted IPFS storage, and a REST-based application gateway. The core contribution is an empirical systems baseline for moving legal-document release decisions from application-only authorization to Fabric-backed `CheckAccess` evaluation on the normal download path. The work does not claim to introduce a new access-control primitive, a Byzantine-fault-tolerant ordering protocol, or a production-ready legal deployment.

The results show that this design can provide ledger-verifiable access-control state while maintaining practical performance for interactive legal workflows. Fabric access checks cost ≈4.4 ms more than the PostgreSQL-only ACL baseline on the evaluated read path — a measurable premium, an order of magnitude inside the interactivity budget — and the REST-gateway Fabric deployment sustained throughput above the estimated workload baseline. The fail-closed outage experiment further showed that successful downloads fell to zero during Fabric unavailability, confirming the evaluated download path's strict denial behavior.

These results substantiate the contributions set out in Section 1.3: a measured relocation of the enforcement boundary to ledger-verifiable access-control state, chaincode-enforced time-bounded grants and revocation, a dual-store audit design separating low-latency operational queries from ledger-verifiable history, and a sixteen-experiment evaluation spanning throughput to fail-closed outage behavior.

Table 7 documents the gaps between framework design and current implementation, bounding the measured prototype relative to the framework design. To the best of the authors' knowledge, none of the prior Fabric + IPFS legal-document systems in Table 1 report the latency cost of placing `CheckAccess` on the critical download path; the closest measurement is the 146–156 ms end-to-end retrieval authorization that RBAC-IPFS reports for generic file sharing under an emulated 50 ms wide-area network [20], which is complementary to, but not directly comparable with, the LAN-scale function-level isolation reported here. The measured read overhead (10.99 vs. 6.44 ms P50) shows that this architectural choice imposes a real but practically negligible interactive cost, roughly an order of magnitude inside the interactivity budget. It also establishes a reproducible baseline against which future on-path enforcement designs can be evaluated.

Future work will focus on per-user Fabric CA enrollment, HSM/WebAuthn-backed key protection, privacy-preserving access verification, formal verification of chaincode policy logic, stronger IPFS persistence governance, and compliance mechanisms for jurisdictions where append-only on-chain metadata raises erasure or data-protection concerns.

Appendix A. Implementation listings

This appendix collects the implementation listings referenced in the main text: browser-side encryption and key derivation, server-side signature verification, the fail-closed download path, and the audit-log trigger schema.

Listing 2: Browser AES-256-GCM document encryption. Fresh 256-bit key and 96-bit IV generated per document via `window.crypto.getRandomValues`. SHA-256 hash computed of plaintext *before* encryption. Server receives only IV-prepended ciphertext.

```
// frontend/src/lib/crypto.ts
export async function encryptDocument(
    file: ArrayBuffer
): Promise<EncryptedDocument> {
  const key = await subtle.generateKey(
      { name: 'AES-GCM', length: 256 },
      true, ['encrypt','decrypt'])
  const iv = crypto.getRandomValues(new Uint8Array(12))
  const [ciphertext, plainHashBuffer, rawKey] = await Promise.all([
    subtle.encrypt({ name: 'AES-GCM', iv }, key, file),
    subtle.digest('SHA-256', file),  // Plaintext hash
    subtle.exportKey('raw', key),
  ])
  // Hash the resulting ciphertext to bind the IPFS payload
  const ivAndCiphertext = concat(iv, ciphertext)
  const cipherHashBuffer = await subtle.digest('SHA-256', ivAndCiphertext)
```

```
  return {
    ciphertextB64: bytesToBase64(new Uint8Array(ciphertext)),
    ivB64:         bytesToBase64(iv),
    plainHashB64:  bytesToBase64(new Uint8Array(plainHashBuffer)),
    cipherHashB64: bytesToBase64(new Uint8Array(cipherHashBuffer)),
    keyB64:        bytesToBase64(new Uint8Array(rawKey)),
  }
}
```

Listing 3: PBKDF2-SHA256 key derivation at 600,000 iterations (OWASP password-storage guidance; exceeds the NIST SP 800-132 minimum work factor). Derives the AES-256-GCM wrapping key protecting both hybrid-wrap and ECDSA private keys in `localStorage`.

```
// frontend/src/lib/crypto.ts
const PBKDF2_ITERATIONS = 600000
export async function derivePbkdf2Key(
    password: string, saltBase64: string
): Promise<CryptoKey> {
  const enc  = new TextEncoder()
  const salt = base64ToBytes(saltBase64)
  const base = await subtle.importKey(
      'raw', enc.encode(password),
      'PBKDF2', false, ['deriveKey'])
  return subtle.deriveKey(
    { name: 'PBKDF2', hash: 'SHA-256',
      salt: salt.buffer,
      iterations: PBKDF2_ITERATIONS },
    base,
    { name: 'AES-GCM', length: 256 },
    false, ['encrypt','decrypt'])
}
```

Listing 4: Server-side ECDSA P-256 verification. `SHA256withECDSAinP1363Format` natively consumes WebCrypto's raw $r\|s$ output without format conversion.

```
// EcdsaVerifier.java
public static boolean verify(
        String compositePayloadB64,
    String signatureB64,
    String publicKeyJwkJson) {
  try {
    byte[] compositePayload =
        Base64.getDecoder().decode(compositePayloadB64);
    byte[] sig =
        Base64.getDecoder().decode(signatureB64);
    PublicKey pub =
        parseJwkPublicKey(publicKeyJwkJson);
    Signature s = Signature.getInstance(
        "SHA256withECDSAinP1363Format");
    s.initVerify(pub);
    // P = H_D (32 B) || H_C (32 B) || UTF-8(id_D)
    // Prototype payload: content hashes + doc ID.
    // Owner, timestamp, filename not signed in
    // prototype; full metadata binding is future work.
```

```
    s.update(compositePayload);
    return s.verify(sig);
  } catch (Exception e) {
    log.warn("ECDSA verification error: {}",
        e.getMessage());
    return false;
  }
}
```

Listing 5: Strict fail-closed ACL in the document download path. Layer 1: Spring Security `@PreAuthorize`. Layer 2: `CheckAccess` chaincode. If Fabric is unreachable, the system strictly denies access and logs `FABRIC_OUTAGE_ACCESS_DENIED`.

```
// DocumentService.java - downloadCiphertext()
String requesterMspId = requester.getFirm() == null
    ? null
    : requester.getFirm().getMspId();
if (requesterMspId == null || requesterMspId.isBlank()) {
  throw new AccessDeniedException(
      "Requester has no firm/MSP binding.");
}
boolean allowed;
try {
  allowed = fabricGatewayService.checkAccess(
      docId.toString(),
      requester.getId().toString(),
      requesterMspId);
} catch (FabricException e) {
  auditService.log("FABRIC_OUTAGE_ACCESS_DENIED",
      requester.getId(),
      "DOCUMENT", docId.toString(), null,
      "{\"reason\":\"" + e.getMessage() + "\"}");
  throw new ServiceUnavailableException(
      "Fabric network unreachable. Access strictly denied.");
}
if (!allowed) throw new AccessDeniedException(
    "Access denied by chaincode.");
```

Listing 6: INSERT-only enforcement on `audit_log`. Row-level triggers block UPDATE and DELETE; a statement-level trigger blocks TRUNCATE (which bypasses row-level triggers in PostgreSQL). TRUNCATE privilege is also revoked from application roles.

```
-- 001-initial-schema.sql
CREATE OR REPLACE FUNCTION
    audit_log_prevent_modification()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'UPDATE' THEN
    RAISE EXCEPTION
      'audit_log is append-only: UPDATE not permitted';
  ELSIF TG_OP = 'DELETE' THEN
    RAISE EXCEPTION
      'audit_log is append-only: DELETE not permitted';
  END IF;
```

```
  RETURN NULL;
END; $$;

CREATE TRIGGER audit_log_no_modify
  BEFORE UPDATE OR DELETE ON audit_log
  FOR EACH ROW
  EXECUTE FUNCTION audit_log_prevent_modification();

-- TRUNCATE protection (statement-level;
-- row-level triggers cannot catch TRUNCATE)
CREATE OR REPLACE FUNCTION
    audit_log_prevent_truncate()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION
    'audit_log is append-only: TRUNCATE not permitted';
  RETURN NULL;
END; $$;

CREATE TRIGGER audit_log_no_truncate
  BEFORE TRUNCATE ON audit_log
  FOR EACH STATEMENT
  EXECUTE FUNCTION audit_log_prevent_truncate();

-- Revoke TRUNCATE from application roles
REVOKE TRUNCATE ON audit_log FROM app_role;
```

CRediT authorship contribution statement

Golam Bari Fardeen: Conceptualization, Methodology, Software, Investigation, Writing – original draft. **Namare Shakib Angkon:** Methodology, Software, Visualization, Writing – original draft, Writing – review & editing. **Syed Zayan Anwar:** Software, Validation, Formal analysis, Writing – review & editing. **Md. Masum Mizi:** Software, Resources, Visualization. **Shinthia Rahman:** Resources, Validation. **Md. Motaharul Islam:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Institute for Advanced Research Publication Grant of United International University, under Grant IAR 2025 Pub

093.

Data availability

The raw measurement data for all sixteen experiments (per-sample CSVs with per-run environment records), the load generators, topology generators, and analysis and plotting scripts that produce every figure and table in this paper, together with the measured-versus-manuscript reconciliation log, are archived as a citable replication package at `https://doi.org/10.5281/zenodo.XXXXXXX` (Zenodo), with the development history available at `https://github.com/meteorboyF/Fardeen_Codex_Dhaka_Meetup_Hackathon` (directories `results/` and `experiments/`, branch `prototype-fixes`; the post-campaign two-host validation dataset, including the client-side trial logs and RTT records, is on branch `linux-validation`).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to assist with language editing, manuscript structuring and LaTeX formatting. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] American Bar Association, 2023 cybersecurity tech report, accessed: Dec. 14, 2025 (Dec. 2023).
URL `https://www.americanbar.org/groups/law_practice/resources/tech-report/2023/2023-cybersecurity-techreport/`

[2] Verizon Business, 2025 data breach investigations report: Small- and Medium-Sized Business Snapshot, pDF. Accessed: Dec. 14, 2025 (May 2025).
URL `https://www.verizon.com/business/resources/infographics/2025-dbir-smb-snapshot.pdf`

[3] Information Commissioner's Office, Data security incident trends, [Online], accessed: Jun. 2026 (2024).
URL `https://ico.org.uk/action-weve-taken/data-security-incident-trends/`

[4] D. Yaga, P. Mell, N. Roby, K. Scarfone, Blockchain technology overview, NIST Internal Report 8202, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Oct. 2018). doi:10.6028/NIST.IR.8202.

[5] K. Kent, M. Souppaya, Guide to computer security log management, Special Publication 800-92, National Institute of Standards and Technology (Sep. 2006). doi:10.6028/NIST.SP.800-92.

[6] Joint Task Force, Security and privacy controls for information systems and organizations, NIST Special Publication 800-53 Revision 5 (Update 1), National Institute of Standards and Technology (NIST) (Sep. 2020). doi:10.6028/NIST.SP.800-53r5.

[7] D. Batista, A. L. Mangeth, I. Frajhof, P. H. Alves, R. Nasser, G. Robichez, G. M. Silva, F. P. d. Miranda, Exploring blockchain technology for chain of custody control in physical evidence: A systematic literature review, Journal of Risk and Financial Management 16 (8) (2023). doi:10.3390/jrfm16080360.

[8] Y. Cai, M. Feng, A time-bound data access control scheme based on attribute-based encryption, in: Proceedings of the 8th International Conference on Cyber Security and Information Engineering, ICCSIE '23, Association for Computing Machinery, 2023, pp. 52–56. doi:10.1145/3617184.3617781.

[9] A. Ahmad, M. Saad, A. Mohaisen, Secure and transparent audit logs with blockaudit, Journal of Network and Computer Applications 145 (2019) 102406. doi:10.1016/j.jnca.2019.102406.

[10] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, J. Yellick, Hyperledger fabric: A distributed operating system for permissioned blockchains, in: Proceedings of the Thirteenth EuroSys Conference (EuroSys '18), ACM, 2018, pp. 1–15. doi:10.1145/3190508.3190538.

[11] J. Benet, IPFS: Content addressed, versioned, P2P file system, arXiv preprint arXiv:1407.3561, accessed: Dec. 14, 2025 (2014).
URL https://arxiv.org/abs/1407.3561

[12] Hyperledger Fabric Documentation Contributors, The ordering service, hyperledger Fabric documentation (release-2.4). Licensed under CC BY 4.0. Revision 2a135189. Accessed: Dec. 25, 2025 (2022).
URL https://hyperledger-fabric.readthedocs.io/en/release-2.4/orderer/ordering_service.html

[13] D. Ongaro, J. Ousterhout, In search of an understandable consensus algorithm, in: 2014 USENIX Annual Technical Conference (USENIX ATC 14), USENIX Association, Philadelphia, PA, 2014, pp. 305–319.

[14] M. J. Dworkin, E. Barker, J. R. Nechvatal, J. Foti, L. E. Bassham, E. Roback, J. F. Dray Jr., Advanced encryption standard (AES), Federal Information Processing Standards Publication (FIPS) 197 (Nov. 2001). doi:10.6028/NIST.FIPS.197.

[15] H. Halpin, The W3C web cryptography API: Motivation and overview, in: Proceedings of the 23rd International Conference on World Wide Web, WWW '14 Companion, Association for Computing Machinery, 2014, pp. 959–964. doi:10.1145/2567948.2579224.

[16] L. Chen, D. Moody, A. Regenscheid, A. Robinson, Digital signature standard (DSS), Federal Information Processing Standards Publication (FIPS) 186-5 (Feb. 2023). doi:10.6028/NIST.FIPS.186-5.

[17] J. Nielsen, Usability Engineering, Morgan Kaufmann, San Francisco, CA, USA, 1993.

[18] R. B. Miller, Response time in man-computer conversational transactions, in: Proceedings of the December 9-11, 1968, Fall Joint Computer Conference, Part I, AFIPS '68 (Fall, part I), Association for Computing Machinery, New York, NY, USA, 1968, p. 267–277. doi:10.1145/1476589.1476628.
URL https://doi.org/10.1145/1476589.1476628

[19] International Legal Technology Association, ILTA technology survey 2023, Tech. rep., ILTA (2023).
URL https://www.iltanet.org/research/surveys

[20] N. Liu, S. Lu, F. Guan, W. Ren, A blockchain-based traceable access control scheme for IPFS, Blockchain: Research and ApplicationsIn press, journal pre-proof (2026). doi:10.1016/j.bcra.2026.100503.

[21] R. Mukta, S. Pal, K. Chowdhury, M. Hitchens, H.-y. Paik, S. S. Kanhere, Zero trust-driven access control delegation using blockchain, Blockchain: Research and Applications 7 (1) (2026) 100319. doi:10.1016/j.bcra.2025.100319.

[22] M. Steichen, B. Fiz, R. Norvill, W. Shbair, R. State, Blockchain-based, decentralized access control for IPFS, in: 2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), IEEE, 2018, pp. 1499–1506. doi:10.1109/Cybermatics_2018.2018.00253.

[23] J. Abou Jaoude, R. G. Saade, Blockchain applications–usage in different domains, IEEE Access 7 (2019) 45360–45381. doi:10.1109/ACCESS.2019.2902501.

[24] F. Casino, T. K. Dasaklis, C. Patsakis, A systematic literature review of blockchain-based applications: Current status, classification and open issues, Telematics and Informatics 36 (2019) 55–81. doi:10.1016/j.tele.2018.11.006.

[25] H. Alanzi, M. Alkhatib, Towards improving privacy and security of identity management systems using blockchain technology: A systematic review, Applied Sciences 12 (2022) 12415. doi:10.3390/app122312415.

[26] H. Precht, J. Hüllmann, J. Marx Gómez, Paperless everything: A systematic literature review for the design of blockchain-based document management systems, Distributed Ledger Technologies: Research and Practice 5 (2) (Jun. 2026). doi:10.1145/3737296.

[27] S. Liu, Q. Zheng, A study of a blockchain-based judicial evidence preservation scheme, Blockchain: Research and Applications 5 (2) (2024) 100192. doi:10.1016/j.bcra.2024.100192.

[28] D. Kim, S.-Y. Ihm, Y. Son, Two-level blockchain system for digital crime evidence management, Sensors 21 (9) (2021). doi:10.3390/s21093051.

[29] B. Onyeashie, M. Abubakar, P. Leimich, S. Mckeown, G. Russell, Privacy-preserving and scalable digital evidence management: A hyperledger fabric architecture with growth projections for law enforcement,

in: 2025 International Conference on New Trends in Computing Sciences (ICTCS), 2025, pp. 105–112. doi:10.1109/ICTCS65341.2025.10989348.

[30] A. Verma, P. Bhattacharya, D. Saraswat, S. Tanwar, NyaYa: Blockchain-based electronic law record management scheme for judicial investigations, Journal of Information Security and Applications 63 (2021) 103025. doi:10.1016/j.jisa.2021.103025.

[31] J. Hanafi, Y. Prayudi, A. Luthfi, IPFSChain: Interplanetary file system and hyperledger fabric collaboration for chain of custody and digital evidence management, International Journal of Computer Applications 183 (2021) 24–31. doi:10.5120/ijca2021921808.

[32] J. Guo, K. Zhao, Z. Liang, K. Min, Efficient and secure emr storage and sharing scheme based on hyperledger fabric and ipfs, Applied Sciences 14 (2024) 5005. doi:10.3390/app14125005.

[33] J. Chen, C. Zhang, Y. Yan, Y. Liu, FileWallet: A file management system based on IPFS and Hyperledger Fabric, Computer Modeling in Engineering & Sciences 130 (2) (2022) 949–966. doi:10.32604/cmes.2022.017516.

[34] M. S. Peelam, V. Chamola, A. K. Sharma, B. K. Chaurasia, Decentralized trust: NFT and blockchain-enabled evidence system using fog computing, Blockchain: Research and Applications 7 (1) (2026) 100321. doi:10.1016/j.bcra.2025.100321.

[35] Z. Notash, S. Jamali, R. Fotohi, An adaptive blockchain-enabled access control framework for secure and privacy-preserving e-health data sharing, Blockchain: Research and ApplicationsIn press, journal pre-proof (2026). doi:10.1016/j.bcra.2026.100492.

[36] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, M. Lentczner, Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud, in: 21st Annual Network and Distributed System Security Symposium (NDSS 2014), Internet Society, San Diego, CA, 2014, nDSS proceedings are unpaginated. doi:10.14722/ndss.2014.23212.

[37] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, M. Wang, Zanzibar: Google's consistent, global authorization system, in: 2019 USENIX Annual Technical Conference (USENIX ATC '19), USENIX Association, Renton, WA, 2019, pp. 33–46.

[38] X. Qian, W. Wu, An efficient ciphertext policy attribute-based encryption scheme from lattices and its implementation, in: 2021 IEEE 6th International Conference on Computer and Communication Systems (ICCCS), 2021, pp. 732–742. doi:10.1109/ICCCS52626.2021.9449182.

[39] Hyperledger Fabric Documentation Contributors, Membership service provider (MSP), hyperledger Fabric documentation (latest). Licensed under CC BY 4.0. (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/membership/membership.html

[40] Hyperledger Fabric Documentation Contributors, Private data, hyperledger Fabric documentation (latest). Licensed under CC BY 4.0. (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/private-data/private-data.html

[41] Hyperledger Fabric Documentation Contributors, Endorsement policies, hyperledger Fabric documentation (release-2.4). Licensed under CC BY 4.0. (2022).
URL https://hyperledger-fabric.readthedocs.io/en/release-2.4/endorsement-policies.html

[42] IPFS Docs, Pin a file with IPFS, quickstart guide (Last updated: 2025-12-15) (Dec. 2025).
URL https://docs.ipfs.tech/quickstart/pin/

[43] IPFS Docs, Using garbage collection in Kubo, iPFS documentation page (Kubo tutorials) (Dec. 2025).
URL https://docs.ipfs.tech/how-to/kubo-garbage-collection/

[44] A. H. Lone, R. N. Mir, Forensic-chain: Blockchain based digital forensics chain of custody with PoC in Hyperledger Composer, Digital Investigation 28 (2019) 44–55. doi:10.1016/j.diin.2019.01.002.

[45] M. Cebe, E. Erdin, K. Akkaya, H. Aksu, S. Uluagac, Block4forensic: An integrated lightweight blockchain framework for forensics applications of connected vehicles, IEEE Communications Magazine 56 (10) (2018) 50–57. doi:10.1109/MCOM.2018.1800137.

[46] J. Hernando-Corrochano, R. Pastor-Vargas, R. Hernández-Berlinches, Trusted wills for digital assets using blockchain: a practical case, Blockchain: Research and Applications 6 (2025) 100289. doi:10.1016/j.bcra.2025.100289.

[47] M. Jlil, K. Jouti, C. Loqman, Blockchain and smart contracts based system for criminal record management, Indonesian Journal of Electrical Engineering and Computer Science 37 (1) (2025) 365–379. doi:10.11591/ijeecs.v37.i1.pp365-379.

[48] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, K. Scarfone, Guide to attribute based access control (ABAC) definition and considerations, NIST Special Publication 800-162, National Institute of Standards and Technology (2014). doi:10.6028/NIST.SP.800-162.

[49] U. Waheed, S. A. Khan, M. Masud, H. Jamshed, T. A. Jumani, N. U. R. Malik, Blockchain-based, dynamic attribute-based access control for smart home energy systems, Energies 18 (8) (2025). doi:10.3390/en18081973.

[50] Hyperledger Fabric Documentation Contributors, What's new in hyperledger fabric, accessed: Dec. 26, 2025 (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/whatsnew.html

[51] Hyperledger Fabric Documentation Contributors, The ordering service, hyperledger Fabric documentation. Accessed: Jun. 20, 2026 (2026).
URL https://hyperledger-fabric.readthedocs.io/en/latest/orderer/ordering_service.html

[52] European Parliament and Council of the European Union, Regulation (EU) 2023/1543 on European production orders and European preservation orders for electronic evidence in criminal proceedings, oJ L 191, 28.7.2023, pp. 118–180. Last accessed: 16 July 2026 (2023).
URL https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32023R1543

[53] EDRM, The EDRM model, version 2.0, electronic Discovery Reference Model, EDRM (edrm.net), CC BY 4.0. Last accessed: 16 July 2026 (2023).
URL https://edrm.net/edrm-model/

[54] American Bar Association Standing Committee on Ethics and Professional Responsibility, Formal opinion 477r: Securing communication of protected client information, revised May 22, 2017. Last accessed: 16 July 2026 (May 2017).
URL https://www.americanbar.org/products/ecd/chapter/348777154/

[55] Federal Rules of Evidence, Rule 902(13)–(14): Self-authenticating evidence — certified records generated by an electronic process or system; certified data copied from an electronic device, storage medium, or file, 2017 amendment with advisory committee notes. Last accessed: 16 July 2026 (2017).
URL https://www.law.cornell.edu/rules/fre/rule_902

[56] American Bar Association Standing Committee on Ethics and Professional Responsibility, Formal opinion 483: Lawyers' obligations after an electronic data breach or cyberattack, october 17, 2018. Last accessed: 16 July 2026 (October 2018).
URL https://www.americanbar.org/content/dam/aba/administrative/professional_responsibility/ethics-opinions/aba-formal-op-483.pdf

[57] European Parliament and Council of the European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation), Regulation 2016/679, Official Journal of the European

Union, oJ L 119, 4 May 2016, pp. 1–88. In force since 25 May 2018. Accessed: Jan. 2026 (Apr. 2016).
URL https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679

[58] Hyperledger Fabric CA Documentation Contributors, Welcome to hyperledger fabric CA (certificate authority), accessed: Dec. 26, 2025 (2024).
URL https://hyperledger-fabric-ca.readthedocs.io/en/latest/

[59] E. Barker, A. Roginsky, Transitioning the use of cryptographic algorithms and key lengths, NIST Special Publication 800-131A Revision 2, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Mar. 2019). doi:10.6028/NIST.SP.800-131Ar2.

[60] R. Sandhu, E. Coyne, H. Feinstein, C. Youman, Role-based access control models, Computer 29 (2) (1996) 38–47. doi:10.1109/2.485845.

[61] M. Dworkin, Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC, NIST Special Publication 800-38D, National Institute of Standards and Technology (NIST) (Nov. 2007). doi:10.6028/NIST.SP.800-38D.

[62] R. Barnes, K. Bhargavan, B. Lipp, C. A. Wood, Hybrid public key encryption, Request for Comments RFC 9180, Internet Engineering Task Force (Feb. 2022). doi:10.17487/RFC9180.

[63] IEEE Standards Association, IEEE standard specifications for public-key cryptography – amendment 1: Additional techniques, IEEE Standard 1363a-2004, Institute of Electrical and Electronics Engineers, defines ECIES (Elliptic Curve Integrated Encryption Scheme). Accessed: Jan. 2026 (2004). doi:10.1109/IEEESTD.2004.94612.

[64] A. Rundgren, B. Jordan, S. Erdtman, JSON canonicalization scheme (JCS), RFC 8785, IETF (2020).
URL https://www.rfc-editor.org/rfc/rfc8785

[65] OWASP Foundation, Password storage cheat sheet, https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (2024).

[66] M. Sönmez Turan, E. Barker, W. Burr, L. Chen, Recommendation for password-based key derivation: Part 1: Storage applications, NIST Special Publication 800-132, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Dec. 2010). doi:10.6028/NIST.SP.800-132.

[67] B. Kaliski, J. Jonsson, A. Rusch, PKCS #1: RSA cryptography specifications version 2.2, Request for Comments 8017, RFC Editor (Nov. 2016). doi:10.17487/RFC8017.

[68] K. O.-B. O. Agyekum, Q. Xia, E. B. Sifah, C. N. A. Cobblah, H. Xia, J. Gao, A proxy re-encryption approach to secure data sharing in the internet of things based on blockchain, IEEE Systems Journal 16 (1) (2022) 1685–1696. doi:10.1109/JSYST.2021.3076759.

[69] Y. Kim, S.-O. Lee, K. Ko, Secure outsourced blockchain-based medical data sharing system using proxy re-encryption, Applied Sciences 11 (2021) 9422. doi:10.3390/app11209422.

[70] M. Cooper, K. B. Schaffer, Security requirements for cryptographic modules, Federal Information Processing Standards Publication (FIPS) 140-3 (Mar. 2019). doi:10.6028/NIST.FIPS.140-3.

[71] D. Temoshok, J. Fenton, N. Lefkovitz, A. Regenscheid, J. Richer, P. Grassi, Digital identity guidelines, NIST Special Publication 800-63-4, National Institute of Standards and Technology (NIST), supersedes SP 800-63-3 (withdrawn 2025-08-01). Accessed: Jan. 2026 (Jul. 2025). doi:10.6028/NIST.SP.800-63-4.

[72] A. Kumar, J. Hodges, J. Jones, M. B. Jones, G. Mandyam, Web authentication: An api for accessing public key credentials –Level 3, Candidate recommendation snapshot, W3C (2026).
URL https://www.w3.org/TR/webauthn-3/

[73] M. B. Jones, J. Bradley, N. Sakimura, JSON Web Token (JWT), RFC 7519 (May 2015). doi:10.17487/RFC7519.

[74] A. Barger, Y. Manevich, H. Meir, Y. Tock, A byzantine fault-tolerant consensus library for hyperledger fabric, in: 2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), IEEE, 2021, pp. 1–9. doi:10.1109/ICBC51069.2021.9461099.

[75] Hyperledger Fabric Documentation Contributors, Configuring and operating a BFT ordering service, hyperledger Fabric documentation. Accessed: Jun. 20, 2026 (2026).
URL https://hyperledger-fabric.readthedocs.io/en/latest/bft_configuration.html

[76] P. Grassi, M. Garcia, J. Fenton, Digital identity guidelines, NIST Special Publication 800-63-3, National Institute of Standards and Technology (NIST), includes updates as of 2020-03-02. Withdrawn 2025-08-01; superseded by SP 800-63-4. Accessed: Dec. 20, 2025 (Jun. 2017). doi:10.6028/NIST.SP.800-63-3.

[77] Hyperledger Fabric Documentation Contributors, Fabric chaincode lifecycle, accessed: Dec. 26, 2025 (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/chaincode_lifecycle.html

[78] E. Politou, F. Casino, E. Alepis, C. Patsakis, Blockchain mutability: Challenges and proposed solutions, IEEE Transactions on Emerging Topics in Computing 9 (4) (2021) 1972–1986. doi:10.1109/TETC.2019.2949510.

[79] M. Finck, Blockchain and the general data protection regulation: Can distributed ledgers be squared with european data protection law?, Study PE 634.445, European Parliamentary Research Service (EPRS), European Parliament (Jul. 2019). doi:10.2861/535.

[80] Hyperledger Fabric Documentation Contributors, CouchDB as the state database, accessed: Dec. 26, 2025 (2022).
URL https://hyperledger-fabric.readthedocs.io/en/release-2.4/couchdb_as_state_database.html

[81] P. Webb, D. Syer, J. Long, S. Nicoll, R. Winch, A. Wilkinson, M. Halbritter, Spring boot reference documentation, version 3.2.x, vMware, Inc. Accessed: Jan. 2026 (2024).
URL https://docs.spring.io/spring-boot/docs/3.2.12/reference/html/

[82] The PostgreSQL Global Development Group, PostgreSQL 16 documentation: Trigger functions, accessed: Jan. 2026 (2024).

URL https://www.postgresql.org/docs/16/plpgsql-trigger.html

[83] O. Abdullah Lajam, T. Ahmed Helmy, Performance evaluation of IPFS in private networks, in: Proceedings of the 2021 4th International Conference on Data Storage and Data Engineering, DSDE '21, Association for Computing Machinery, 2021, pp. 77–84. doi:10.1145/3456146.3456159.

[84] Hyperledger Fabric Contributors, fabric-shim.ChaincodeStub class (hyperledger fabric contract api), aPI documentation (fabric-chaincode-node) (2025).
URL https://hyperledger.github.io/fabric-chaincode-node/main/api/fabric-shim.ChaincodeStub.html

[85] LF Decentralized Trust, Hyperledger caliper, project page (blockchain performance benchmarking tool). Accessed: Dec. 12, 2025 (2021).
URL https://www.hyperledger.org/use/caliper

[86] A. Woznica, M. Kędziora, Performance and scalability evaluation of a permissioned blockchain based on the hyperledger fabric, sawtooth and iroha, Computer Science and Information Systems 19 (2022) 659–678. doi:10.2298/csis210507002w.

[87] A. Thakur, V. Ranga, R. Agarwal, Workload dynamics implications in permissioned blockchain scalability and performance, Cluster Computing 27 (2024) 11569–11593. doi:10.1007/s10586-024-04550-z.

[88] T. Benson, A. Akella, D. A. Maltz, Network traffic characteristics of data centers in the wild, in: Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement, IMC '10, Association for Computing Machinery, New York, NY, USA, 2010, p. 267–280. doi:10.1145/1879141.1879175.
URL https://doi.org/10.1145/1879141.1879175

[89] P. Thakkar, S. Nathan, B. Viswanathan, Performance benchmarking and optimizing hyperledger fabric blockchain platform, in: 2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), IEEE, 2018, pp. 264–276. doi:10.1109/MASCOTS.2018.00034.

[90] G. Ateniese, B. Magri, D. Venturi, E. Andrade, Redactable blockchain – or – rewriting history in Bitcoin and friends, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2017, pp. 111–126. doi:10.1109/EuroSP.2017.23.

[91] E. Barker, Recommendation for key management: Part 1 – general, NIST Special Publication 800-57 Part 1 Revision 5, National Institute of Standards and Technology (NIST), Gaithersburg, MD (May 2020). doi:10.6028/NIST.SP.800-57pt1r5.

[92] Hyperledger Fabric Documentation Contributors, Channels, hyperledger Fabric documentation (latest). Licensed under CC BY 4.0. (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/channels.html

[93] J. Groth, On the size of pairing-based non-interactive arguments, in: Proceedings of the 35th Annual International Conference on Advances in Cryptology–EUROCRYPT 2016, Part II, Springer-Verlag, Berlin, Heidelberg, 2016, pp. 305–326. doi:10.1007/978-3-662-49896-5_11.

[94] T. Lyons, L. Courcelas, K. Timsit, Blockchain and the GDPR, Thematic report, European Union Blockchain Observatory and Forum (Oct. 2018).
URL https://blockchain-observatory.ec.europa.eu/document/download/2df0bb3c-61fd-4ac6-921b-ff50fc32aeb9_en

[95] European Data Protection Board, Guidelines 01/2025 on pseudonymisation, Guidelines 01/2025, European Data Protection Board, guidelines 01/2025. Draft for public consultation adopted 16 January 2025; consultation closed 14 March 2025; finalisation pending at time of submission (January 2025).
URL https://www.edpb.europa.eu/system/files/2025-01/edpb_guidelines_202501_pseudonymisation_en.pdf

[96] M. Finck, F. Pallas, They who must not be identified–distinguishing personal from non-personal data under the gdpr, International Data Privacy Law 10 (1) (2020) 11–36. doi:10.1093/idpl/ipz026.

[97] A. Permenev, D. Dimitrov, P. Tsankov, D. Drachsler-Cohen, M. Vechev, VerX: Safety verification of smart contracts, in: 2020 IEEE Symposium

on Security and Privacy (SP), IEEE, 2020, pp. 1661–1677. doi:10.1109/SP40000.2020.00024.

[98] S. Siddamsetti, Anomaly detection in blockchain using machine learning, Journal of Electrical Systems 20 (3) (2024) 619–634. doi:10.52783/jes.2988.

[99] M. Sporny, A. Guy, M. Sabadello, D. Reed, Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations, W3C Recommendation, accessed: Dec. 14, 2025 (Jul. 2022).
URL https://www.w3.org/TR/did-1.0/

[100] M. Sporny, D. Longley, D. Chadwick, Verifiable credentials data model v1.1, W3C Recommendation, accessed: Jan. 2026 (Mar. 2022).
URL https://www.w3.org/TR/vc-data-model/